

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Advanced Big Data Analytics

Assignment I

Smart Energy Grid Analytics

Ingestion, storage benchmarking, analysis, continuous aggregates, dashboards and forecasting for 4,000 smart meters with PostgreSQL, TimescaleDB, EMQX and Python

Group members

No.	Full name	Student ID
1	Patrick Niyonshuti	101420
2	Erneste Uwimpuhwe	101414
3	Gisele Uwibereyeho	101366

Source code and README on GitHub

`https://github.com/erneste-uwimpuhwe/Advanced-Big-Data-Analytics-Assignment-I-Smart-Energy-Grid-Analytics`

Lecturer: Temitope Oguntade

Issued 1 October 2026 | Due 15 October 2026, 11:59 PM CAT

Table of Contents

Introduction . . . **3**
- Aim and questions . . . 3
- Approach and tools . . . 3
- Main findings . . . 3
- How the report is organised . . . 4

1 Build the ingestion pipeline . . . **5**
- 1.1 Confirm the environment . . . 5
- 1.2 Create the database structures . . . 9
- 1.3 Capture and publish a pilot stream . . . 12

2 Capture and prepare the historical dataset . . . **16**
- 2.1 Capture and load the complete stream . . . 16
- 2.2 Investigate data quality . . . 19
- 2.3 Clean and preserve a consistent benchmark . . . 21

3 Compare chunk intervals . . . **23**
- 3.1 Create and verify three configurations . . . 24
- 3.2 Implement and time the five workloads . . . 25
- 3.3 Explain the chunk results . . . 29

4 Compare compression strategies . . . **30**
- 4.1 Create the two compressed configurations . . . 30
- 4.2 Measure storage and compression cost . . . 34
- 4.3 Repeat the query comparison . . . 35
- 4.4 Select a storage configuration . . . 39

5 Analyse demand, regions, reporting quality and abnormal consumption . . . **40**
- 5.1 Demand patterns over time . . . 40
- 5.2 Regions and customer types . . . 43
- 5.3 Reporting quality and late data . . . 45
- 5.4 Detecting abnormal consumption . . . 46
- 5.5 Recommendation to management . . . 48

6 Continuous aggregates and data freshness . . . **49**
- 6.1 Create the continuous aggregates . . . 49
- 6.2 Test data freshness . . . 52

7 Dashboards . . . **55**
- 7.1 Raw dashboard . . . 55
- 7.2 Aggregate dashboard . . . 57
- 7.3 Check that the two dashboards agree . . . 59
- 7.4 Compare refresh performance . . . 60

8 Forecasting hourly demand . . . **65**
- 8.1 Dataset, features and split . . . 65
- 8.2 Baseline: previous day, same hour . . . 67
- 8.3 Ridge regression and model selection . . . 67
- 8.4 Is the forecast useful? . . . 68

9 Submit the report and code . . . **72**
- 9.1 Organise the report . . . 72
- 9.2 Make the work reproducible . . . 72
- 9.3 Document contributions . . . 76

Introduction

An electricity utility needs to know how its customers use energy, how reliably the meter data arrives and whether next-day demand can be forecast well enough for planning. This report documents a complete analytics project for such a utility. It uses the readings of **4,000 smart meters** in **10 regions** (400 meters per region), recorded **every 15 minutes** from 1 to 28 September 2026 (CAT). That is a planned total of **10,752,000 readings**.

The readings come from the supplied generator `smart_meter_simulator.py`, which reproduces typical problems of real metering data: about 1 % of the readings never arrive, about 0.5 % arrive twice, about 2 % arrive late, and 40 abnormal consumption events are injected. The generator is used unchanged, and every result in this report was produced by our own programs and SQL scripts.

Aim and questions

The project answers five practical questions:

- How should the readings be ingested and stored so that they are complete, consistent and fast to query? (Sections 1 and 2)
- Which chunk interval and which compression strategy suit the utility's queries best? (Sections 3 and 4)
- What do the data show about demand, regions, customer types, reporting quality and abnormal consumption? (Section 5)
- How much faster can reporting be with stored summaries, and what do they cost in storage and freshness? (Sections 6 and 7)
- Can next-day hourly regional demand be forecast more accurately than with a simple baseline? (Section 8)

Approach and tools

The data are stored in **PostgreSQL 18.6 with TimescaleDB 2.30.2**, received through an **EMQX 6.3.1** MQTT broker (in Docker) and processed with **Python 3.13** (psycopg2, paho-mqtt, pandas, scikit-learn, matplotlib and Dash). All measurements were made on one laptop (Intel Core i5-8250U, 8 GB RAM, SSD, Windows 11). Every benchmark uses one warm-up execution followed by measured executions on the same machine, and the medians are reported as warm-cache measurements.

Main findings

- **Pipeline.** A pilot of 2,000 readings reached 2,000 at all four counting stages after a queue overflow in the first run (1,804 received) was diagnosed and corrected. The complete stream of 10,696,848 readings was then loaded, and the generator, CSV and database counts agreed.
- **Data quality.** Completeness is 98.9931 %. Removing 53,113 identical duplicate copies leaves 10,643,735 unique readings; the 108,265 missing readings are kept as gaps, never as zero consumption.
- **Storage.** One-day chunks were the best compromise. Time-ordered compression reduced the table from 1,343.7 MB to 438.7 MB (ratio 3.06) but makes single-meter lookups slower.
- **Demand.** The highest 15-minute demand was 7,993.7 kW on Monday 28 September at 13:00. Weekend demand is about 21 % lower, energy grows about 2 % per week, and regional differences follow the customer mix: industrial meters are 10.2 % of the meters but use 57.6 % of the energy.

- **Abnormal consumption.** A median-based rule detected all 40 injected events with no false flags.
- **Dashboards.** The Aggregate dashboard refreshes 7.1 times faster than the Raw dashboard (median 1,117.8 ms against 7,956.9 ms) for 18.03 MB of storage and a one-off refresh of 335.7 s.
- **Forecast.** The selected Ridge regression model has an MAE of 4.549 kWh against 42.708 kWh for the previous-day baseline, an improvement of 89.3 %.

How the report is organised

Section	Content
1	Environment, database structures and the MQTT pilot stream
2	Capture and bulk load of the full dataset; data quality investigation and cleaning
3	Comparison of three chunk intervals with five workloads
4	Comparison of two compression strategies: storage, compression cost and query speed
5	Demand patterns, regions and customer types, reporting quality, abnormal consumption and the management recommendation
6	Continuous aggregates and the freshness test
7	Raw and Aggregate dashboards: verification and refresh performance
8	Next-day hourly forecast: baseline, Ridge regression and evaluation
9	Reproducibility (repository, README, pilot re-run) and group contributions

Conventions. All dates and times are in CAT (Africa/Kigali, UTC+02:00). Figures are numbered by subsection (for example Figure 5.1-2) and each screenshot shows the executed command or action, its result, the taskbar and the clock. Energy is reported in kWh and power in kW. The programs, SQL scripts, benchmark timings and README are in the GitHub repository `https://github.com/erneste-uwimpuhwe/Advanced-Big-Data-Analytics-Assignment-I-Smart-Energy-Grid-Analytics` (see also Section 9.2).

1 Build the ingestion pipeline

1.1 Confirm the environment

All work was carried out on one laptop (host name **Erneste**). **EMQX 6.3.1** runs in a Docker container (`emqx/emqx:latest`, port 1883 for MQTT and 18083 for the dashboard). **PostgreSQL 18.6** runs natively as the Windows service `postgresql-x64-18`, and **TimescaleDB 2.30.2** was added with its Windows installer, whose timescaledb-tune step adjusted the memory and parallelism settings and switched JIT off.

One database, **smart_grid**, holds every table and view in this assignment. Drive C: had only about 5.8 GB free, so the database was created in a tablespace **d_space** located at `D:\pgdata`; all tables, chunks, indexes and aggregates are therefore stored on drive D:. TimescaleDB was enabled with `CREATE EXTENSION timescaledb`, and `ALTER DATABASE smart_grid SET timezone = 'Africa/Kigali'` makes every session use CAT. Daily buckets are aligned to CAT midnight by passing the time zone to `time_bucket('1 day', ts, 'Africa/Kigali')`, which returned `2026-10-02 00:00:00+02` for the test day.

Recorded environment

Item	Value
CPU model	Intel Core i5-8250U @ 1.60 GHz
Cores	4 physical cores, 8 logical processors
RAM	8,035 MB (about 7.8 GB)
Operating system	Microsoft Windows 11 Pro, version 10.0.26200 (build 26200)
Storage type	SSD: SK hynix BC901 HFS512GEJ9X108N, 512 GB; database stored on drive D:
Python	3.13.6 (virtual environment on D:\auca_a1\venv)
EMQX	6.3.1, in a Docker container (Docker Desktop with WSL 2 backend); no CPU or RAM limit set
PostgreSQL	18.6, x86_64-windows, 64-bit; runs natively as a Windows service (not in a container or VM); no CPU or RAM limit
TimescaleDB	2.30.2 (required: 2.10 or later)
shared_buffers	2008MB
work_mem	8035kB
max_parallel_workers_per_gather	4
jit	off
Database and storage	smart_grid in tablespace d_space (D:\pgdata); session time zone Africa/Kigali

Clock and execution times. The Windows time zone is (UTC+02:00) Harare, Pretoria, which is CAT, and the Windows Time service was started and synchronised with `time.windows.com`. Every Command Prompt used `prompt $D $T $P$G`, so each screenshot shows the CAT date and time of every command next to its result, together with the taskbar clock. Before recording, `w32tm /query /status` showed the source `time.windows.com`, stratum 5 (synchronised over NTP), leap indicator 0 and a last successful synchronisation at 17:07:21 CAT on 5 October 2026.

Broker. `docker ps` shows the container `emqx` up with ports 1883 and 18083 published; `emqx ctl status` reports node `emqx@172.17.0.2` version 6.3.1 as started, and `emqx ctl broker` confirms version 6.3.1 (the container clock runs in UTC: 15:29 UTC = 17:29 CAT). The dashboard Overview shows one running core node; its connection, topic and subscription counts are 0 whenever no pilot client is connected.

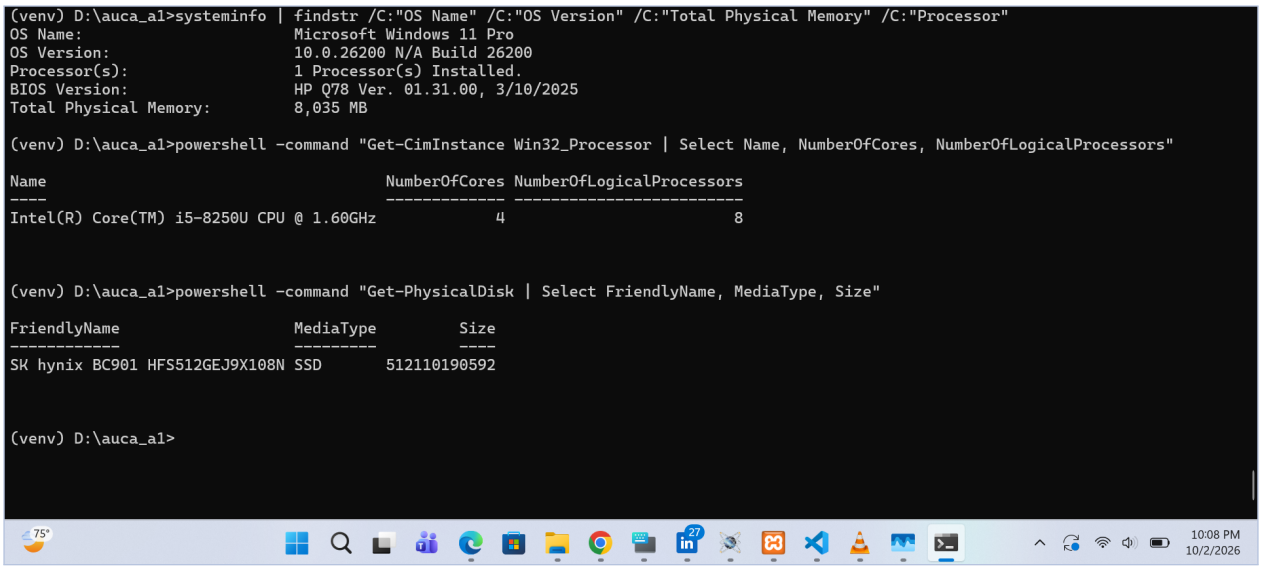


Figure 1.1-1: CPU model and cores, RAM, OS version and SSD storage

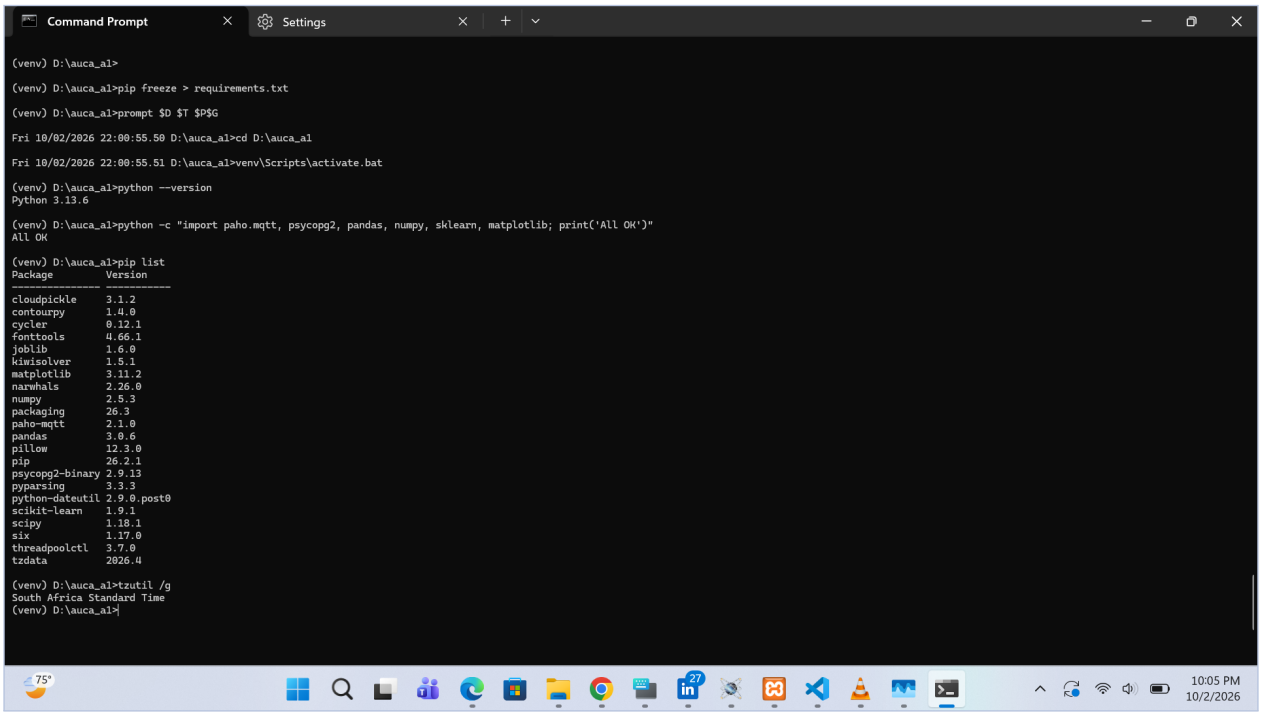


Figure 1.1-2: Python 3.13.6 virtual environment, required packages and CAT time zone

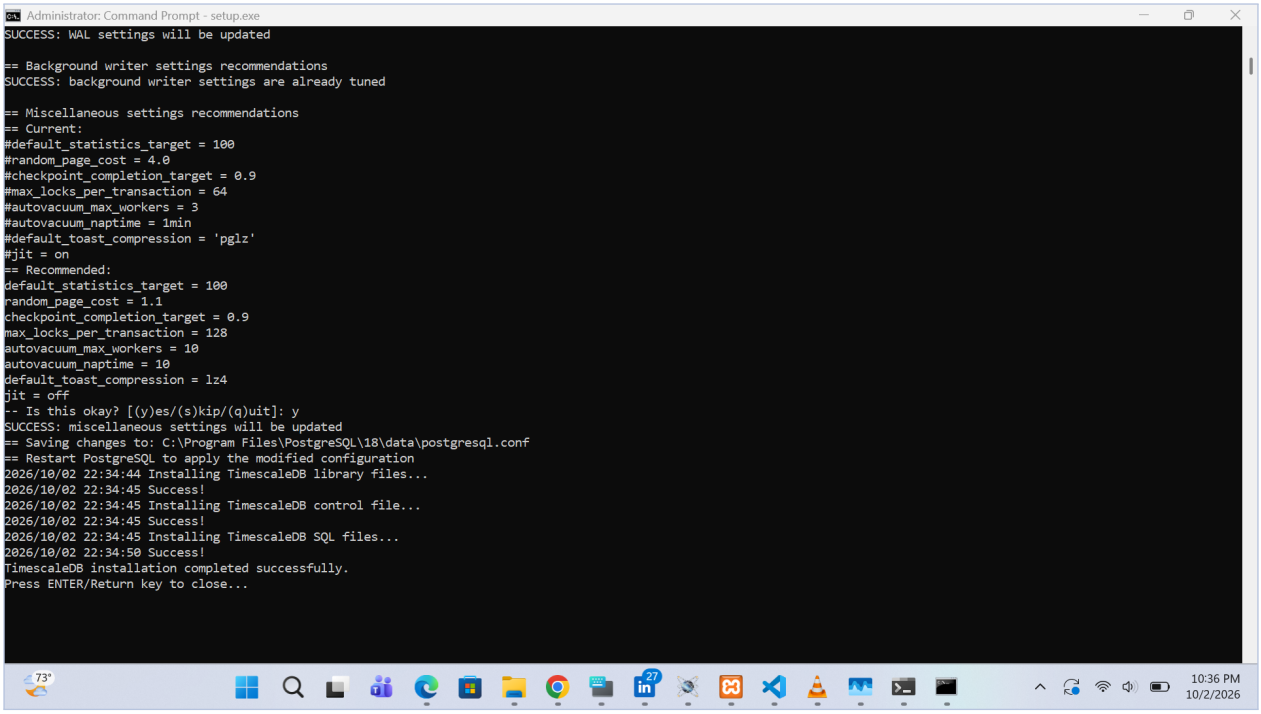


Figure 1.1-3: TimescaleDB installer and timescaledb-tune completed successfully

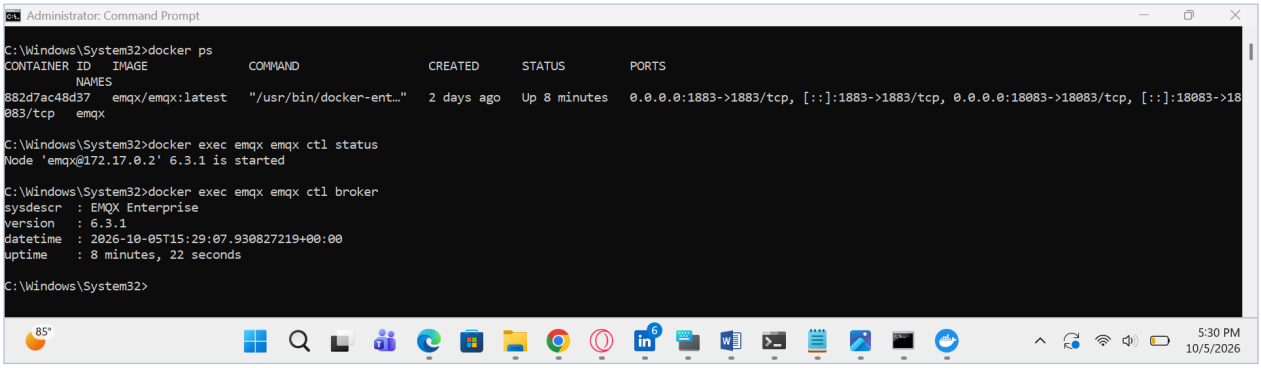


Figure 1.1-4: EMQX container running with ports 1883 and 18083; node emqx@172.17.0.2 version 6.3.1 started

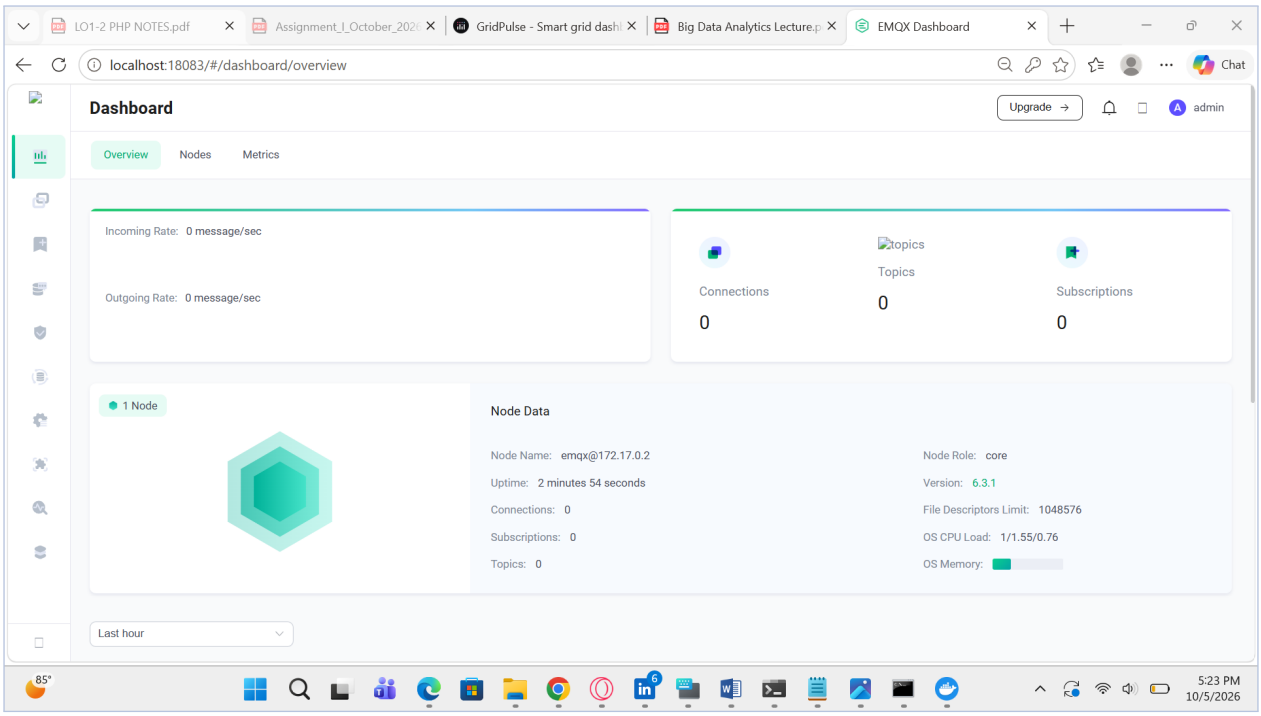


Figure 1.1-5: EMQX dashboard overview: one running core node, version 6.3.1 (no clients connected at that moment)

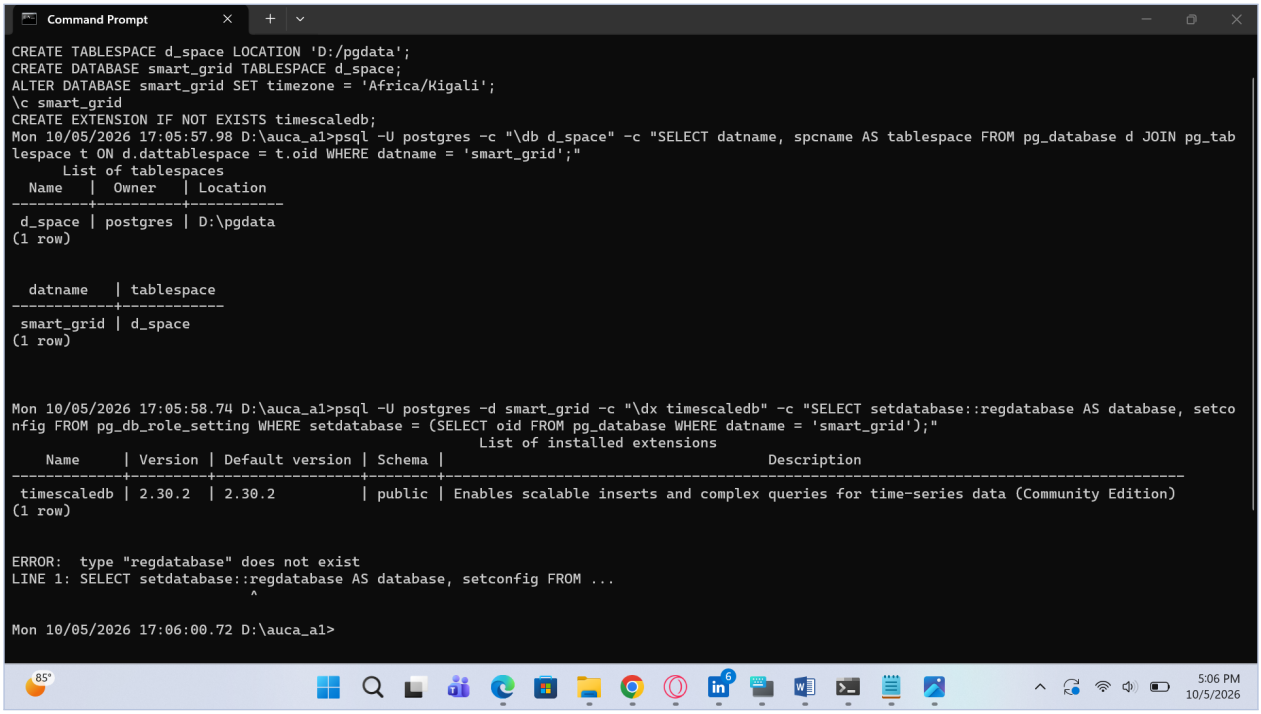


Figure 1.1-6: Setup script (tablespace d_space, database smart_grid, Africa/Kigali time zone, TimescaleDB extension) and proof: d_space at D:\pgdata, smart_grid in d_space, timescaledb 2.30.2 enabled

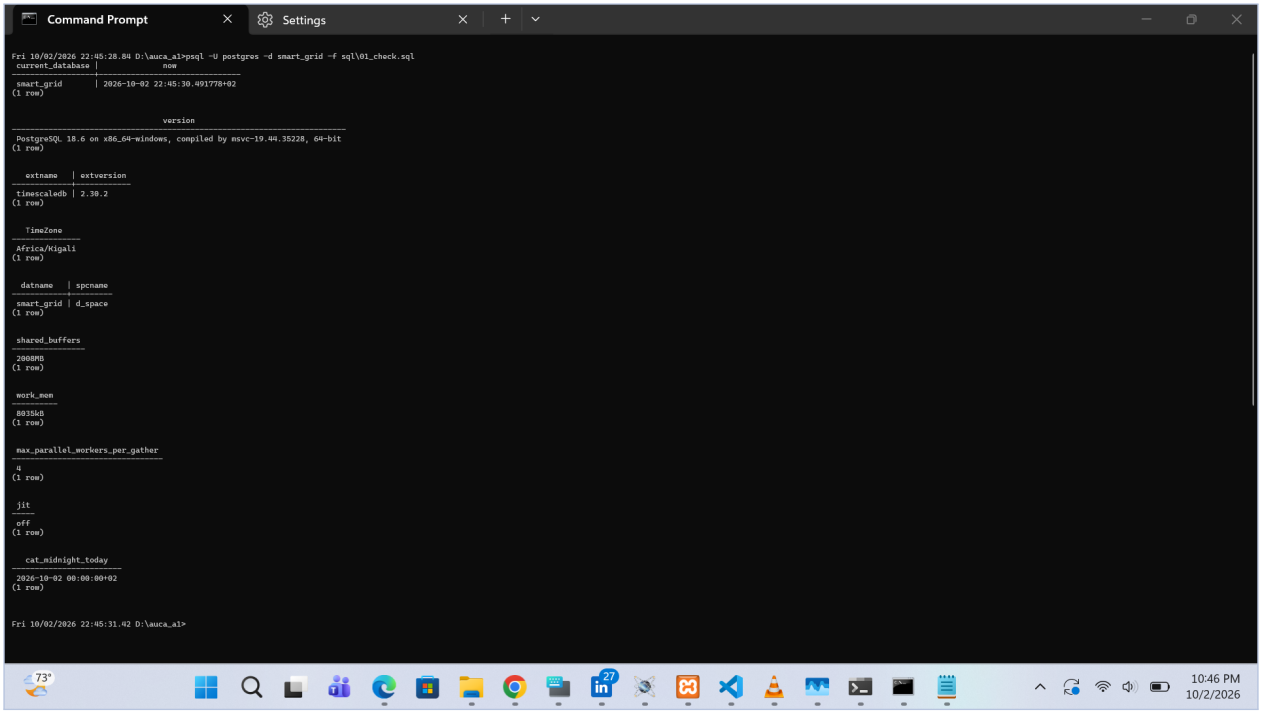


Figure 1.1-7: Connection to smart_grid, PostgreSQL 18.6, TimescaleDB 2.30.2, session time zone Africa/Kigali, tablespace d_space, the four settings and a CAT-midnight daily bucket

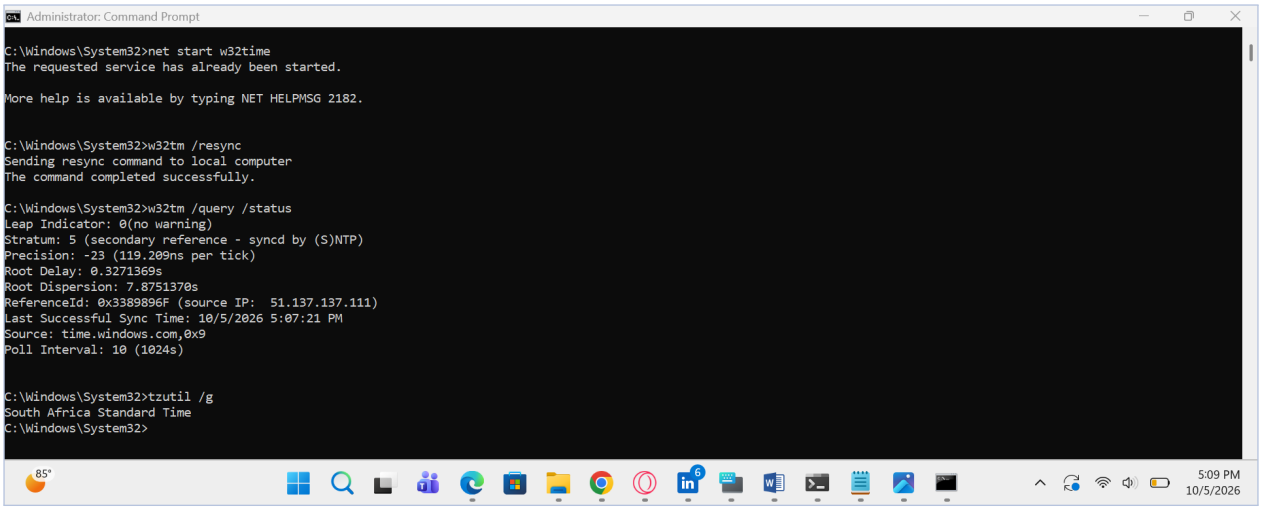


Figure 1.1-8: Windows clock synchronised with time.windows.com over NTP; time zone South Africa Standard Time (UTC+02:00, CAT)

1.2 Create the database structures

The structures were created with `sql/02_tables.sql`. **energy_live** is a regular PostgreSQL table and **energy_readings** is a TimescaleDB hypertable partitioned by event_time with **one-day chunks**, created with `create_hypertable('energy_readings', by_range('event_time', INTERVAL '1 day'), create_default_indexes => FALSE)`. Both tables have the eight required columns and no primary key or unique constraint, so duplicate readings are allowed at this stage. In the hypertable, event_time is NOT NULL because TimescaleDB requires a non-null partitioning column. Default indexes were switched off so that the 10.7-million-row bulk load is fast; identical indexes are created later on every hypertable (Section 3.1).

The **meters** table has meter_id as a BIGINT primary key, region and customer_type as TEXT, and base_power_kw and power_factor as DOUBLE PRECISION. The program `src/load_meters.py` imports the un-

changed generator, calls `meter_metadata()` and inserts each returned dictionary as one row (the table is truncated first so the program can be re-run safely). It received **4,000 dictionaries** and the table contains **4,000 rows**: 10 regions of 400 meters each, with 2,584 residential, 1,010 commercial and 406 industrial meters.

Data dictionary

energy_live (regular table) and **energy_readings** (hypertable, one-day chunks) have identical columns:

Column	PostgreSQL type	Meaning	Unit
meter_id	BIGINT	Ten-digit meter identifier	none
event_time	TIMESTAMPTZ	Start of the 15-minute measurement interval	timestamp (CAT)
received_time	TIMESTAMPTZ	Simulated arrival time (not the database insertion time)	timestamp (CAT)
power_kw	DOUBLE PRECISION	Average power over the interval	kW
voltage_v	DOUBLE PRECISION	Voltage	V
current_a	DOUBLE PRECISION	Current (power factor 0.95)	A
frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
energy_kwh	DOUBLE PRECISION	Energy used in the interval, about power_kw x 0.25 h	kWh

meters (regular table):

Column	PostgreSQL type	Meaning	Unit
meter_id	BIGINT (primary key)	Meter identifier	none
region	TEXT	Region 1 to Region 10	none
customer_type	TEXT	residential, commercial or industrial	none
base_power_kw	DOUBLE PRECISION	Baseline power level of the meter	kW
power_factor	DOUBLE PRECISION	Ratio of real to apparent power (0.95)	none (unitless)

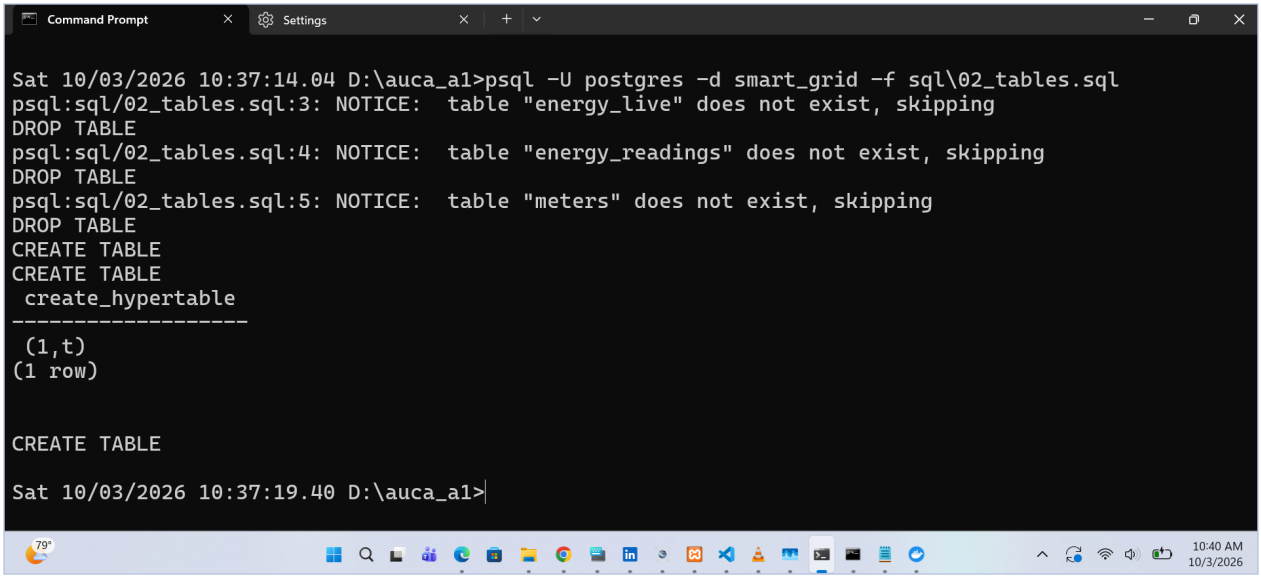


Figure 1.2-1: Creation of energy_live, the energy_readings hypertable and meters

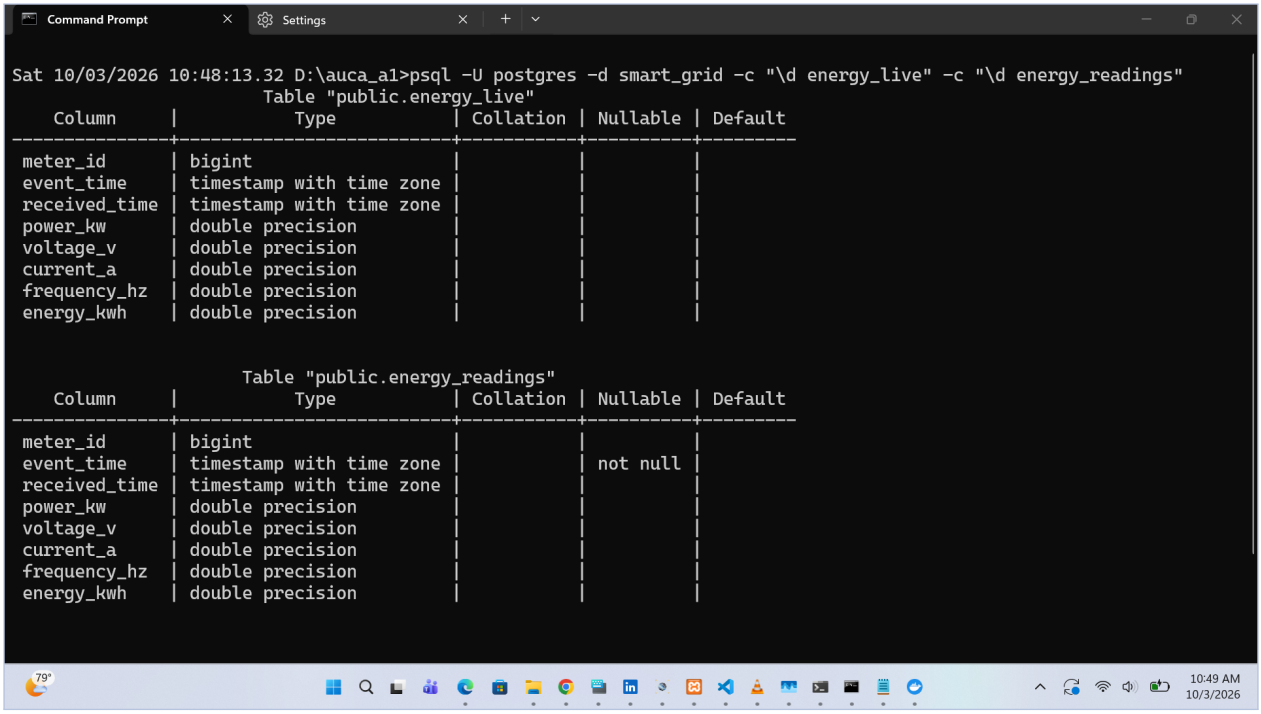


Figure 1.2-2: Column names and types of energy_live and energy_readings

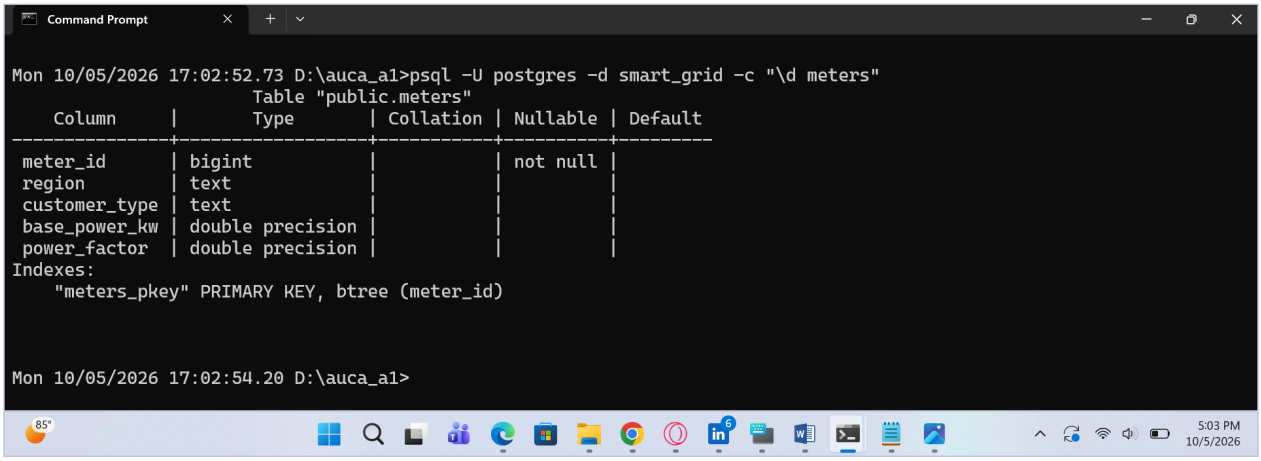


Figure 1.2-3: Column names, types and primary key of the meters table (\d meters)

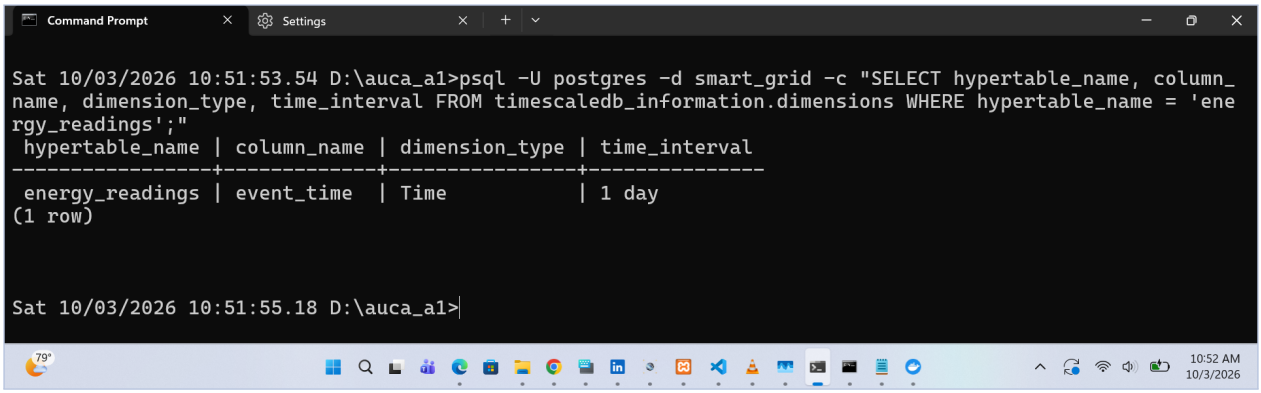


Figure 1.2-4: Hypertable chunk interval of energy_readings: 1 day on event_time

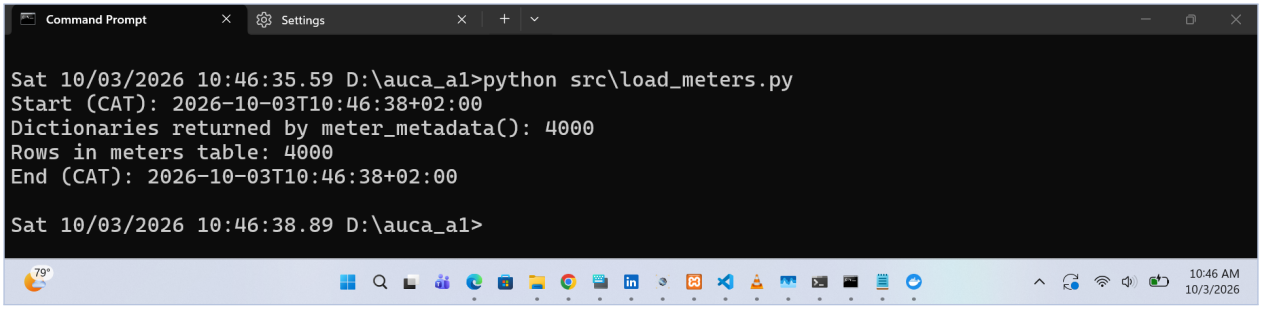


Figure 1.2-5: Python program loading meter metadata: 4,000 dictionaries and 4,000 rows

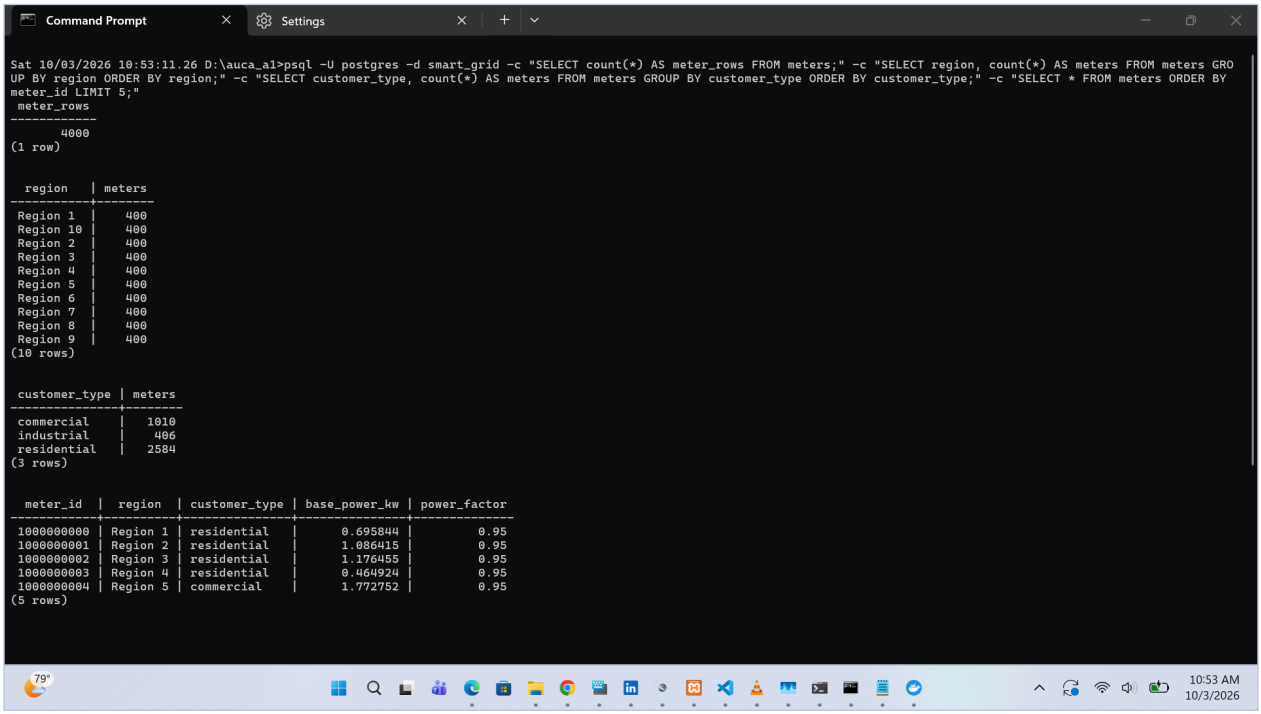


Figure 1.2-6: Query confirming 4,000 metadata rows, 400 meters per region and the customer types

1.3 Capture and publish a pilot stream

The publisher `src/pilot_publisher.py` imports the unchanged generator, takes the first 2,000 dictionaries from `iter_readings()`, converts each to JSON and publishes one message to `energy/meters/{meter_id}` with **QoS 1** and **retain = false**. The subscriber `src/pilot_subscriber.py` subscribes to `energy/meters/#` at QoS 1, parses each JSON message and inserts its eight fields into energy_live. The subscriber empties energy_live, confirms it holds 0 rows and subscribes before the publisher is started. Both programs log their counts and any losses, redeliveries, parsing errors or failed inserts to `results/pilot_*.log`. The pilot replays historical readings without waiting 15 minutes between records.

Pilot counts

Stage	Run 1 (fault)	Run 2 (after correction)
Dictionaries taken from the generator	2,000	**2,000**
Messages acknowledged by the broker (QoS 1 PUBACK)	2,000	**2,000**
Messages received by the subscriber	1,804	**2,000**
Rows in energy_live after all inserts were committed	1,804	**2,000**

Stage	Run 1 (fault)	Run 2 (after correction)
Difference from the 2,000 target	196 lost between broker and subscriber	0

Both runs include **6 deliberate duplicate copies** from the generator (1,994 unique meter and event_time pairs); they were counted as separate messages and rows, and a query on energy_live confirms 2,000 rows and 1,994 unique pairs.

Investigation of the run 1 difference

In the first run the broker acknowledged all 2,000 messages but the subscriber received only 1,804, a difference of **196**. The subscriber log showed no parse errors, no insert errors and no redelivered (DUP) messages, so nothing was lost inside the subscriber or the database. EMQX's broker metrics showed `messages.received = 2000`, `messages.delivered = 1804` and `delivery.dropped.queue_full = 196`. The publisher sent all 2,000 messages in about 0.06 s, while the subscriber inserted each message into PostgreSQL individually inside its MQTT callback. Messages therefore accumulated in the subscriber's broker-side session queue, and EMQX dropped the 196 that arrived while that queue was full.

Correction. The subscriber was changed so that its message callback only places each message in a local in-memory queue; a separate loop parses the messages and inserts them in batches of 200. The broker queue therefore no longer fills up. The pilot was repeated with an empty energy_live table and the subscriber started first: all four counts equalled **2,000**, the broker's `delivery.dropped.queue_full` counter stayed at 196 (no new drops; received 4,000 and delivered 3,804 across both runs), and the subscriber logged no redeliveries, parse errors or insert errors.

Run 1 evidence (fault)

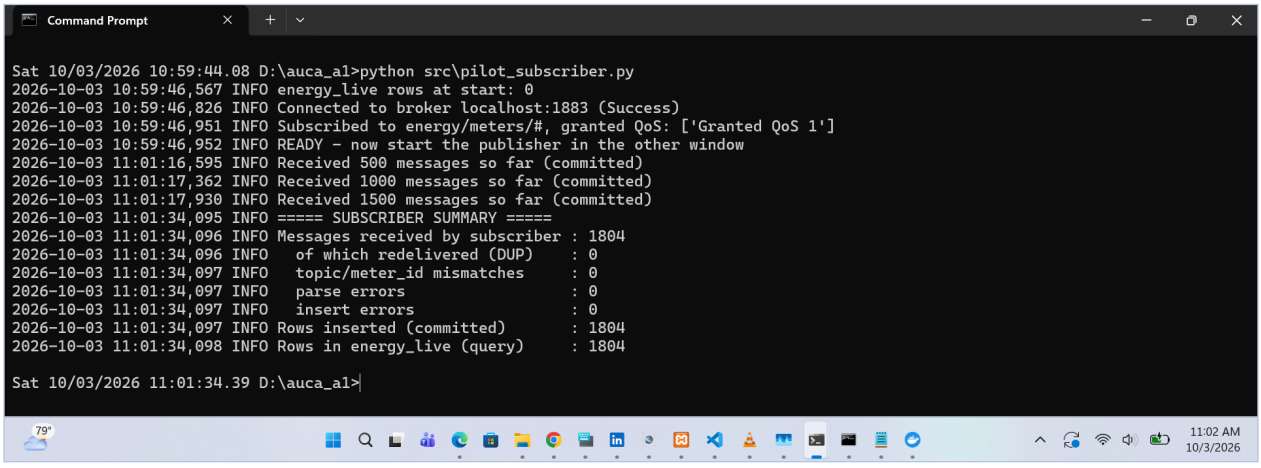


Figure 1.3-1: Run 1 subscriber summary: 1,804 messages received and 1,804 rows

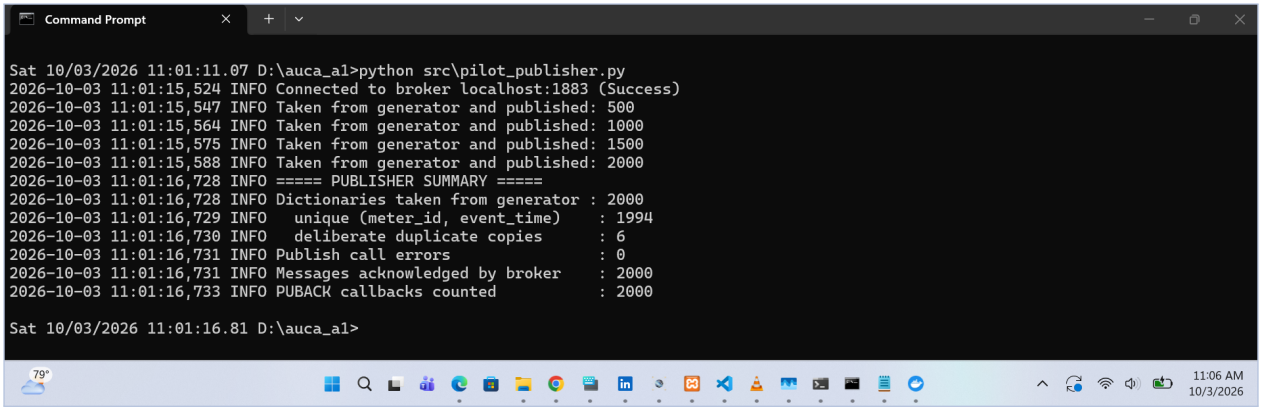


Figure 1.3-2: Run 1 publisher: 2,000 dictionaries taken and 2,000 acknowledged by the broker

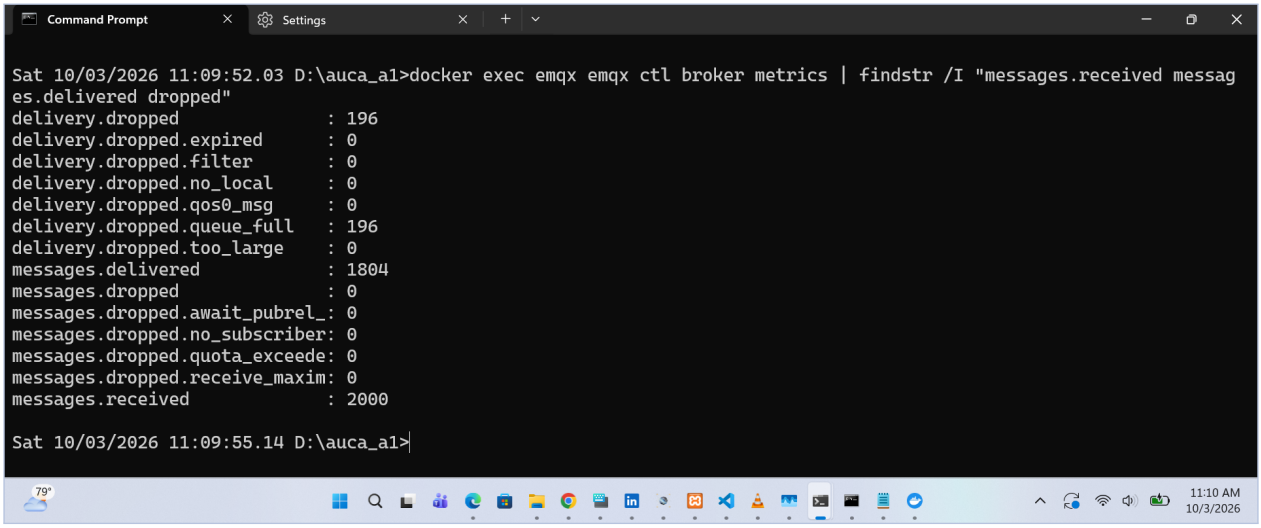


Figure 1.3-3: EMQX metrics after run 1: 2,000 received, 1,804 delivered, 196 dropped because the queue was full

Run 2 evidence (after correction)

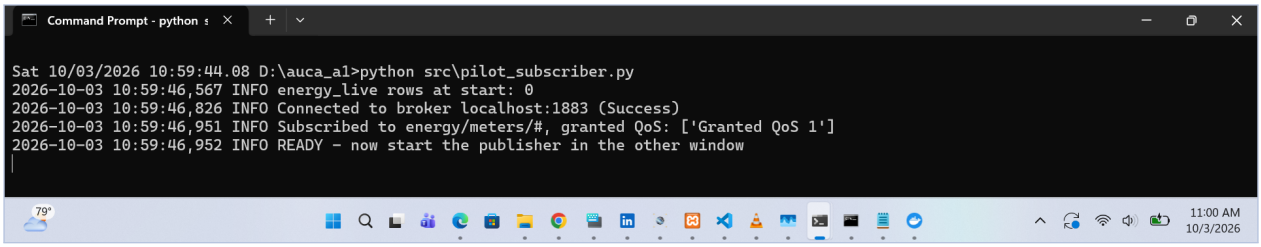


Figure 1.3-4: Subscriber started first: energy_live empty (0 rows), subscribed to energy/meters/# at QoS 1

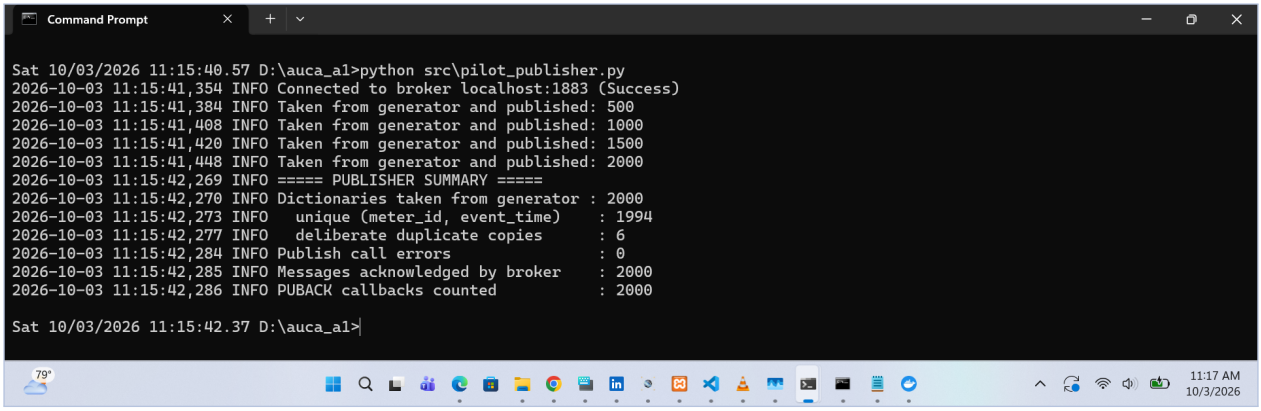


Figure 1.3-5: Publisher completion: 2,000 dictionaries taken and 2,000 messages acknowledged

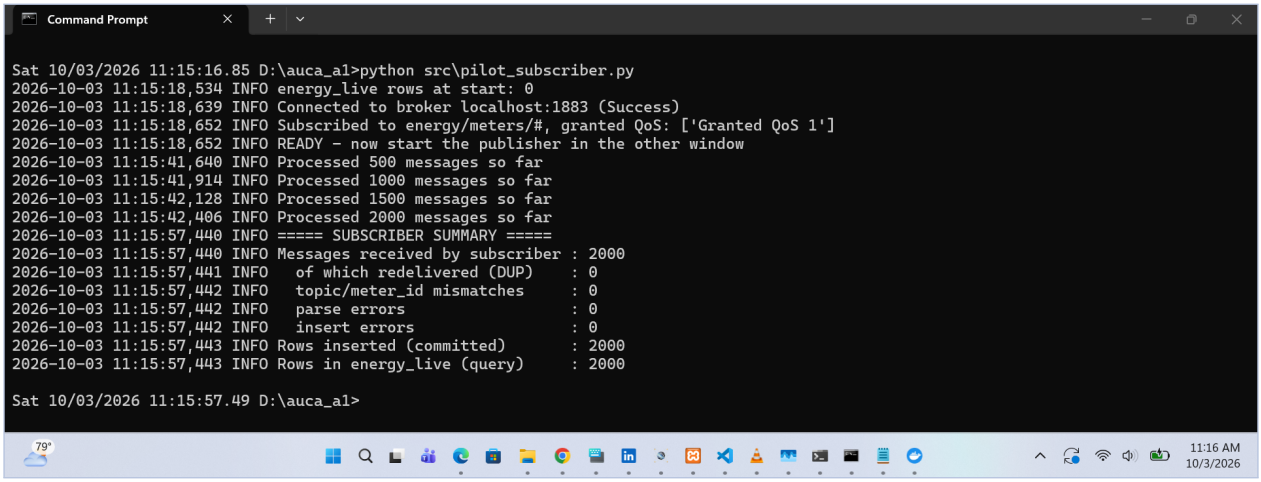


Figure 1.3-6: Subscriber summary: 2,000 messages received, 0 errors, 2,000 rows in energy_live

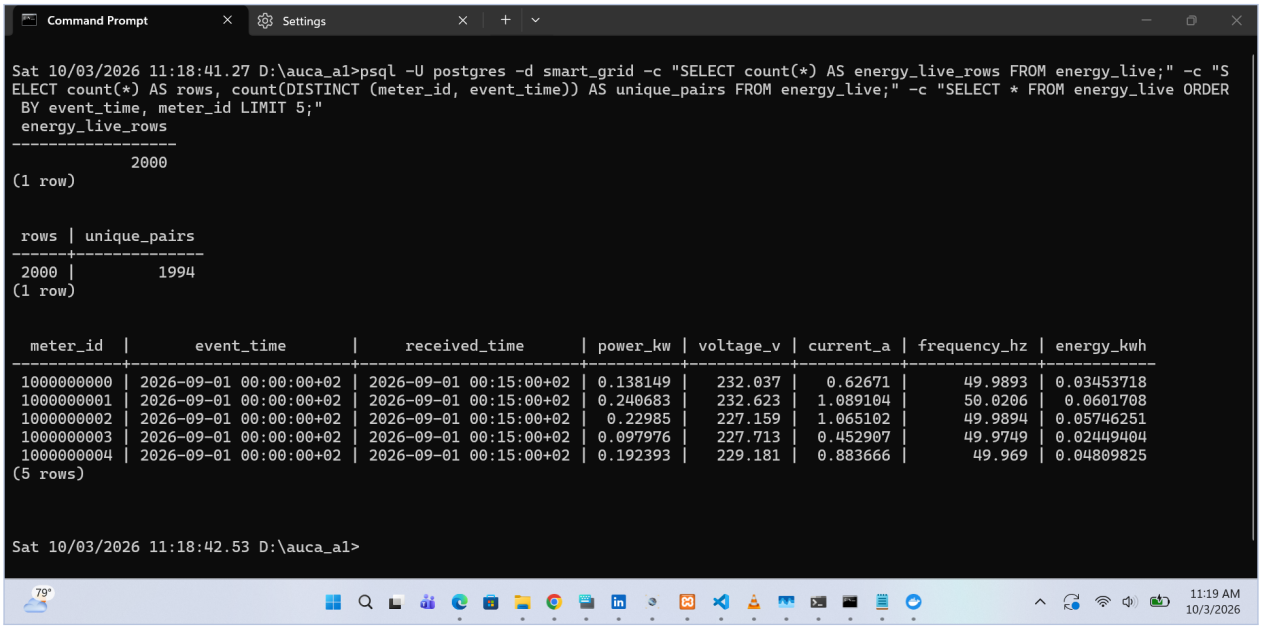


Figure 1.3-7: Database query: 2,000 stored rows (1,994 unique pairs) and sample rows

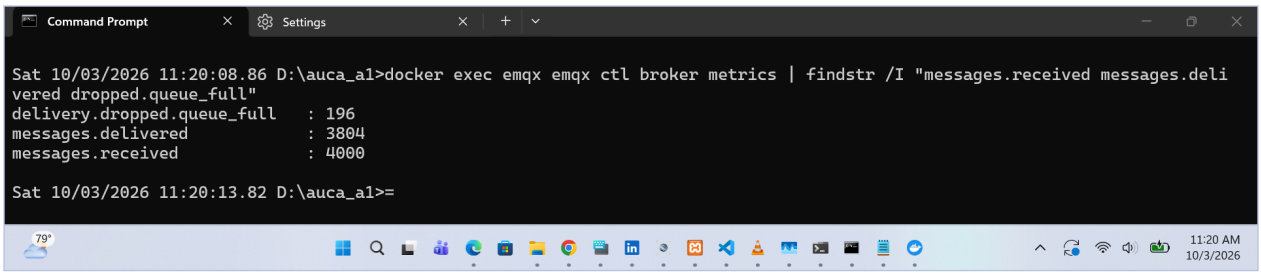


Figure 1.3-8: EMQX metrics after run 2: no new queue-full drops

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

The program `src/capture_and_load.py` first calls `dataset_info()` and records the expected counts, then starts a **fresh** `iter_readings()` iterator covering 4,000 meters every 15 minutes from 00:00 on 1 September to 00:00 on 29 September 2026 (exclusive), 10,752,000 planned readings.

dataset_info() entry	Expected count
planned_readings	10,752,000
expected_counts.emitted_readings	10,696,848
expected_counts.unique_readings	10,643,735
expected_counts.duplicate_readings	53,113
expected_counts.delayed_unique_readings	213,090
expected_counts.missing_readings	108,265

Ingestion route: CSV file + PostgreSQL COPY. Phase 1 consumes the generator progressively and writes the records to `D:\auca_a1\data\energy_readings_full.csv` in batches of 100,000 rows, printing progress every million dictionaries; no list of millions of records is ever held in memory. Phase 2 bulk-loads the CSV into the initially empty energy_readings table with one `COPY ... FROM STDIN (FORMAT csv, HEADER true)` and commits. Before loading, energy_readings held 0 rows and `max_wal_size` was confirmed at 1 GB so that the transaction log could not fill drive C:. The pilot data was left in energy_live.

Measure	Value
Dictionaries consumed from the generator	**10,696,848**
CSV data rows (excluding header)	**10,696,848** (file size 1,108.1 MB)
Committed rows in energy_readings	**10,696,848**
expected_counts.emitted_readings	10,696,848, so all three counts agree
CSV file generation	187.5 s
Database load (first database write to final commit)	131.9 s
Total elapsed (start of generation to final commit)	**323.6 s** (includes 3.9 s counting the CSV rows between the phases)

Database checks after loading

Check	Result
Total rows	10,696,848
Distinct meters	4,000
Earliest event_time	2026-09-01 00:00:00+02
Latest event_time	2026-09-28 23:45:00+02 (last interval before 29 Sep 00:00, exclusive)
Chunks	29
Table data size	957.8 MB
Index size	0 MB (indexes are created after cleaning, Section 3.1)
Combined size (all chunks)	957.8 MB

The index size is 0 bytes at this stage because the hypertable was created without default indexes for a fast bulk load; identical indexes are added to all three chunk configurations after cleaning (Section 3.1). energy_readings has **29 one-day chunks rather than 28** because TimescaleDB aligns chunk boundaries to UTC midnight, which is 02:00 CAT. The 28 CAT days therefore span 29 UTC-aligned chunks: the first chunk holds only 1 September 00:00–01:45 CAT and the last holds 28 September 02:00–23:45 CAT. Daily analysis buckets are aligned to CAT midnight separately with `time_bucket` and the Africa/Kigali time zone. After the database load and its screenshots were verified, the intermediate CSV may be deleted; `capture_and_load.py` regenerates it.

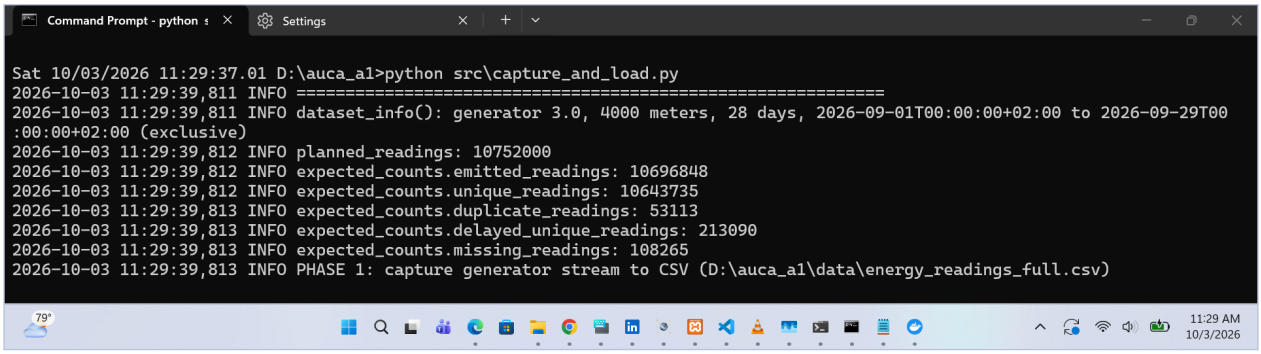


Figure 2.1-1: dataset_info(): generator 3.0, period, planned readings and expected counts recorded at the start

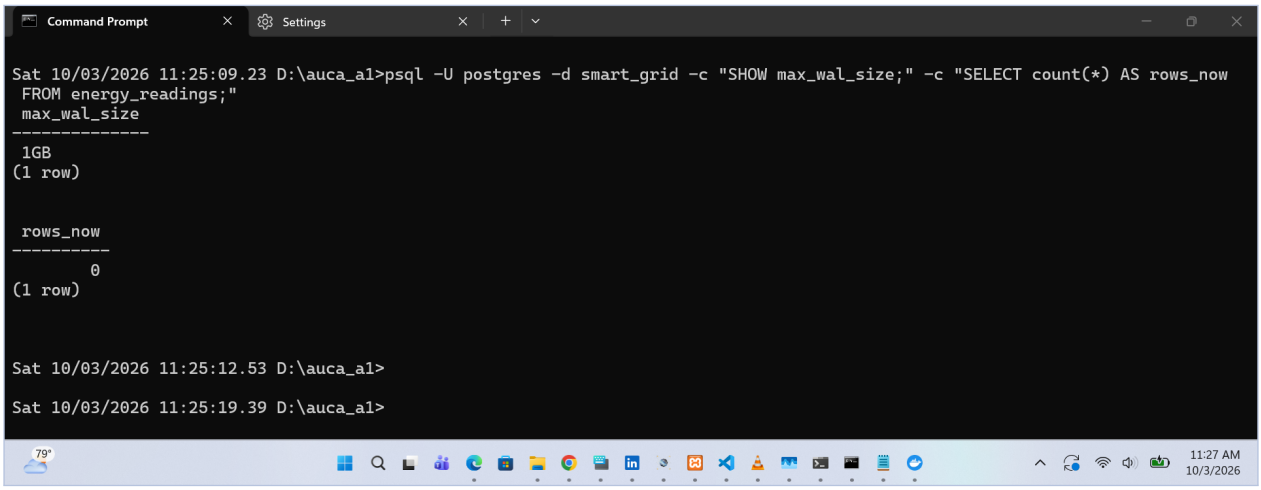


Figure 2.1-2: Before loading: max_wal_size 1 GB and energy_readings empty (0 rows)

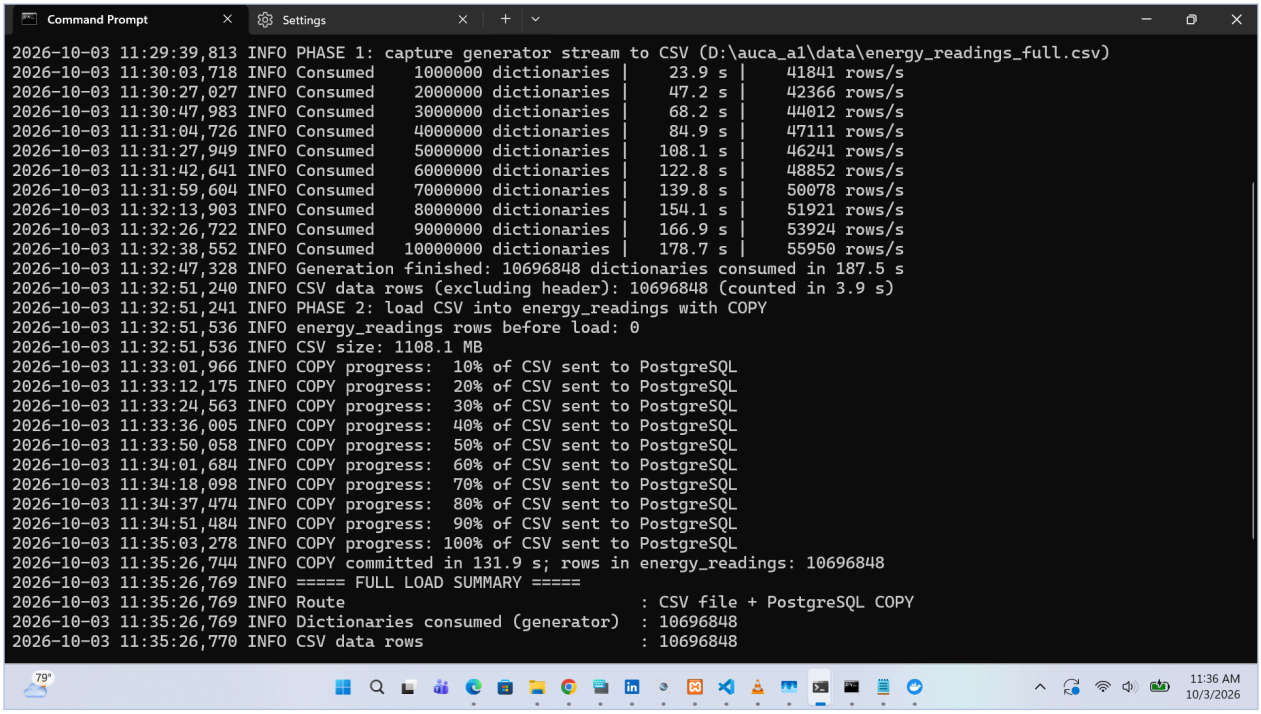


Figure 2.1-3: Progressive capture (progress every 1,000,000 dictionaries), CSV row count and COPY progress

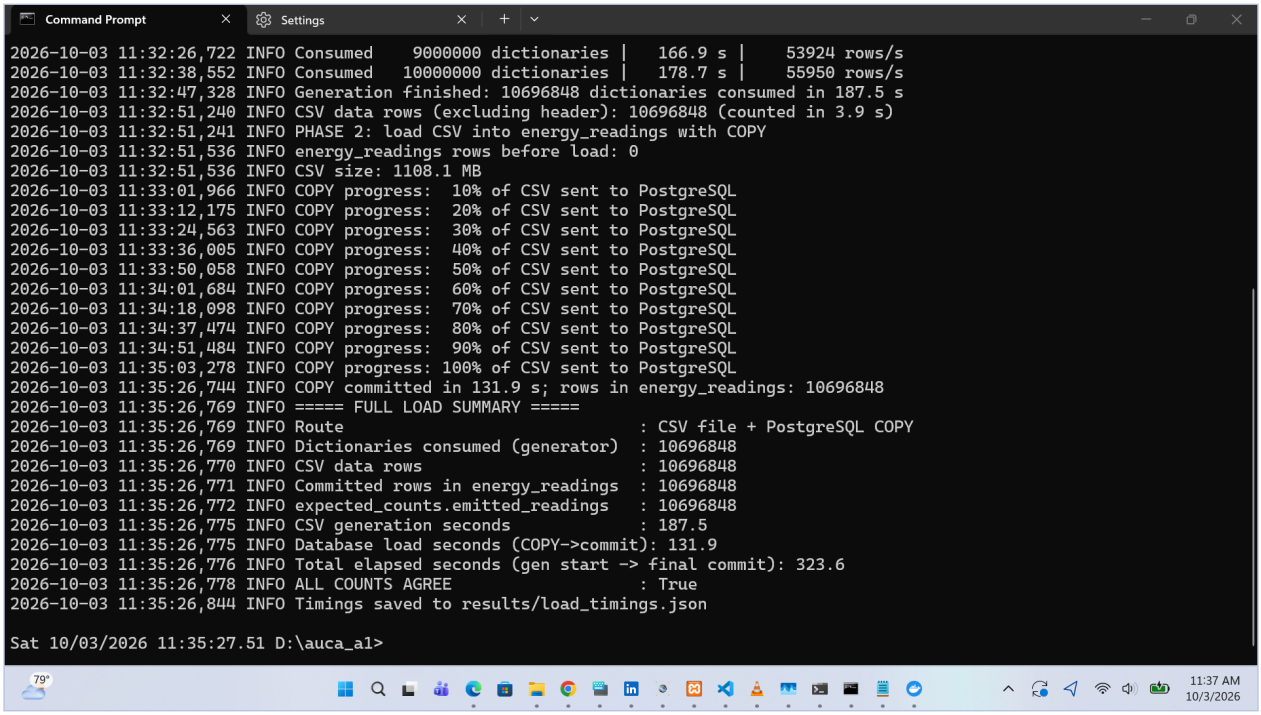


Figure 2.1-4: Full load summary: 10,696,848 consumed, in the CSV and committed; generation 187.5 s, load 131.9 s, total 323.6 s

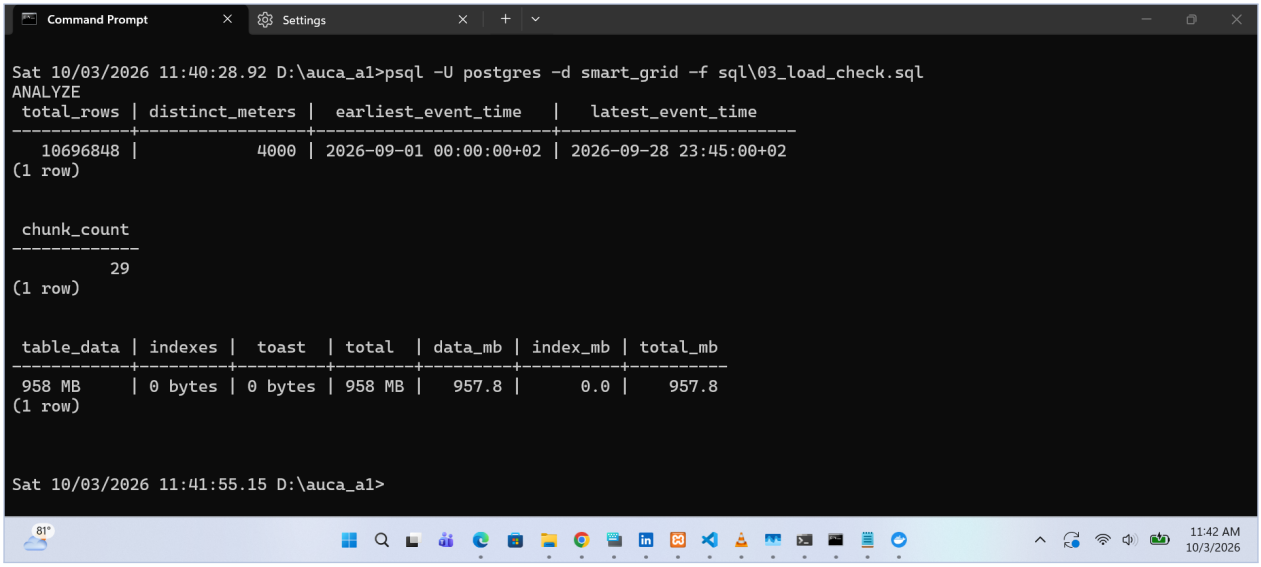


Figure 2.1-5: Distinct meters, earliest and latest event_time, chunk count and table, index and total size

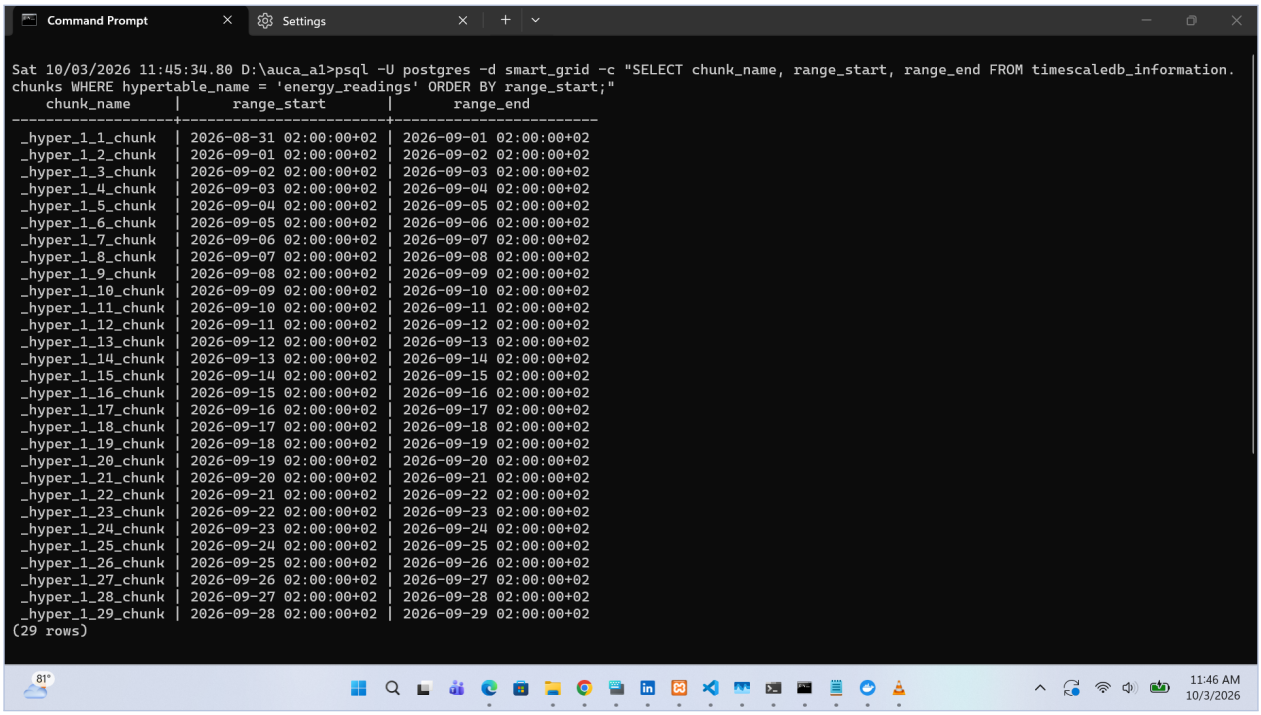


Figure 2.1-6: The 29 one-day chunks, aligned 02:00 to 02:00 CAT (UTC midnight)

2.2 Investigate data quality

`sql/04_quality.sql` queried energy_readings before cleaning. A unique reading is one meter_id and event_time pair; a temporary table held one row per observed pair with its number of copies. Missing pairs were counted directly against the planned grid (every meter crossed with every 15-minute interval), extra copies as copies beyond one per pair, and delayed unique readings as pairs whose received_time is later than the interval end (event_time + 15 minutes).

Entry	Expected	Measured	Match
planned_readings	10,752,000	10,752,000	yes
emitted_readings	10,696,848	10,696,848	yes

Entry	Expected	Measured	Match
unique_readings	10,643,735	10,643,735	yes
duplicate_readings (extra copies)	53,113	53,113	yes
delayed_unique_readings	213,090	213,090	yes
missing_readings	108,265	108,265 (counted directly from the grid)	yes

Percentage	Formula	Value
Missing	100 x 108,265 / 10,752,000	**1.0069 %**
Duplicate	100 x 53,113 / 10,643,735	**0.4990 %**
Delayed	100 x 213,090 / 10,643,735	**2.0020 %**
Completeness	100 x 10,643,735 / 10,752,000	**98.9931 %**

Consistency checks. Unique readings plus extra copies equal the emitted readings (10,643,735 + 53,113 = 10,696,848), and planned readings minus unique readings equal the missing readings (10,752,000 − 10,643,735 = 108,265); both checks returned true. Delays of the delayed readings range from exactly 30 to 180 minutes, and the other 10,430,645 unique readings arrived on time, exactly 15 minutes after event_time.

Meters with the most missing readings (2,688 planned each): 1000003622 (Region 3, commercial, 47 missing), 1000000106 (Region 7, industrial, 45), 1000000366 (Region 7, commercial, 45) and 1000001259 (Region 10, residential, 44), followed by a **12-way tie at 43** missing readings for fifth place; 1000000189 (Region 10, commercial) is the lowest meter_id in the tie. Even the worst meter is missing only 47 of 2,688 readings (1.75 %), so missing readings are spread thinly across meters rather than concentrated in a few faulty ones.

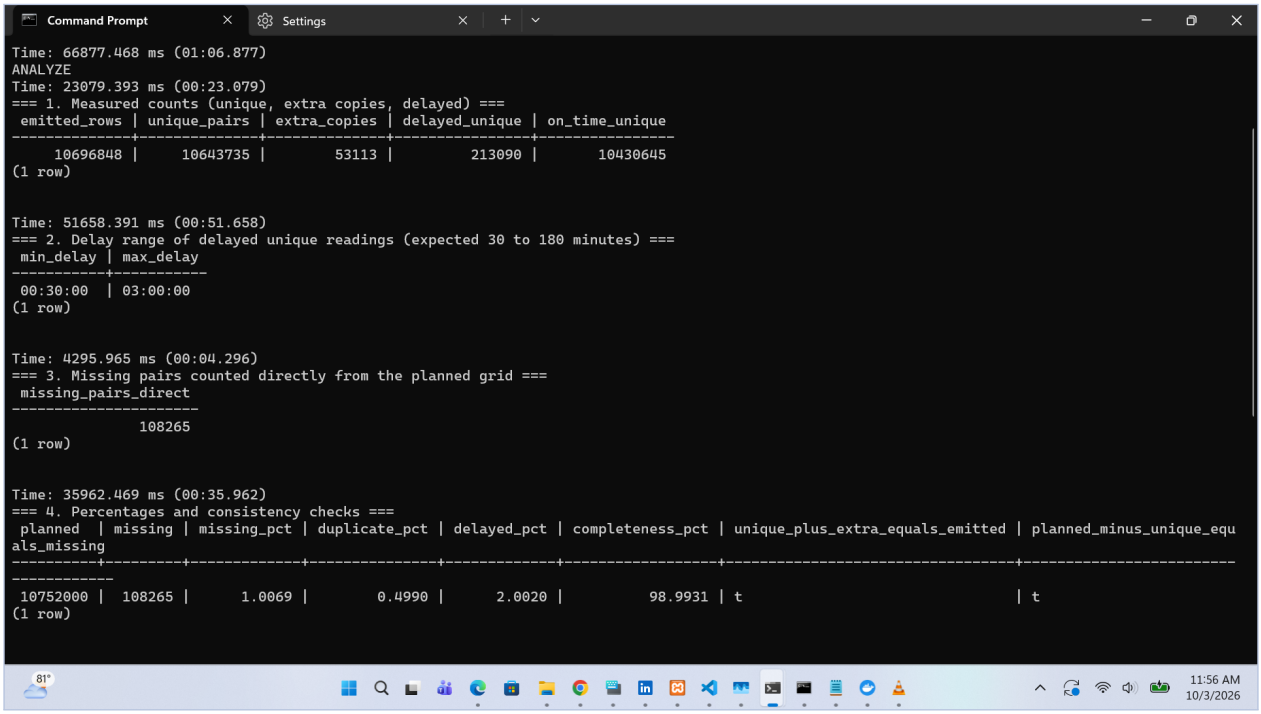


Figure 2.2-1: Measured counts, delay range (30 to 180 minutes), missing pairs from the planned grid, percentages and consistency checks

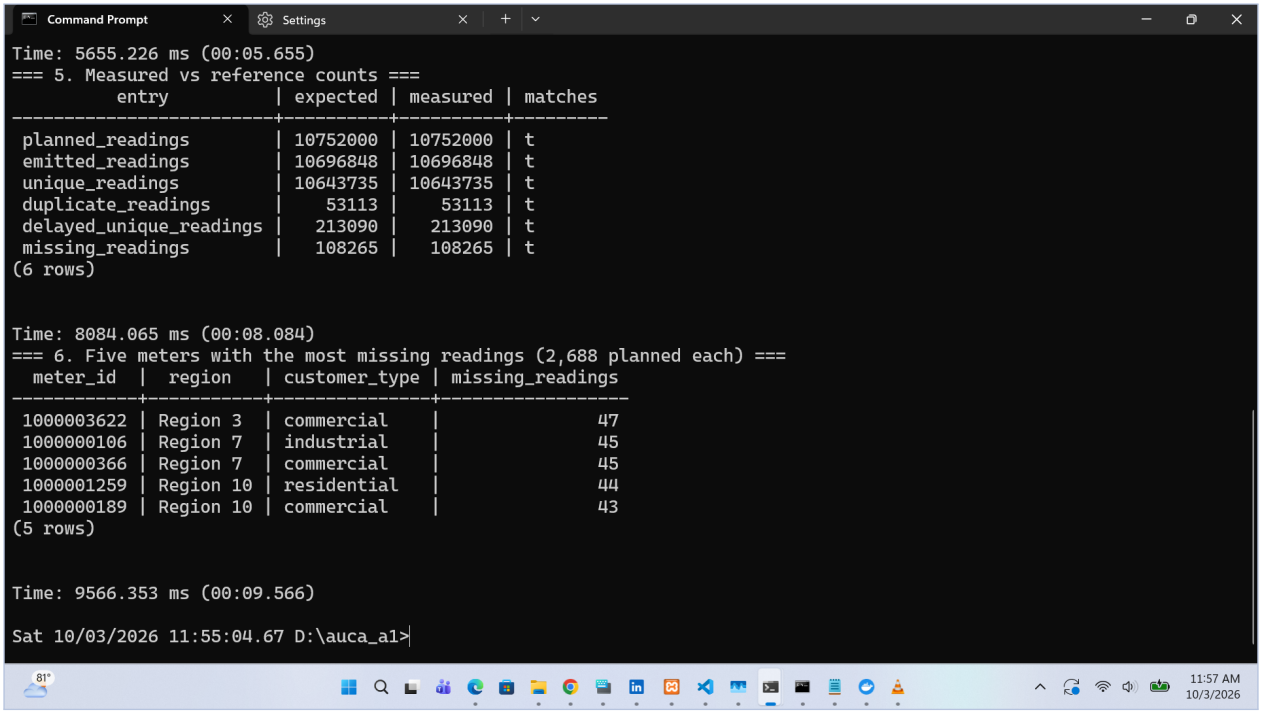


Figure 2.2-2: Measured counts against the reference table (all match) and the five meters with the most missing readings

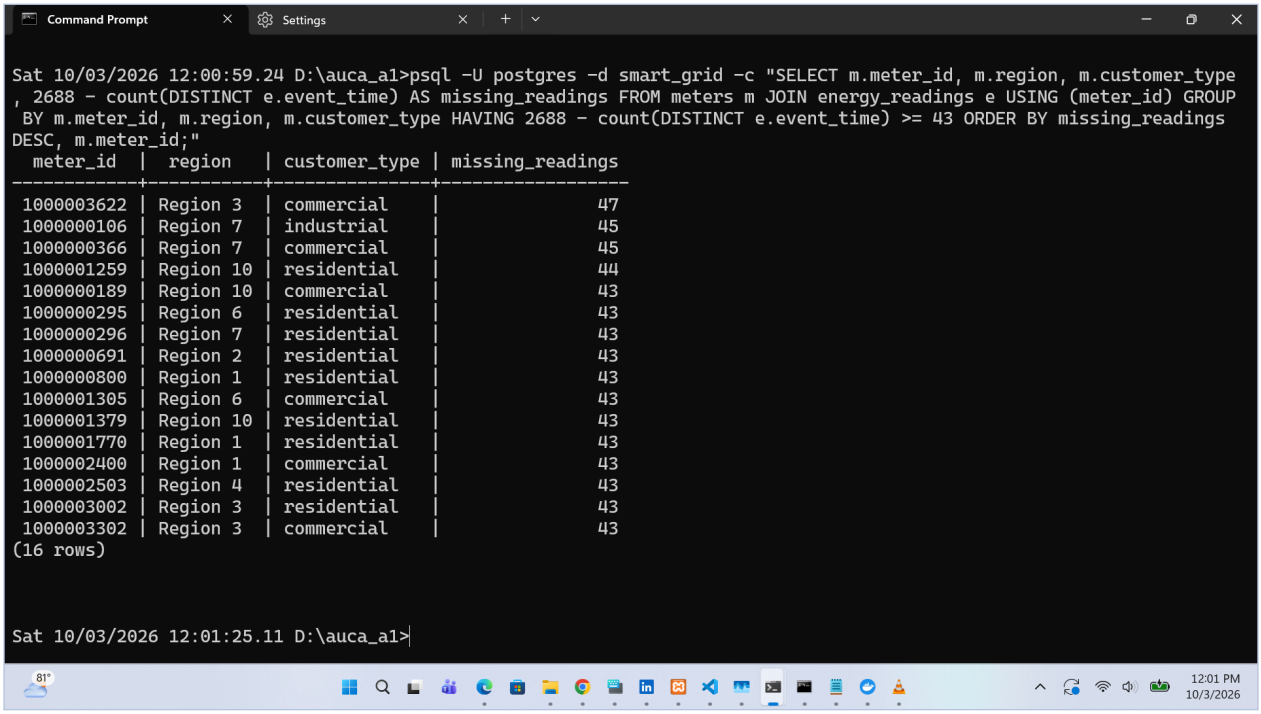


Figure 2.2-3: All meters with 43 or more missing readings, showing the tie at fifth place

2.3 Clean and preserve a consistent benchmark

`sql/05_clean.sql` first stored the 53,113 duplicate pairs in a temporary table and numbered the 106,226 rows belonging to them, partitioned by all eight columns. **Investigation:** all 53,113 groups contained exactly two copies, identical in received_time, power_kw, voltage_v, current_a, frequency_hz and energy_kwh, so there were **0 conflicting groups**. One copy of each pair was kept and the extra copy deleted, in a single transaction

that rolls back automatically unless exactly 10,643,735 rows remain. The script is repeatable: running it again deletes 0 rows. Missing intervals were left as gaps; nothing was inserted.

The first attempt numbered all 10.7 million rows in one window and hash operation and failed with "out of memory". Because the delete ran inside one transaction it was rolled back and the table was unchanged (10,696,848 rows); the corrected version isolates the duplicate pairs first and only numbers their rows.

Measure	Before cleaning	After cleaning
Row count	10,696,848	**10,643,735** (= expected_counts.unique_ readings)
Summed energy_kwh	3,872,863.893179 kWh	**3,853,704.346816 kWh**
Duplicate meter and event_time pairs	53,113	**0**

The energy reduction, 3,872,863.893179 − 3,853,704.346816 = **19,159.546363 kWh**, equals exactly the energy held in the 53,113 deleted copies, so only duplicated energy was removed. This cleaned table is used for every later storage experiment and analysis.

Why missing readings must not become zero. A missing reading means no measurement was received, not that the customer used no energy: the meter was most likely still consuming power while its transmission failed. Inserting zero would understate total energy (about 1 % of intervals), distort averages and daily totals, create false "unusual drop" anomalies in Section 5.4 and bias the forecast training data downward. Keeping gaps lets each analysis treat them explicitly (for example by reporting completeness), and keeps reported totals honest about being observed consumption.

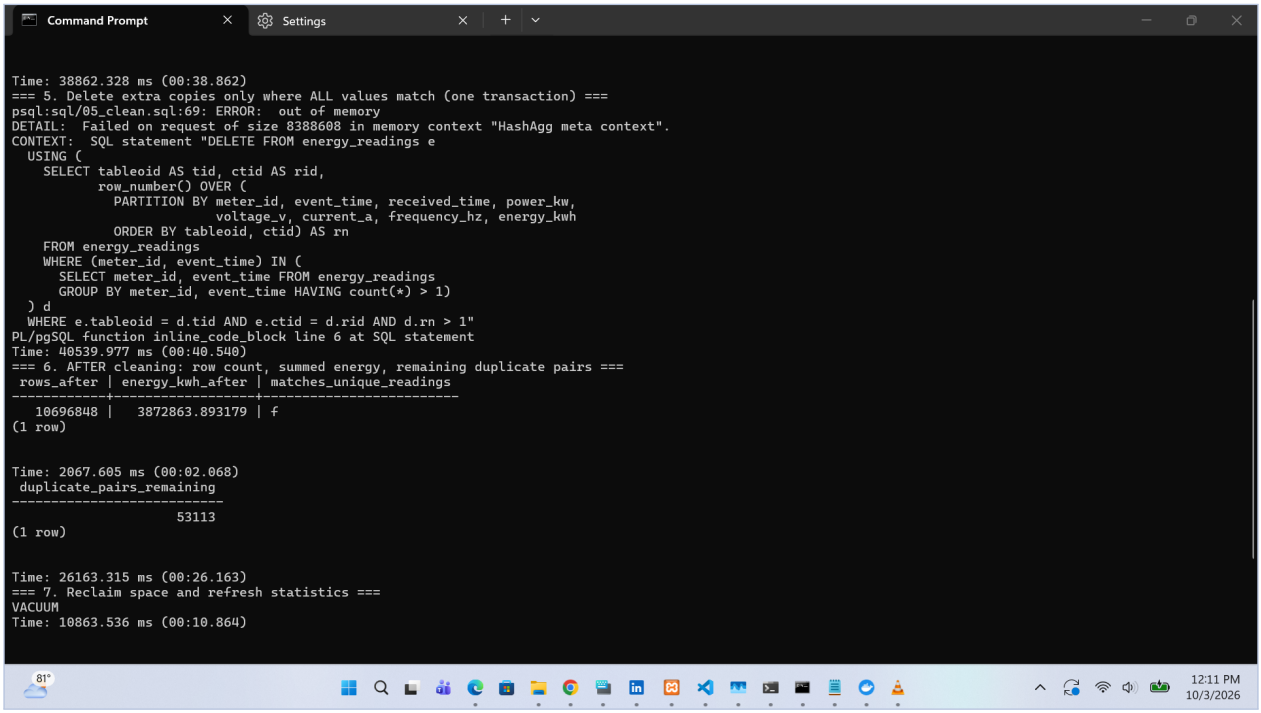


Figure 2.3-1: First cleaning attempt: out of memory, transaction rolled back, table unchanged

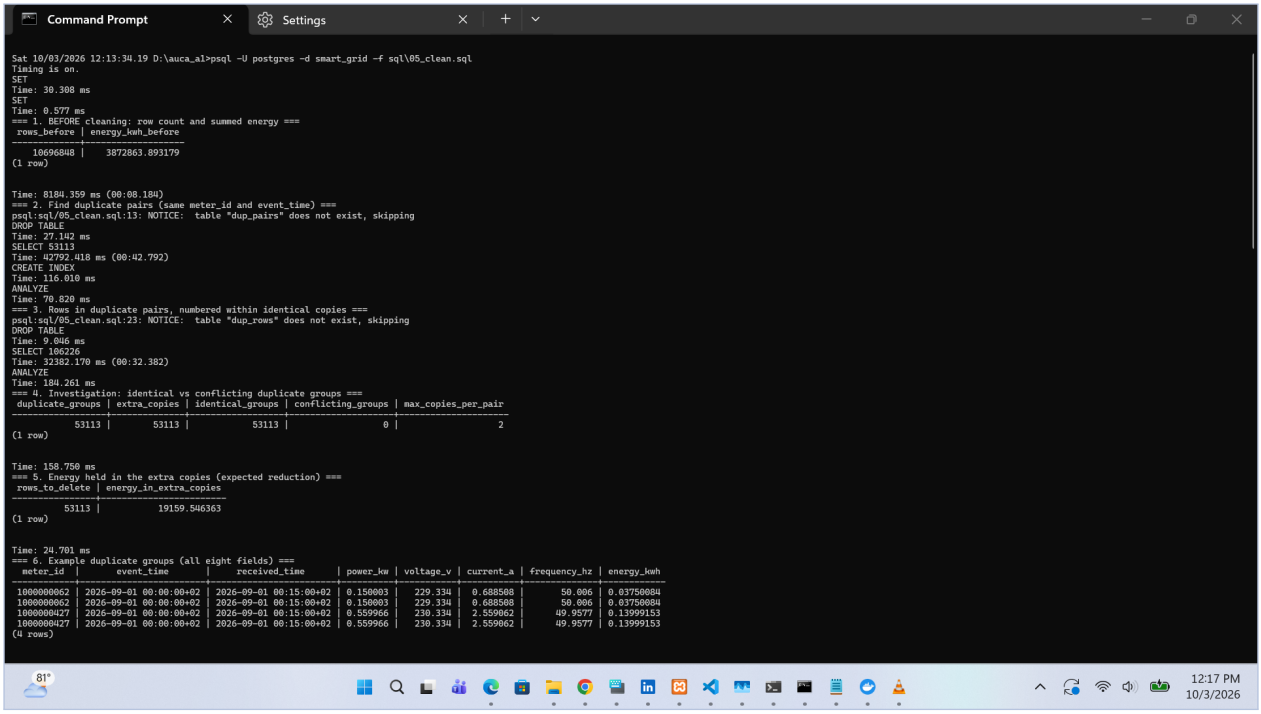


Figure 2.3-2: Before cleaning: row count and energy, duplicate investigation (0 conflicting groups), energy in extra copies and example duplicate rows

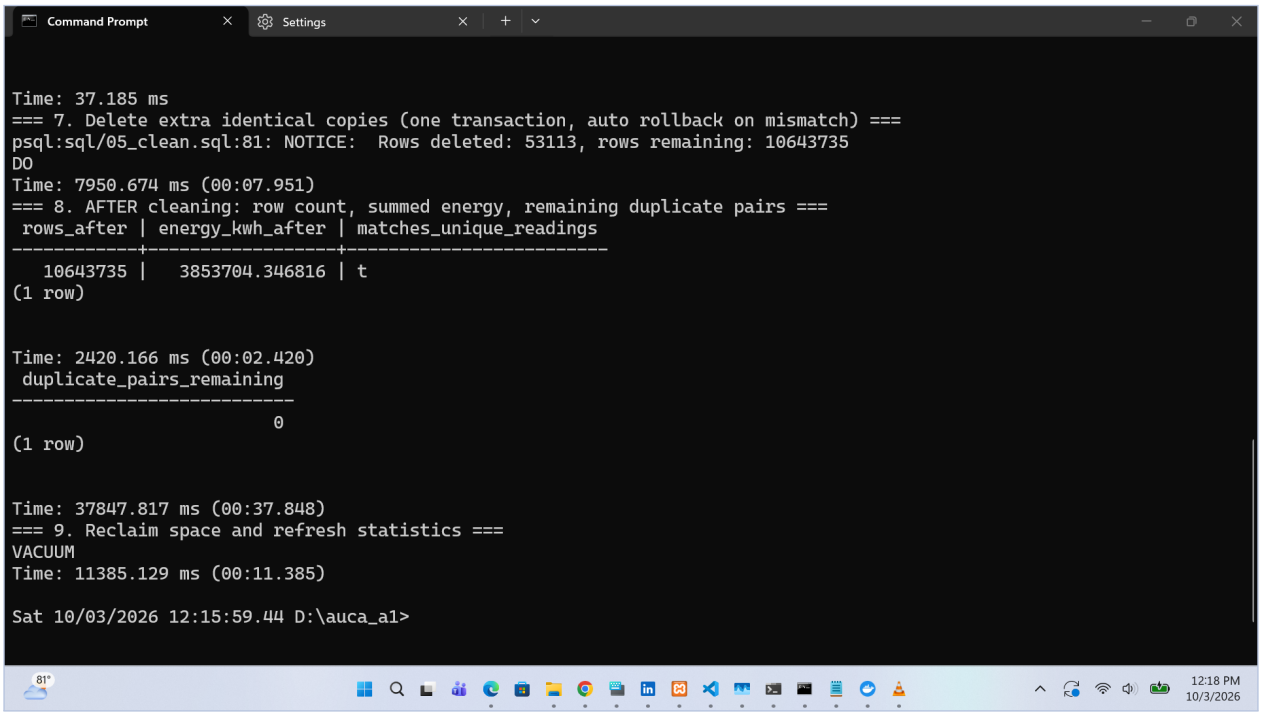


Figure 2.3-3: Cleaning completion: 53,113 rows deleted, 10,643,735 remaining, energy after cleaning and 0 duplicate pairs remaining

3 Compare chunk intervals

3.1 Create and verify three configurations

energy_readings was kept as the one-day hypertable. `sql/06_chunk_configs.sql` created **energy_readings_3h** (three-hour chunks) and **energy_readings_week** (one-week chunks) in the same database, with the same eight columns and event_time partitioning, and copied the 10,643,735 cleaned rows into each (`INSERT ... SELECT`). The same two non-unique B-tree indexes with ascending order were then created on all three tables, on `(event_time)` and on `(meter_id, event_time)`, and all three were analysed (`ANALYZE`) so the planner has current row estimates. Compression is off on all three.

Configuration	Chunk interval	Chunks	Chunk coverage (CAT)	Rows	Meters
energy_readings_3h	3 hours	225	31 Aug 23:00 to 29 Sep 02:00	10,643,735	4,000
energy_readings	1 day	29	31 Aug 02:00 to 29 Sep 02:00	10,643,735	4,000
energy_readings_week	7 days	5	27 Aug 02:00 to 1 Oct 02:00	10,643,735	4,000

Matching data checks. All three tables have the same earliest and latest event_time (2026-09-01 00:00:00+02 and 2026-09-28 23:45:00+02), the same sum of power_kw (15,414,817.387793 kW) and the same sum of energy_kwh (3,853,704.346815 kWh); the check `all_checks_pass` (10,643,735 rows, 4,000 meters, identical dates, sums within 0.001) is true for each. The energy total shows ...815 instead of ...816 from Section 2.3 only because adding 10.6 million floating-point values in a different order changes the last digit, far below the 0.001 tolerance.

The chunk counts are not round numbers because TimescaleDB aligns chunk boundaries to UTC. 28 days x 8 = 224 three-hour chunks plus one, because CAT midnight (22:00 UTC) falls inside a UTC three-hour slot; weekly chunks align to TimescaleDB's fixed weekly origin, so the 28 days spread across 5 partly-filled weeks.

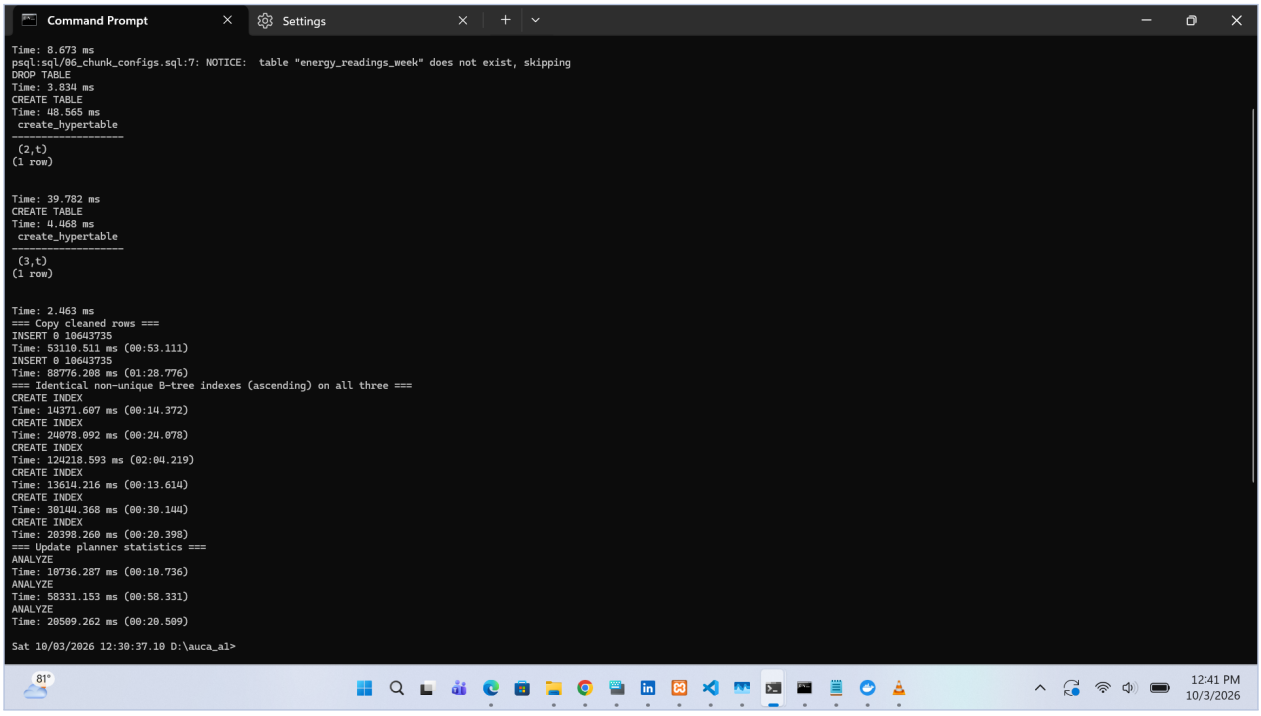


Figure 3.1-1: Creation of the 3-hour and 1-week hypertables, copy of 10,643,735 cleaned rows into each, identical indexes and ANALYZE

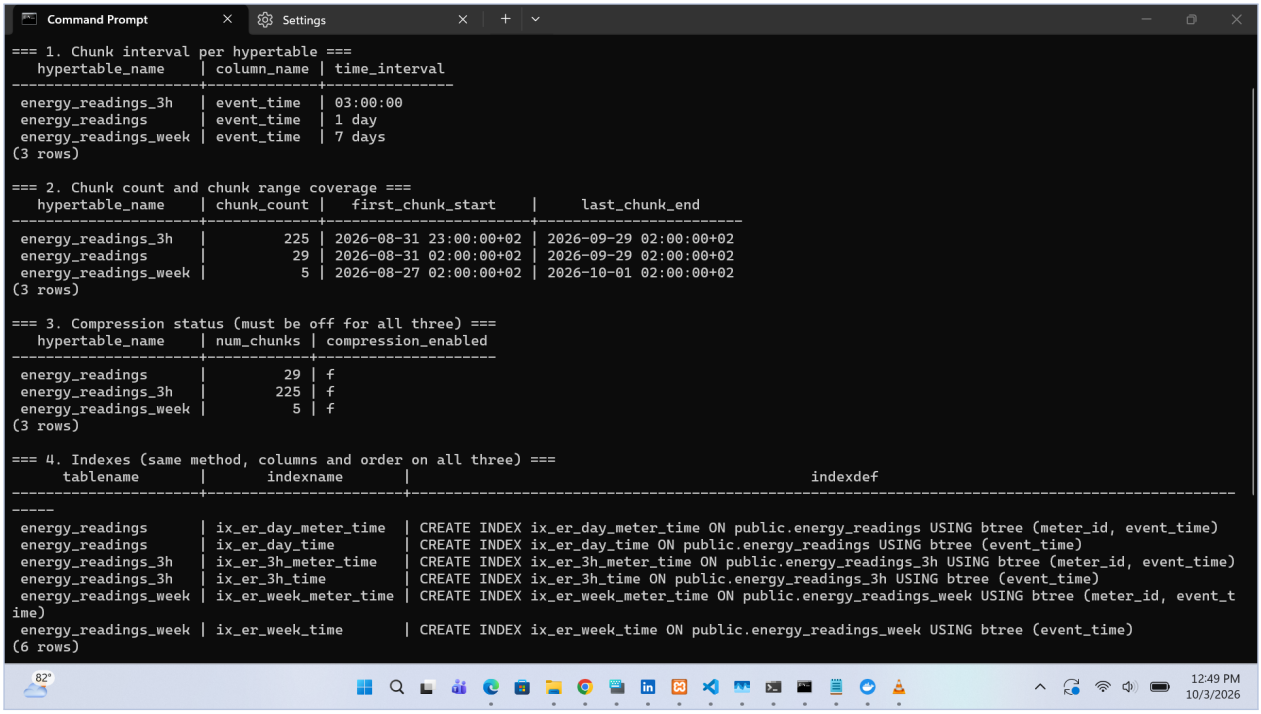


Figure 3.1-2: Chunk interval, chunk count and range, compression off, and the identical index definitions of the three tables

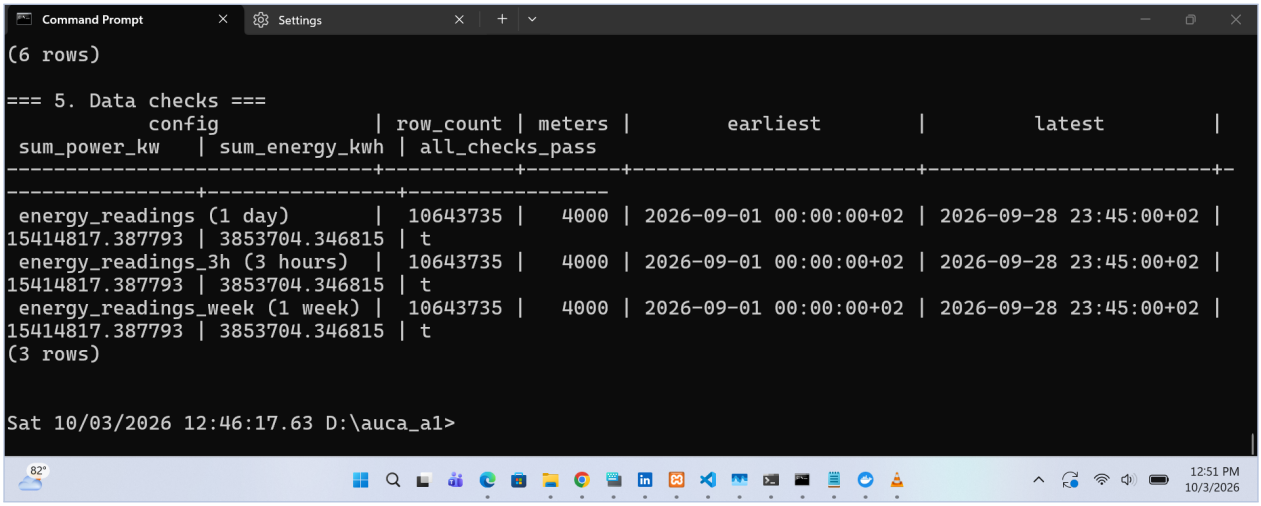


Figure 3.1-3: Data checks: row count, distinct meters, date range and sums identical for the three configurations

3.2 Implement and time the five workloads

The five queries below were run on all three hypertables by `src/benchmark.py`, changing only the source table (`{t}`). Results were compared with energy_readings and matched within 0.001.

Query workloads

Q1 Hourly average power across all readings on 21 September (CAT hours):

```
SELECT time_bucket('1 hour', event_time) AS cat_hour, avg(power_kw) AS avg_power_kw, count(*) AS readings
FROM {t}
WHERE event_time >= '2026-09-21 00:00:00+02' AND event_time < '2026-09-22 00:00:00+02'
GROUP BY cat_hour ORDER BY cat_hour;
```

Q2 The ten 15-minute intervals with the highest total power, 22 to 28 September (exactly ten rows, ties ordered by event_time):

```
SELECT event_time, sum(power_kw) AS total_power_kw
FROM {t}
WHERE event_time >= '2026-09-22 00:00:00+02' AND event_time < '2026-09-29 00:00:00+02'
GROUP BY event_time ORDER BY total_power_kw DESC, event_time ASC LIMIT 10;
```

Q3 Total energy per meter across the full period:

```
SELECT meter_id, sum(energy_kwh) AS total_energy_kwh FROM {t} GROUP BY meter_id ORDER BY meter_id;
```

Q4 Energy total, average power and peak power for meter 1000000000, 22 to 28 September:

```
SELECT sum(energy_kwh) AS energy_kwh, avg(power_kw) AS avg_power_kw, max(power_kw) AS peak_power_kw
FROM {t}
WHERE meter_id = 1000000000
  AND event_time >= '2026-09-22 00:00:00+02' AND event_time < '2026-09-29 00:00:00+02';
```

Q5 Full scan: row count, total energy, and average, minimum and maximum power:

```
SELECT count(*), sum(energy_kwh), avg(power_kw), min(power_kw), max(power_kw) FROM {t};
```

Procedure

Client: Python 3.13.6 with psycopg2 2.9.13 (one connection, autocommit). **Timing tool:** Python `time.perf_counter()`, measuring from statement submission until the last result row was received (`fetchall` completed). **Machine:** Erneste. Benchmarks ran sequentially with unchanged settings, other programs closed, Docker stopped and no loaders or assignment background jobs running. For each query and table there was **one warm-up followed by five measured executions**; PostgreSQL was not restarted and caches were not cleared, so all figures are **warm-cache measurements**. Each run's CAT start and end, duration and machine are stored in `results/bench_section3_runs.csv`. **Session:** 18:38:30 to 18:43:19 CAT on 3 October 2026, **289.3 s** from the first warm-up submission to the last measured result. All 15 result comparisons matched (largest difference 9.5 x 10^-8).

Results (warm cache, milliseconds)

Query	Table	Median	Min	Max
Q1	energy_readings_3h	**308.2**	306.5	712.1
Q1	energy_readings	336.6	317.1	357.8
Q1	energy_readings_week	357.1	343.0	388.2
Q2	energy_readings_3h	**971.4**	849.5	1,045.9
Q2	energy_readings	1,069.4	863.6	1,153.8
Q2	energy_readings_week	1,187.2	1,136.4	1,528.2
Q3	energy_readings_3h	16,332.6	16,081.3	19,489.2
Q3	energy_readings	5,765.4	4,243.8	6,458.2
Q3	energy_readings_week	**3,881.1**	3,281.6	6,218.0
Q4	energy_readings_3h	18.4	17.2	42.4
Q4	energy_readings	6.7	6.2	8.0
Q4	energy_readings_week	**4.0**	3.8	5.9
Q5	energy_readings_3h	2,849.9	2,479.1	3,388.8
Q5	energy_readings	**2,494.1**	2,398.0	3,797.8

Query	Table	Median	Min	Max
Q5	energy_readings_week	2,551.5	2,434.3	4,277.4

One execution plan with actual timing and buffer statistics was captured for each configuration with `EXPLAIN (ANALYZE, BUFFERS)` on Q1 (`sql/08_plan_q1.sql`); Q3 and Q4 plans were saved to `results/plan_q3q4_*.txt`.

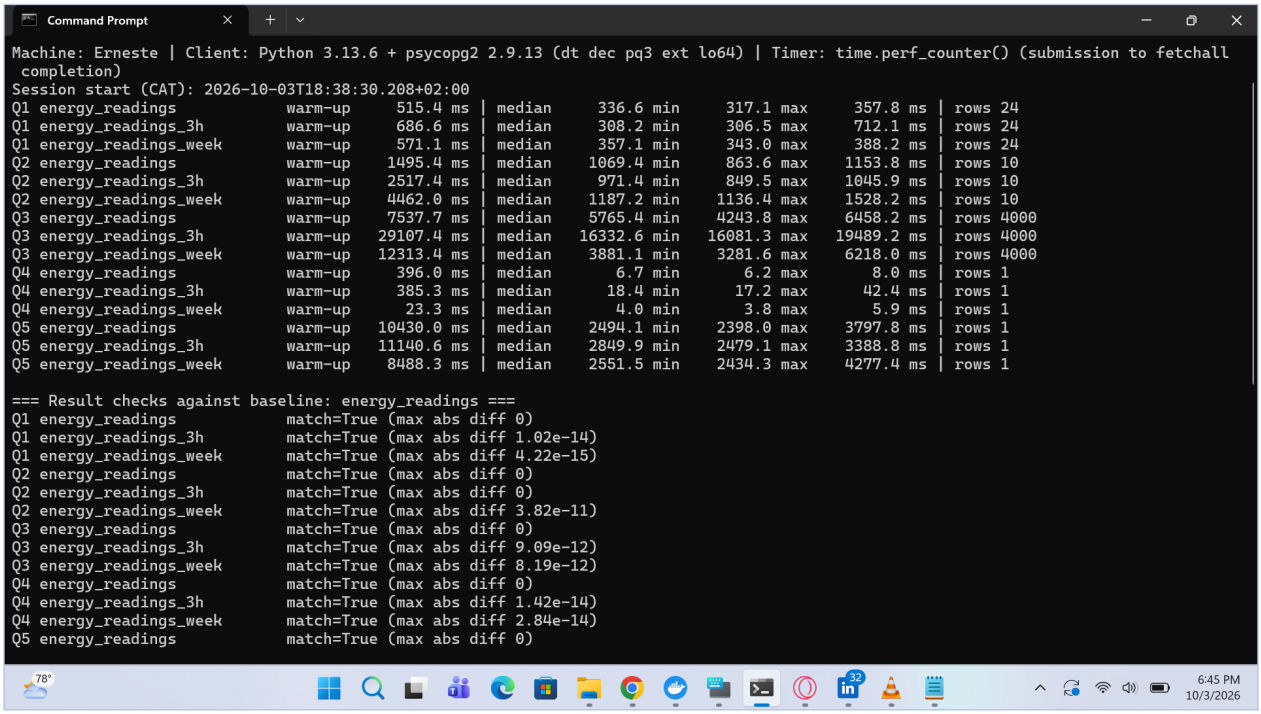


Figure 3.2-1: Benchmark per query and table (warm-up, median, min, max), client, timer, machine, and result checks against the baseline

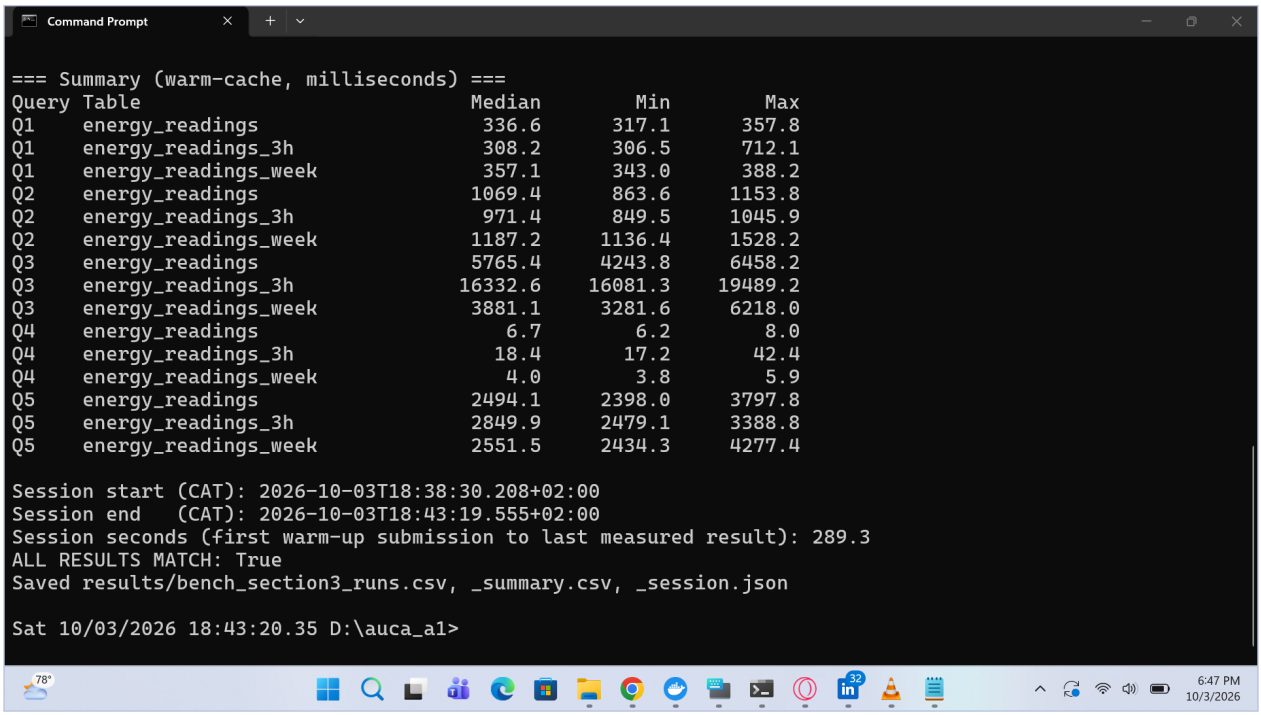


Figure 3.2-2: Timing comparison summary, session start and end, 289.3 session seconds, all results match

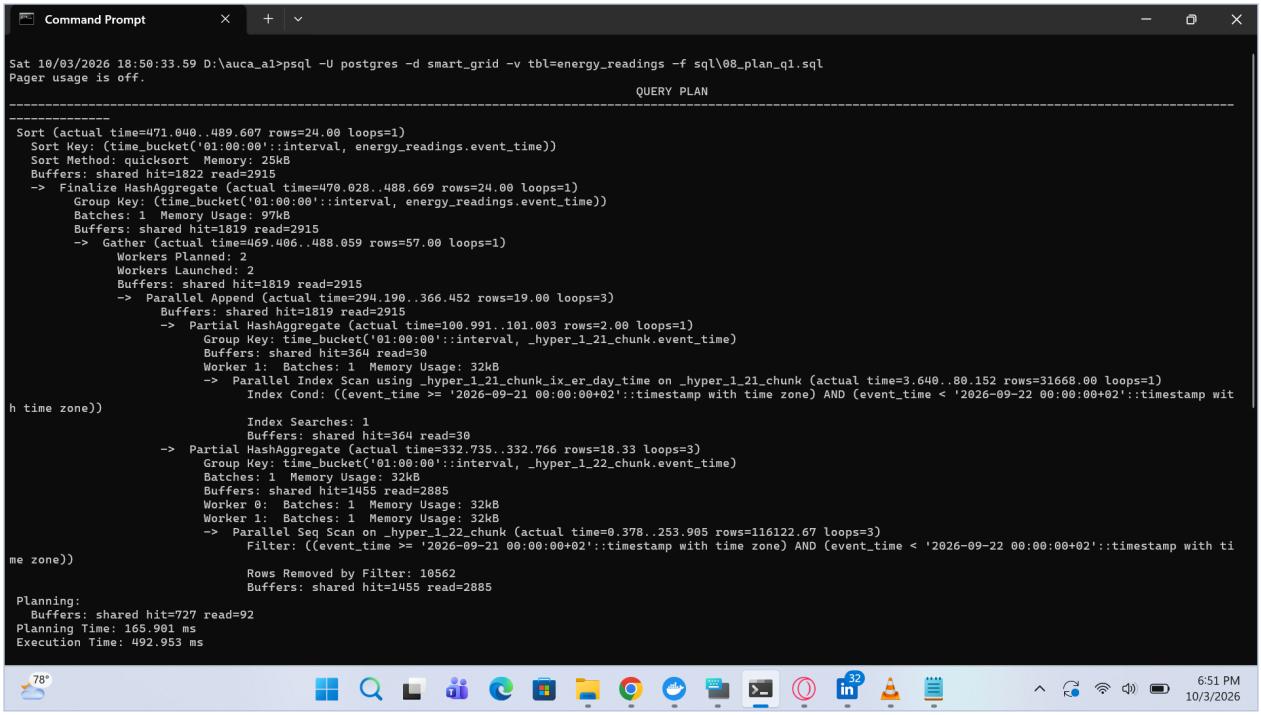


Figure 3.2-3: Q1 actual plan on energy_readings (1 day): 2 chunks, index scan then sequential scan, buffers and timing

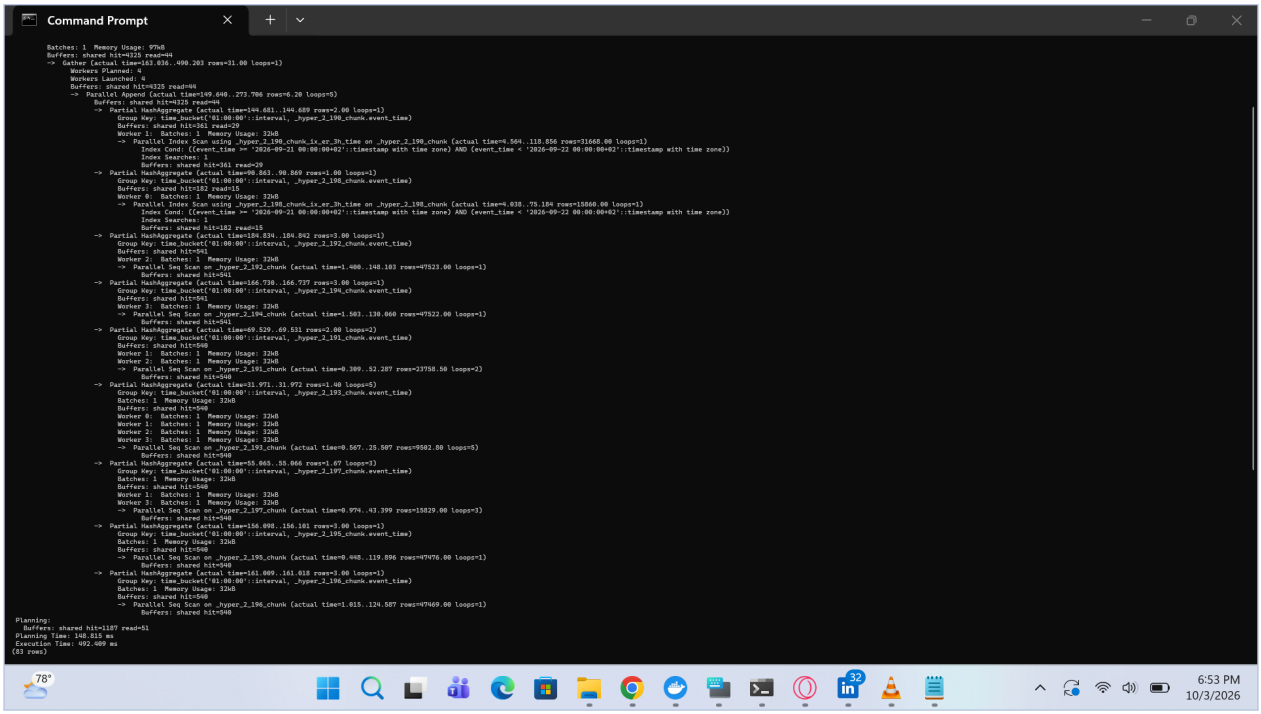


Figure 3.2-4: Q1 actual plan on energy_readings_3h (3 hours): 9 chunks, mostly sequential scans, buffers and timing

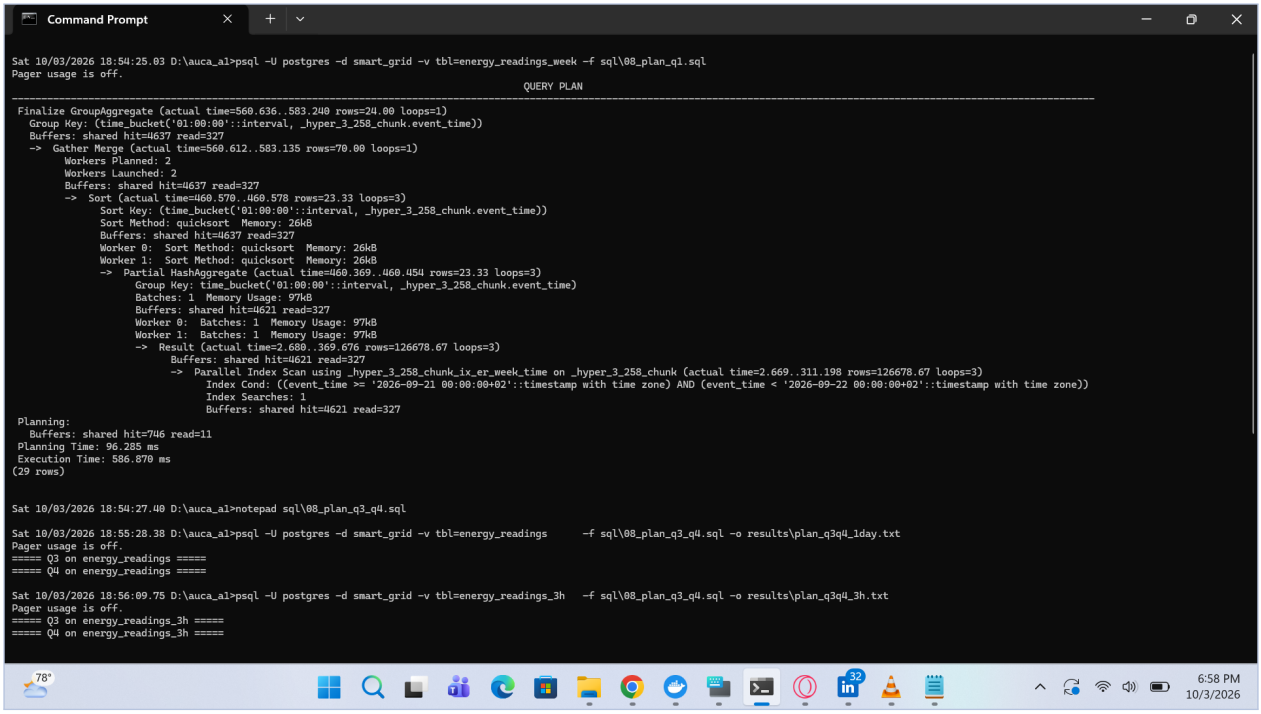


Figure 3.2-5: Q1 actual plan on energy_readings_week (1 week): 1 chunk, parallel index scan, buffers and timing

3.3 Explain the chunk results

Chunks scanned, taken from the actual plans (each Q3/Q4 plan file contains Q3, which reads every chunk, plus Q4):

Query	3 h	1 day	1 week
Q1 (21 Sep only)	9	2	1
Q2 and Q4 (22 to 28 Sep)	57	8	2
Q3 and Q5 (full period)	225	29	5

Q1, fastest: 3-hour chunks (308.2 ms; 8.4 % faster than 1 day at 336.6 ms and 13.7 % faster than 1 week at 357.1 ms). *Observation:* the 3-hour table scanned 9 chunks, mostly with full sequential scans; the 1-day table scanned 2 chunks (an index scan for 00:00–02:00 CAT and a sequential scan of the 02:00–24:00 chunk, removing 10,562 rows by filter); the weekly table used a parallel index scan inside one 7-day chunk. *Explanation:* small chunks match the one-day filter almost exactly, so nearly every row read is needed and the work spreads evenly over parallel workers; a weekly chunk must use its index to pick out one seventh of its rows. The differences are small because all three read the same ~380,000 rows.

Q2, fastest: 3-hour chunks (971.4 ms; 9.2 % faster than 1 day at 1,069.4 ms and 18.2 % faster than 1 week at 1,187.2 ms). *Observation:* the 7-day filter touched 57, 8 and 2 chunks. *Explanation:* the two weekly chunks (17–24 Sep and 24 Sep–1 Oct) are only 2/7 and 5/7 inside the filter, so irrelevant rows must be skipped, whereas small chunks cover the range almost exactly; with only 57 chunks the per-chunk overhead stays smaller than the saving.

Q3, fastest: 1-week chunks (3,881.1 ms; 32.7 % faster than 1 day at 5,765.4 ms and 76.2 % faster than 3 hours at 16,332.6 ms, which is 4.2 times slower). *Observation:* every chunk was read (5, 29 or 225); in the 3-hour plan, planning alone took 2,827 ms against 115 ms for the weekly table. *Explanation:* with no time filter no chunk can be excluded, so cost grows with the number of chunks: each chunk adds planning work and its own partial aggregation over 4,000 meters, about 900,000 partial groups to merge for 225 chunks against

20,000 for 5. The data volume is identical, so the extra time is per-chunk overhead.

Q4, fastest: 1-week chunks (4.0 ms; 40 % faster than 1 day at 6.7 ms and 78 % faster than 3 hours at 18.4 ms). *Observation:* the `(meter_id, event_time)` index was searched in 2, 8 or 57 chunks; the 3-hour plan executed in 61 ms and the weekly plan in 13 ms. *Explanation:* each chunk needs its own index lookup for very few rows, so total time grows with the number of chunks searched. (Plan timings come from a fresh psql session and include first-time catalogue loading, so they exceed the benchmark medians.)

Q5, fastest: 1-day chunks (2,494.1 ms; 2.2 % faster than 1 week at 2,551.5 ms and 12.5 % faster than 3 hours at 2,849.9 ms). *Observation:* all 10,643,735 rows were read in every configuration. *Explanation:* the work is dominated by reading the same ~958 MB, so 1 day and 1 week are practically equal (a 2 % gap, smaller than their run-to-run spread, whose maxima were 3,798 and 4,277 ms); the 3-hour table pays planning and scan overhead for 225 chunks.

Overall. Small chunks help narrow time-range queries (Q1, Q2) because they match the filter closely; large chunks help queries without a time filter or over long periods (Q3, Q4) because there are fewer chunks to plan and search. The 3-hour interval is too fine for this dataset: it gains under 100 ms on Q1 and Q2 but loses more than 10 seconds on Q3. The **1-day interval is never the slowest** and is close to the best on every workload, so it is the balanced choice used in Section 4.

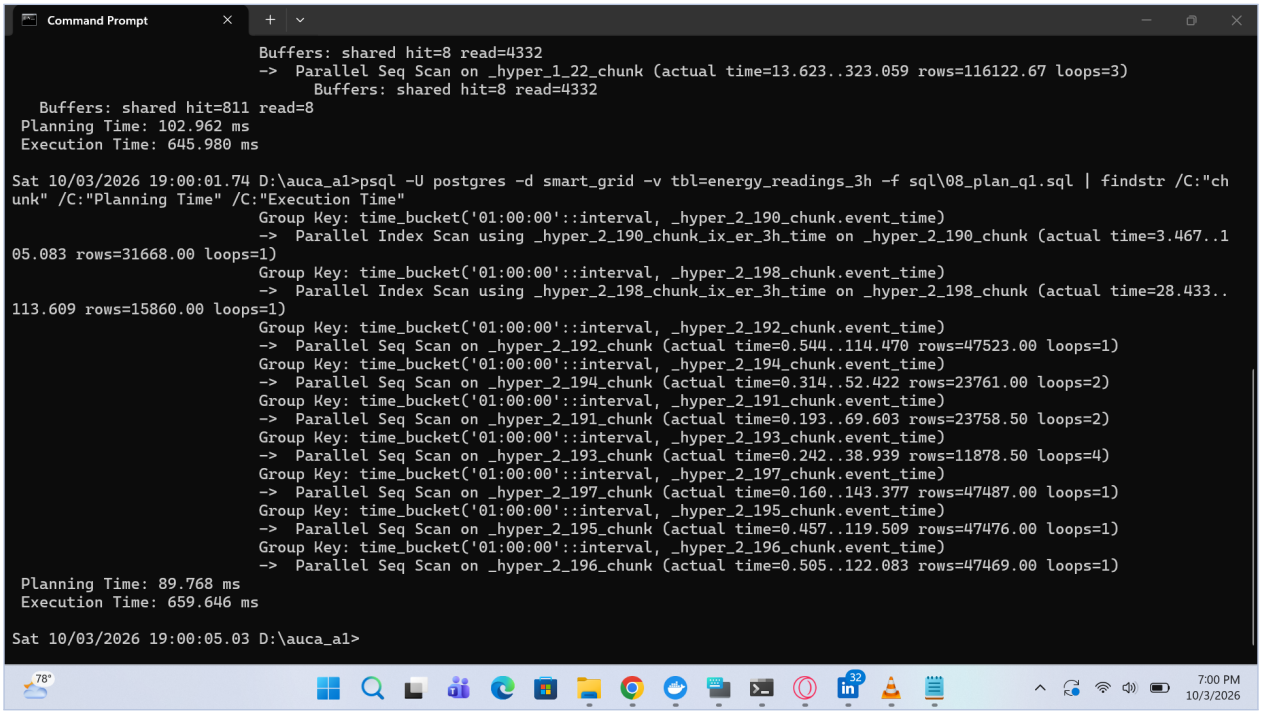


Figure 3.3-1: Q1 plan summaries for the 1-day and 3-hour tables: chunks scanned, planning and execution times

4 Compare compression strategies

4.1 Create the two compressed configurations

The **1-day chunk interval** was chosen from Section 3 because it was never the slowest configuration and was close to the best on every workload. `sql/09_compression_setup.sql` created two empty hypertables with one-day chunks, the same eight columns and the same two indexes as the Section 3 tables:

Table	Order by	Segment by	Policy job
energy_time_compressed	event_time DESC	none (segmentby set to empty)	1000
energy_meter_compressed	event_time DESC	meter_id	1001

Compression used TimescaleDB 2.30.2's settings (`timescaledb.compress`, `compress_orderby`, `compress_segmentby`; version 2.30 names the resulting jobs "Columnstore Policy"). Each table received a policy with `add_compression_policy(..., compress_after => INTERVAL '48 hours')`, which compresses chunks whose end is more than 48 hours old, and both policies were **paused** immediately with `alter_job(job_id, scheduled => false)` until the measurements in 4.3 were complete.

Before loading, one cutoff was recorded and stored in the table `compression_experiment`: **experiment start 3 October 2026 19:07:38 CAT, cutoff = start − 48 hours = 1 October 2026 19:07:38 CAT**. The same cutoff was used for both tables. `src/compress_load.py` then copied the cleaned data into each table **one chunk at a time** (29 source chunk ranges). After each chunk was fully loaded and committed, it was compressed manually with `compress_chunk()` if its end preceded the cutoff; every chunk ends by 29 September 02:00, so all 29 qualified. Each compression was timed separately from loading.

Measure	energy_time_compressed	energy_meter_compressed
Chunks loaded / compressed	29 / 29	29 / 29
Total load time	264.1 s	264.4 s
Total compression time	59.2 s	91.5 s
Average compression time per chunk	2.04 s	3.16 s

Verification. The same 29 chunk ranges are compressed in both tables (`same_ranges_compressed = t`), both policies were still paused, and the data checks from 3.1 match the baseline exactly in both tables: 10,643,735 rows, 4,000 meters, 2026-09-01 00:00 to 2026-09-28 23:45 CAT, sum of power 15,414,817.387793 kW and sum of energy 3,853,704.346815 kWh. All five tables were kept.

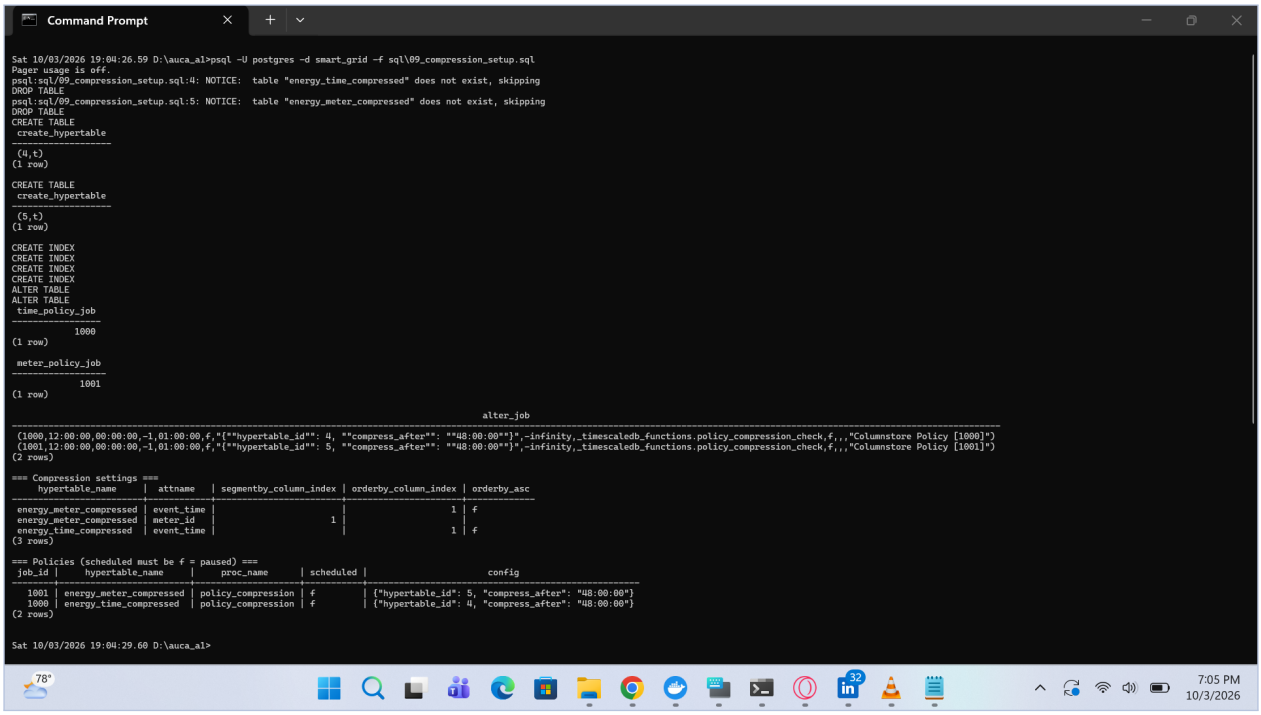


Figure 4.1-1: Both compressed hypertables, indexes, compression settings (ordering and segmentation) and the two paused 48-hour policies

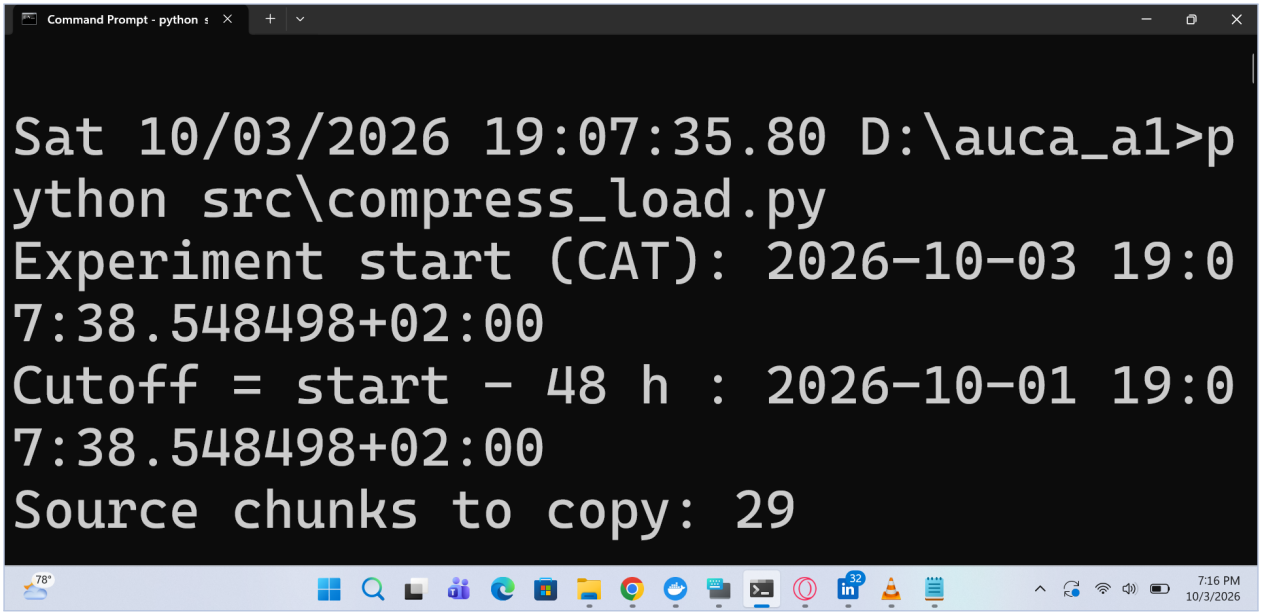


Figure 4.1-2: Experiment start and the single cutoff (start minus 48 hours) used for both tables

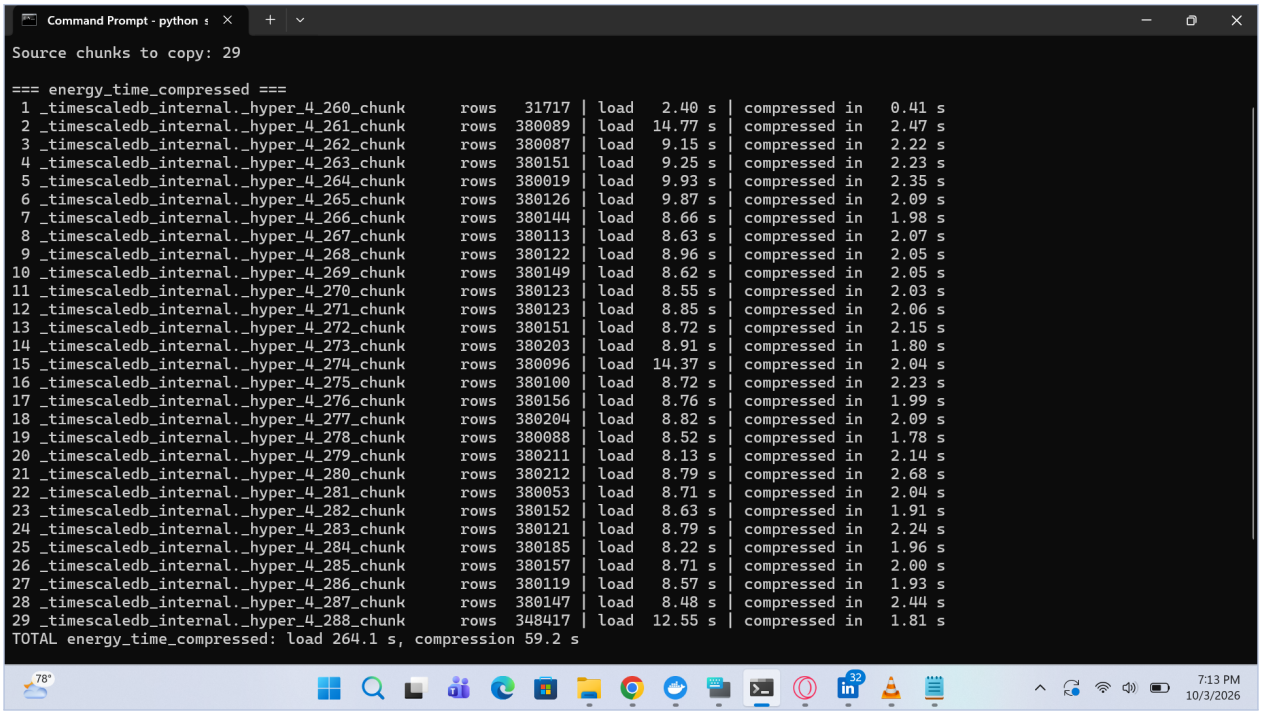


Figure 4.1-3: energy_time_compressed loaded chunk by chunk; each compression timed separately: load 264.1 s, compression 59.2 s

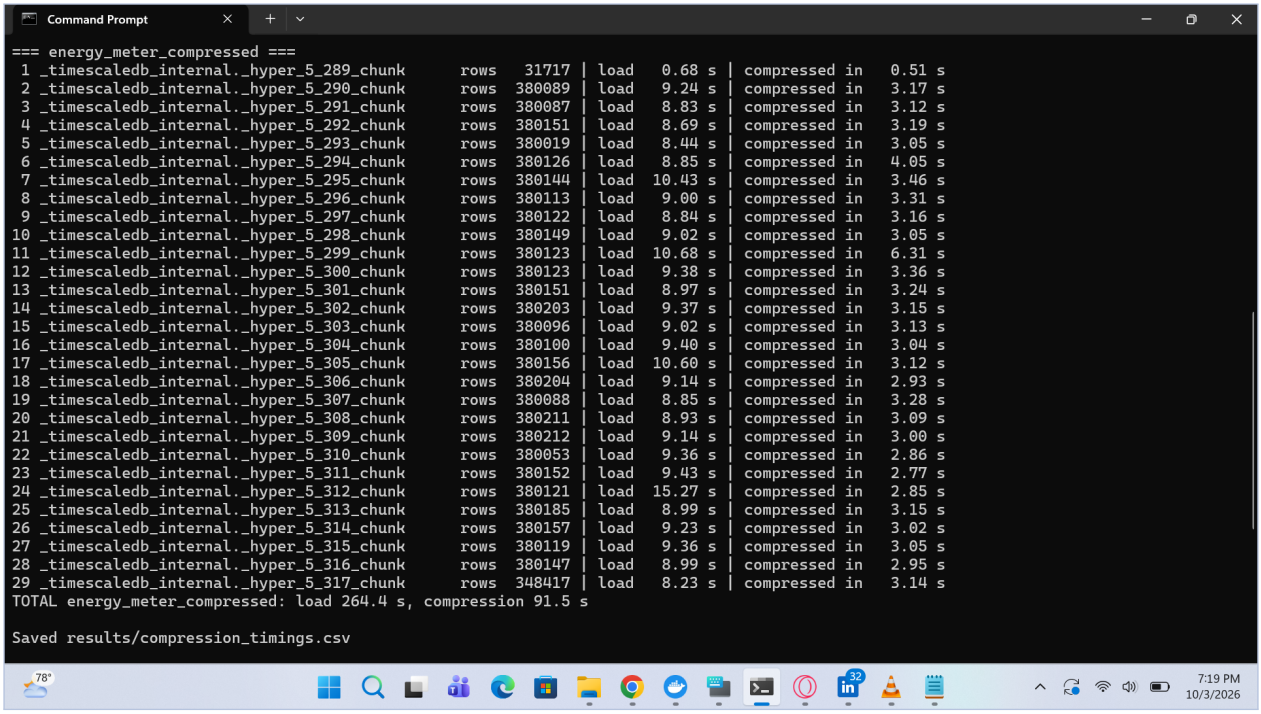


Figure 4.1-4: energy_meter_compressed loaded chunk by chunk: load 264.4 s, compression 91.5 s

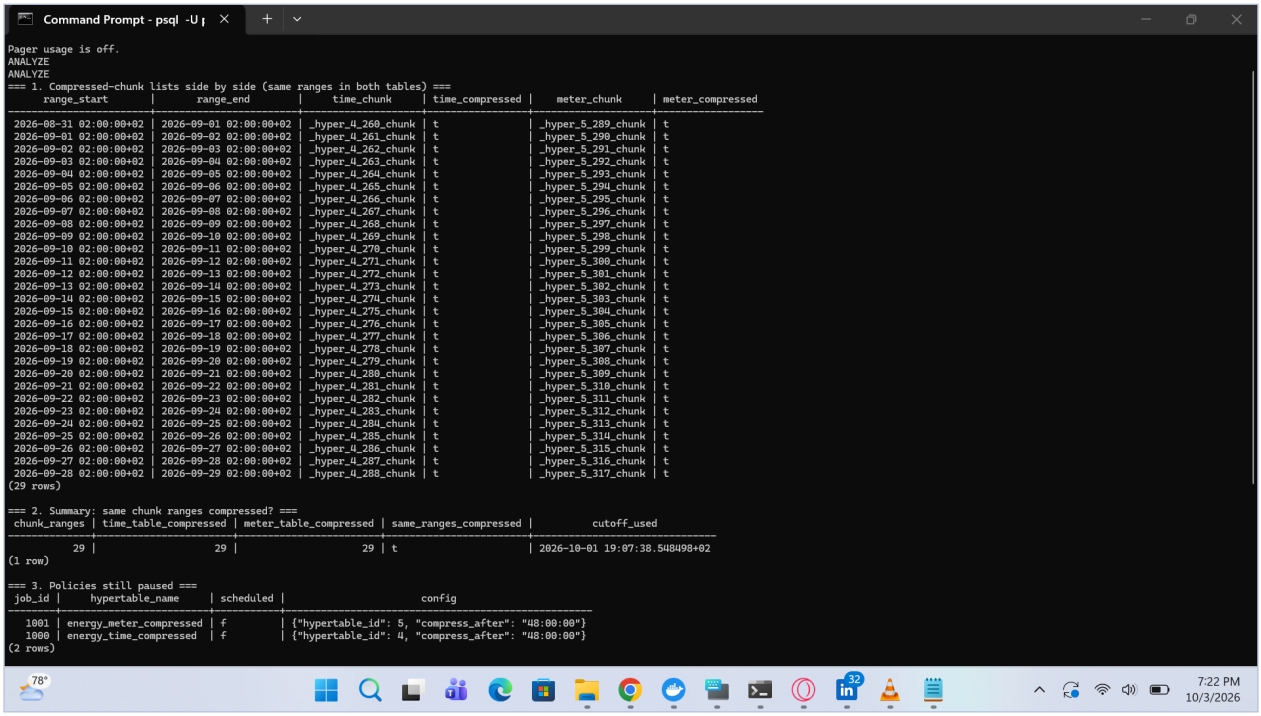


Figure 4.1-5: Compressed-chunk lists side by side: the same 29 ranges compressed in both tables, cutoff used, policies still paused

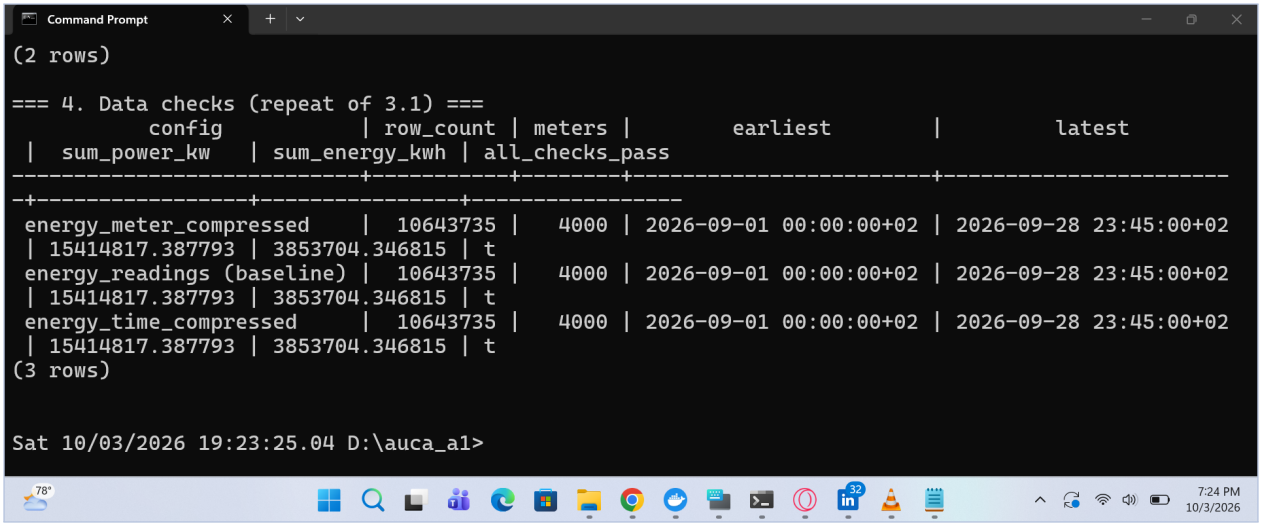


Figure 4.1-6: Data checks repeated from 3.1: both compressed tables match the baseline

4.2 Measure storage and compression cost

The baseline is the uncompressed table with the same chunk interval, **energy_readings** (1 day). Sizes come from `hypertable_detailed_size()`, which includes every chunk, compressed and uncompressed, and its indexes. Compression ratio = baseline total ÷ compressed total; percentage saved = 100 x (baseline − compressed) ÷ baseline; compression cost = the sum of the measured chunk compression times, excluding loading.

Configuration	Data MB	Index MB	TOAST MB	Total MB	Ratio	Saved	Compressed chunks	Rows in compressed chunks	Cost
energy_readings (baseline)	950.9	392.8	0.0	1,343.7	1.00	0 %	n/a	n/a	n/a
energy_time_ compressed	15.0	0.9	422.8	**438.7**	**3.06**	**67.35 %**	29 of 29	100.00 %	**59.2 s**
energy_meter_ compressed	26.8	5.5	469.4	**501.6**	**2.68**	**62.67 %**	29 of 29	100.00 %	**91.5 s**

The percentage of rows in compressed chunks was calculated from the original row counts of those chunk ranges in the cleaned table (10,643,735 of 10,643,735 = 100.00 %). Compressed data is stored mainly in TOAST as columnar arrays, which is why the TOAST column holds most of the compressed size.

Observation and explanation. Time ordering without segmentation achieved the better ratio (3.06 against 2.68) at lower cost (59.2 s against 91.5 s). Without segmentation, TimescaleDB packs consecutive rows into full batches of up to 1,000 rows. With segmentation by meter_id, each daily chunk is split into 4,000 separate groups of only about 95 readings each, so there are many more, much smaller batches; each batch carries its own metadata (minimum and maximum values and headers) and short arrays compress less efficiently, so the segmented table is about 63 MB larger and takes about 55 % longer to compress. TimescaleDB's own "before" size (1,459 MB) is slightly larger than the baseline (1,344 MB) because it measures the freshly loaded chunks, including space not yet reclaimed; the ratio uses the baseline table, as specified.

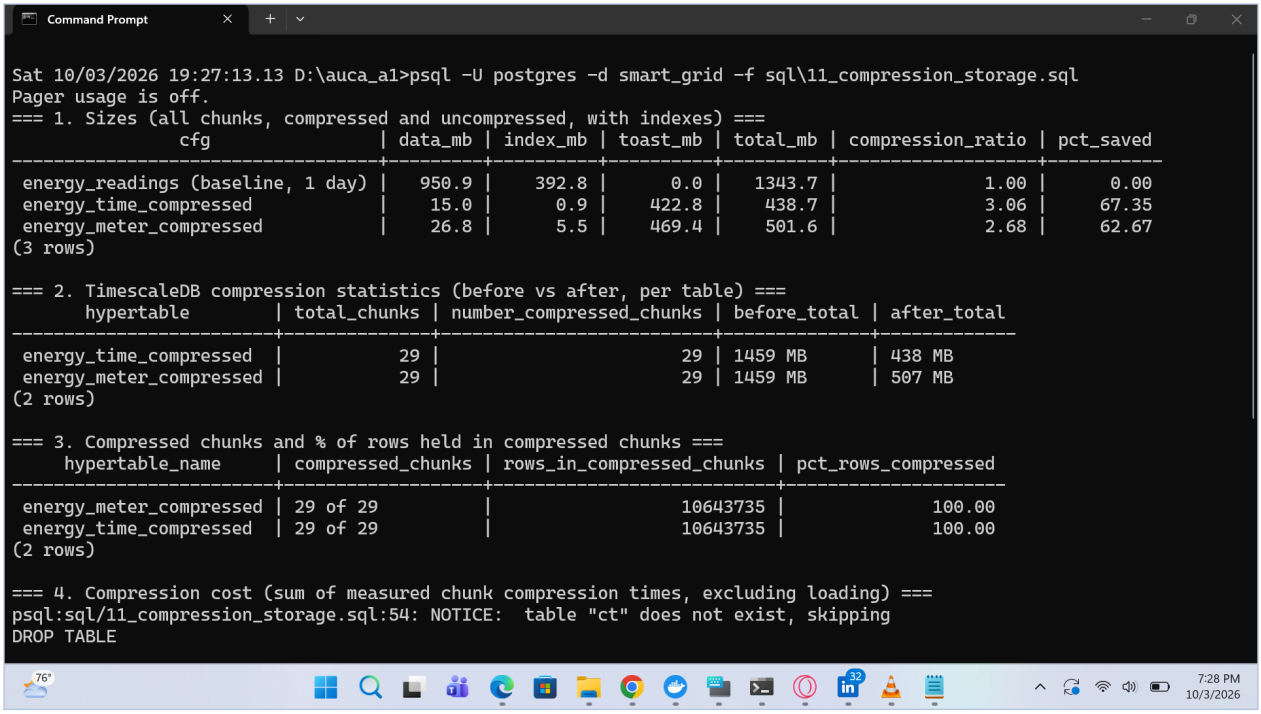


Figure 4.2-1: Sizes, compression ratios and percentage saved; TimescaleDB before/after statistics; compressed chunks and percentage of rows

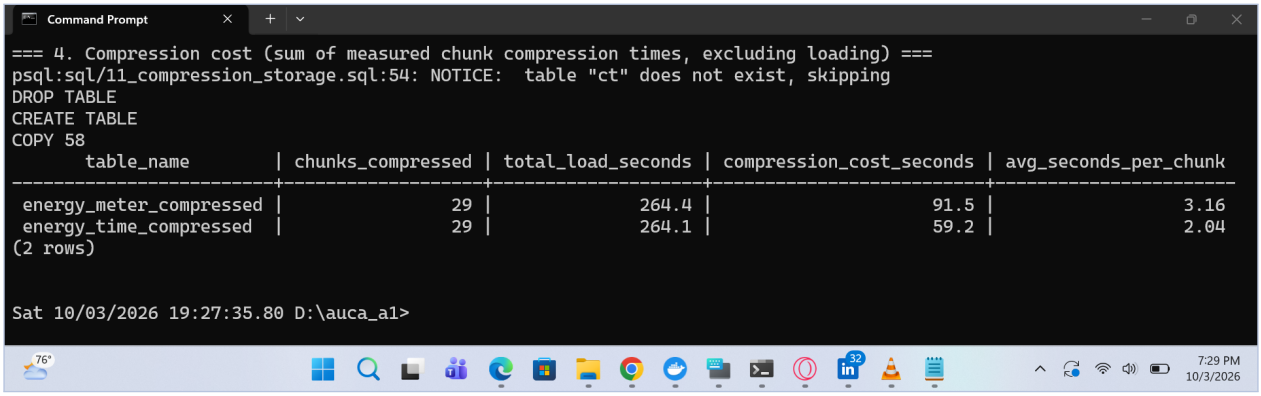


Figure 4.2-2: Compression cost: sum of measured chunk compression times, excluding loading

4.3 Repeat the query comparison

Q1 to Q5 were run on all **five** configurations in one session with the same correctness checks and timing procedure as 3.2 (one warm-up and five measured executions each, warm cache, `time.perf_counter()`, Python 3.13.6 + psycopg2 2.9.13, machine Erneste). **Session:** 19:31:05 to 19:36:56 CAT on 3 October 2026, **350.6 s**. All 25 results matched the baseline within 0.001.

Query	energy_readings (baseline)	energy_readings_ 3h	energy_readings_ week	energy_time_ compressed	energy_meter_ compressed
Q1	352.7 (346.9–386.8)	285.8 (271.1–307.8)	354.9 (329.9–361.6)	**155.7** (154.5–167.9)	1,845.5 (1,674.0–2,232.7)
Q2	942.9 (835.4–1,337.1)	1,031.1 (985.3–1,137.3)	1,129.0 (1,045.4–1,654.9)	564.3 (541.0–678.8)	**481.2** (390.1–846.0)
Q3	3,934.8 (3,567.3–6,415.3)	16,094.7 (15,788.2–17,387.6)	2,936.2 (2,869.0–3,877.1)	**718.5** (588.0–1,413.5)	741.5 (639.9–773.4)
Q4	5.6 (5.2–6.7)	16.7 (14.7–19.6)	**3.9** (3.7–4.1)	218.4 (192.4–242.7)	6.8 (5.4–7.4)

Query	energy_readings (baseline)	energy_readings_ 3h	energy_readings_ week	energy_time_ compressed	energy_meter_ compressed
Q5	2,265.5 (2,237.0–2,470.6)	2,617.4 (2,354.9–7,996.0)	2,354.6 (2,169.3–3,215.5)	**677.2** (561.6–2,059.4)	1,181.3 (972.1–1,482.6)

Values are median milliseconds with (minimum–maximum) of the five measured runs.

Percentage improvement = 100 x (baseline median − compressed median) ÷ baseline median, using the uncompressed one-day table as the baseline; positive values mean faster execution:

Query	Baseline (ms)	Time-compressed (ms)	Improvement	Meter-compressed (ms)	Improvement
Q1	352.7	155.7	**+55.9 %**	1,845.5	**−423.2 %**
Q2	942.9	564.3	+40.2 %	481.2	**+49.0 %**
Q3	3,934.8	718.5	**+81.7 %**	741.5	+81.2 %
Q4	5.6	218.4	**−3,780.7 %**	6.8	−20.1 %
Q5	2,265.5	677.2	**+70.1 %**	1,181.3	+47.9 %

Plan evidence (Q1). On energy_time_compressed, the plan reads 2 compressed chunks through `ColumnarScan` with a vectorised filter and `VectorAgg`, and uses the batch metadata index (`_ts_meta_v2_ first_event_time`) to open only the batches whose time range overlaps 21 September (16 and 175 batches), so execution took 487 ms with about 2,700 buffers. On energy_meter_compressed, every batch covers one meter for a whole day, so time metadata cannot exclude batches: about 2,000 small batches per chunk were decompressed, 174,272 rows were removed by the filter, and execution took 2,119 ms with about 48,400 buffers, explaining the −423 % result.

After the measurements, both compression policies (jobs 1000 and 1001) were **resumed** with `alter_job(job_ id, scheduled => true)` and show `scheduled = t`.

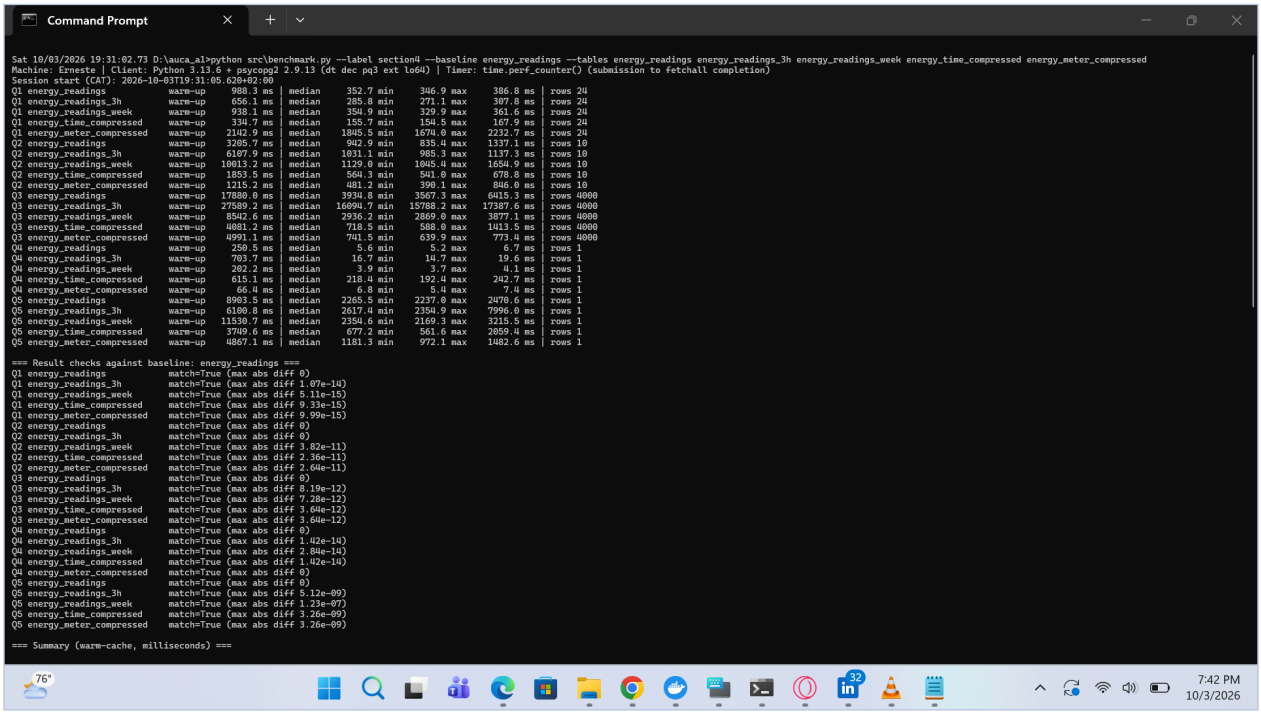


Figure 4.3-1: Benchmark of all five configurations: per-query lines and result checks against the baseline

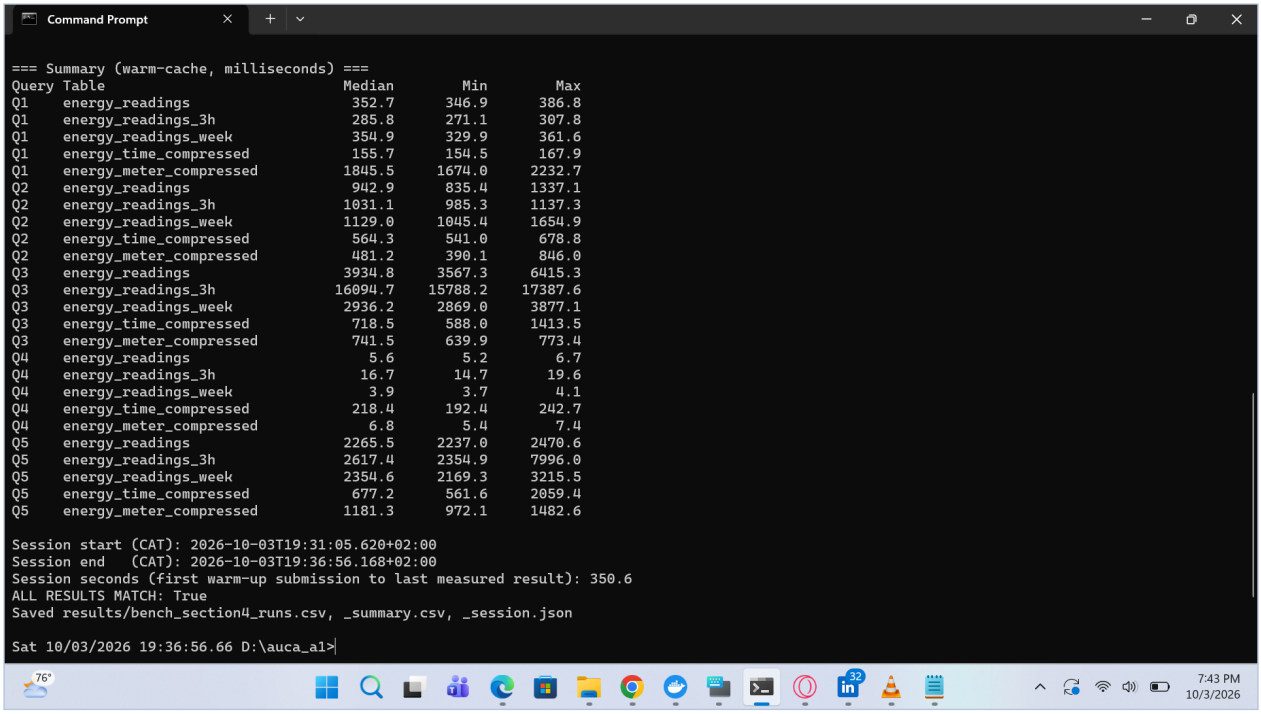


Figure 4.3-2: Timing comparison of all five configurations, session seconds and all results matching

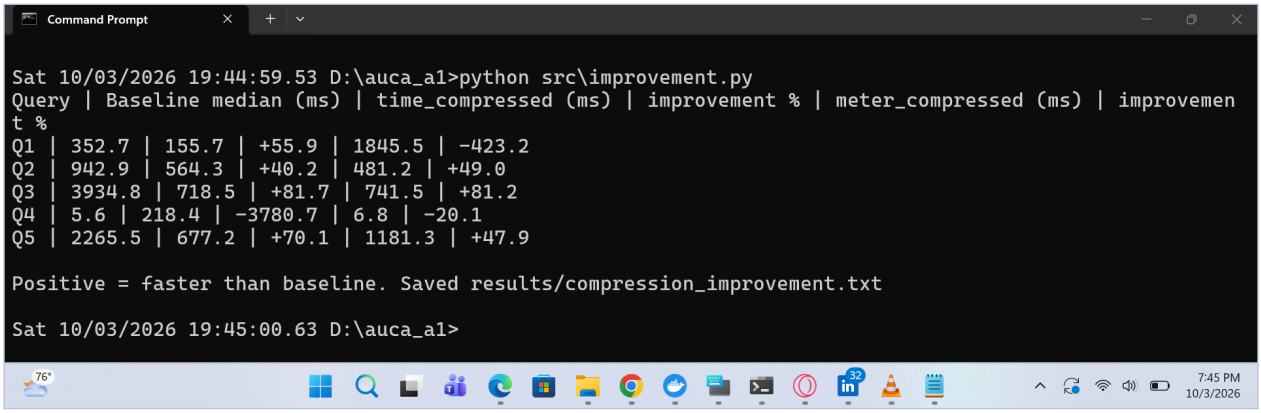


Figure 4.3-3: Percentage improvement of both compressed tables over the baseline

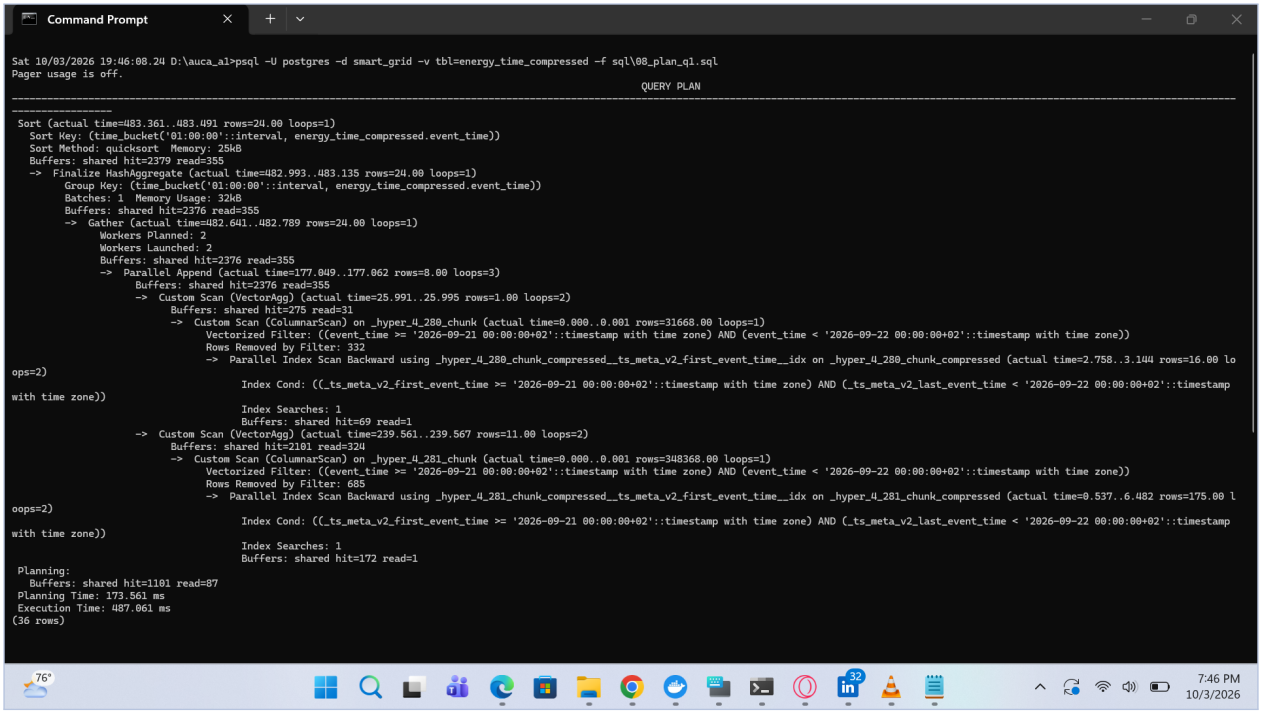


Figure 4.3-4: Actual execution plan (Q1) on energy_time_compressed: ColumnarScan with vectorised filter and aggregation over 2 chunks

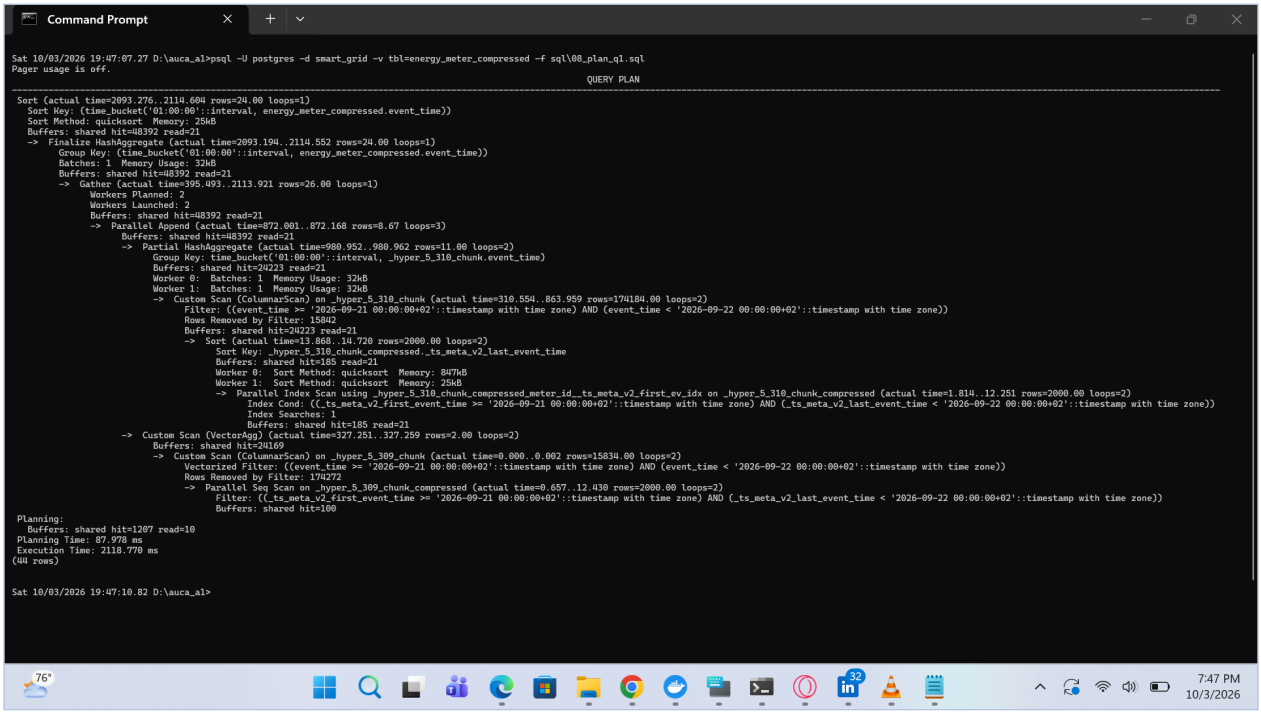


Figure 4.3-5: Actual execution plan (Q1) on energy_meter_compressed: thousands of small per-meter batches decompressed and filtered

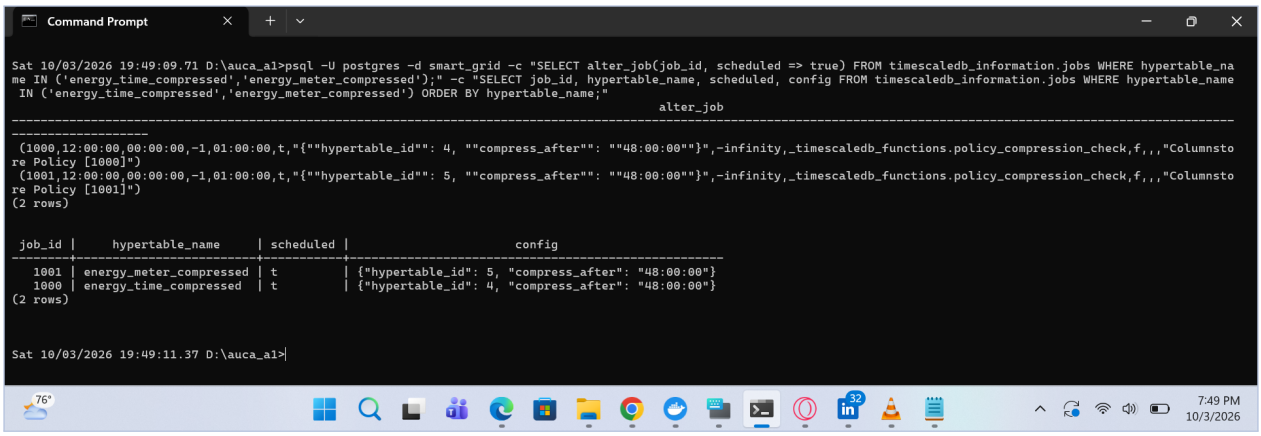


Figure 4.3-6: Compression policies resumed (scheduled = t)

4.4 Select a storage configuration

Effect of meter segmentation on the single-meter workload (Q4). Segmenting by meter_id stores each meter's readings in its own compressed batches, so the query decompresses only meter 1000000000's batches; Q4 stayed at 6.8 ms. Without segmentation, every batch in the 8 relevant chunks (about 3 million rows) must be decompressed and filtered to find one meter, so Q4 took 218.4 ms, 32 times slower.

Effect on workloads across all meters (Q1, Q2, Q3, Q5). Both strategies read far less data than the baseline because compressed columns are about a third of the size. However, segmentation splits each day into 4,000 small batches of about 95 rows, so cross-meter queries pay per-batch decompression and merge overhead. This was worst for Q1 (one day, all meters, grouped by hour), where the time-ordered table was about 12 times faster (155.7 ms against 1,845.5 ms), and on the full scan Q5 (677.2 ms against 1,181.3 ms). Segmentation helped only Q2 (481.2 ms against 564.3 ms).

Recommendation

Choice: 1-day chunks with **time-ordered compression and no segmentation** (energy_time_compressed).

Evidence: it had the best compression ratio (3.06, 67.35 % saved, 438.7 MB against 1,343.7 MB for the baseline), the lowest compression cost (59.2 s against 91.5 s for 29 chunks) and the fastest times on Q1, Q3 and Q5, including 81.7 % faster than the baseline on Q3; daily chunks were never the slowest configuration in Section 3.

Trade-off: single-meter lookups become slower (Q4: 218.4 ms instead of 5.6 ms uncompressed or 6.8 ms with meter segmentation). That cost is acceptable because the utility's reporting is dominated by network, regional and daily aggregates, and per-meter history can be served from the daily per-meter continuous aggregate (Section 6). If customer-facing single-meter queries became the main workload, meter segmentation would be the better choice, at about 14 % more storage and 55 % more compression time.

5 Analyse demand, regions, reporting quality and abnormal consumption

5.1 Demand patterns over time

`src/analysis_5_1.py` queried the cleaned table `energy_readings` and grouped it by **CAT** (Africa/Kigali) calendar day, by seven-day period beginning 1, 8, 15 and 22 September, and by 15-minute time of day. Network energy over the 28 days was **3,853,704.347 kWh**, from about 380,100 readings every day (the count is constant, so differences between days are not caused by missing data).

Daily and weekly energy

Weekday totals rose from **141,195.2 kWh** (Tuesday 1 September) to **152,464.2 kWh** (Monday 28 September). Weekend totals were between **111,680.5 and 118,993.6 kWh**. The average weekend day (115,284 kWh) is about **21 % lower** than the average weekday (146,572 kWh). Both weekday and weekend totals increase from one week to the next:

Seven-day period	Energy (kWh)	Change from previous week	Readings
1 to 7 September	934,199.1	n/a	2,660,768
8 to 14 September	953,715.0	+19,515.9 kWh (+2.1 %)	2,660,984
15 to 21 September	973,230.0	+19,515.0 kWh (+2.0 %)	2,661,000
22 to 28 September	992,560.3	+19,330.3 kWh (+2.0 %)	2,660,983

Demand grows by about **19,500 kWh (2.0 to 2.1 %) every week**, an almost constant increment, so the trend is steady growth rather than a one-off jump.

Time-of-day profile and highest demand

Network demand per 15-minute interval is the sum of power (kW) over all meters. The three highest intervals of the whole period were all on **Monday 28 September**:

Rank	Interval start (CAT)	Network demand (kW)	Readings
1	28 Sep 13:00	**7,993.691**	3,965
2	28 Sep 12:45	7,943.947	3,969
3	28 Sep 13:15	7,943.636	3,952

The average profiles differ clearly between weekdays and weekends. On **weekdays** demand is lowest at night (about 4,400 kW), climbs to about 6,600 kW by 08:00, reaches its main peak of **7,641.6 kW at 13:15**, eases to about 7,050 kW in the late afternoon and has a second, lower peak of about 7,100 kW at 18:00 to 18:45. On **weekends** demand is lowest at night (about 3,550 kW), shows a morning bump of about 5,400 kW at 08:00, stays between about 5,050 and 5,350 kW during the day and peaks at **5,992.1 kW at 18:45**.

Explanation

- **Weekend drop.** The weekday midday level and the difference between weekdays and weekends come mainly from the 1,010 commercial and 406 industrial meters (5.2): businesses operate on weekdays, so on weekends their load falls and the profile is shaped by the 2,584 residential meters.

- **Weekday midday peak.** Commercial business-hours load is added to a constant industrial load, giving one broad peak around 13:00.
- **Weekend evening peak.** Residential evening use (cooking, lighting, appliances) dominates when commercial load is low, so the highest weekend demand is at 18:45.
- **Steady growth.** The weekly increase of about 2 % is steady and should be monitored for capacity planning.

Implications for the utility

- Capacity must be planned for the weekday midday peak (about 8 MW and rising about 2 % per week), not for the average.
- Storage should charge overnight and at weekends, when demand is low, and discharge between 12:00 and 14:00 on weekdays to reduce the peak.
- Network maintenance is least disruptive at night and at weekends.

Power (kW) and energy (kWh)

kW is power, the instantaneous rate at which energy is used; the highest kW value sets the capacity the network must be able to supply. **kWh is energy**, power multiplied by time; it is what customers are billed for and what the utility must purchase. They are linked: a demand of 7,993.7 kW sustained for one 15-minute interval is 7,993.7 x 0.25 = **about 1,998 kWh**.

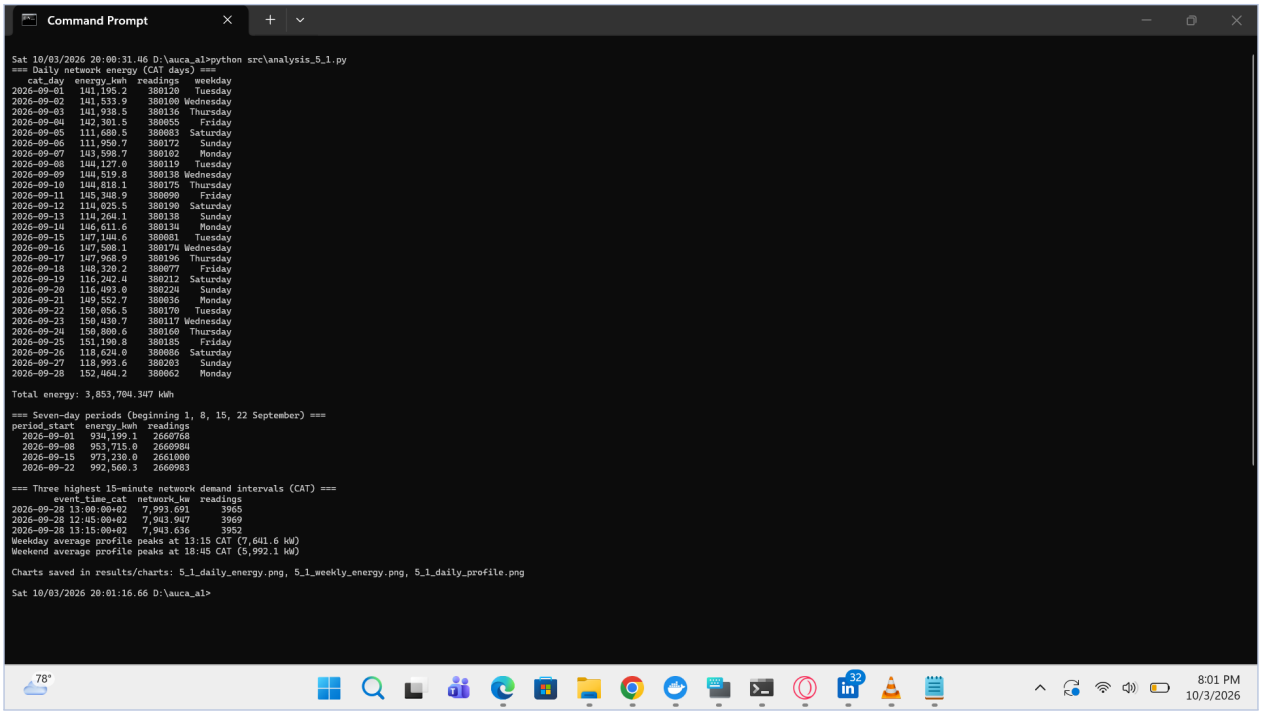


Figure 5.1-1: Output of src/analysis_5_1.py: daily energy by CAT day, seven-day periods and the three highest 15-minute demand intervals

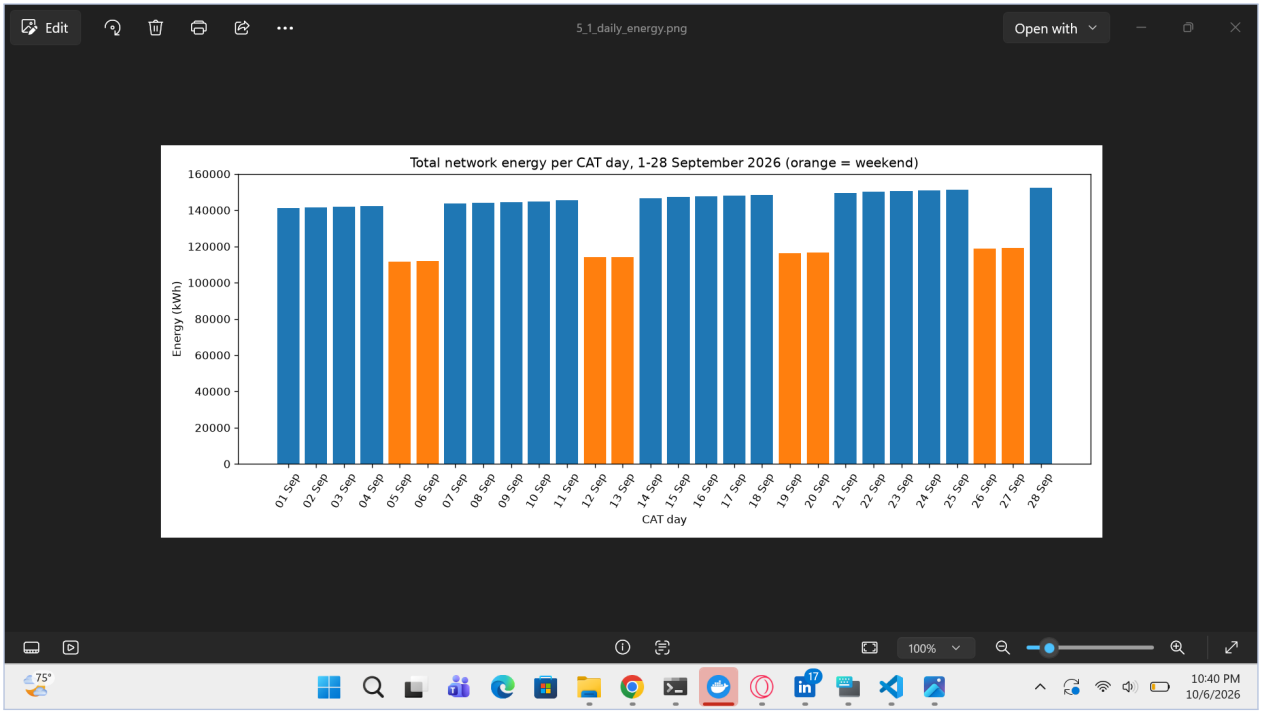


Figure 5.1-2: Total network energy per CAT day, 1 to 28 September 2026 (weekends in orange)

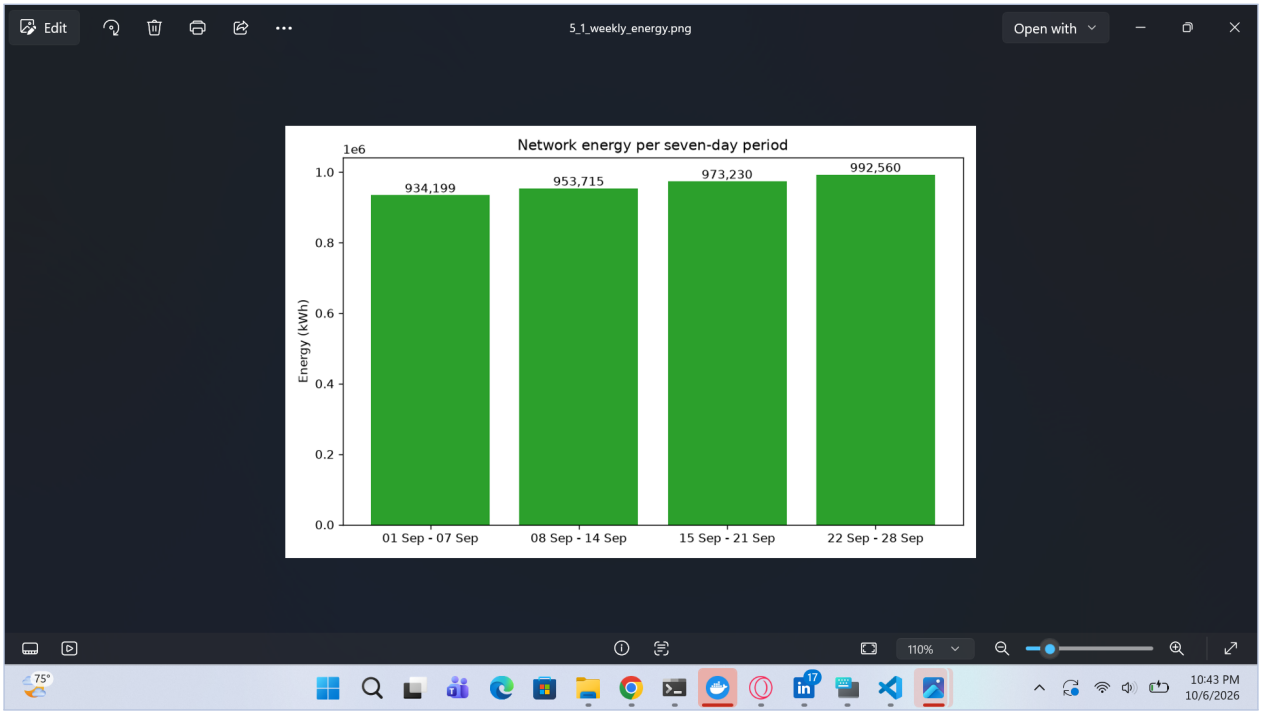


Figure 5.1-3: Network energy per seven-day period

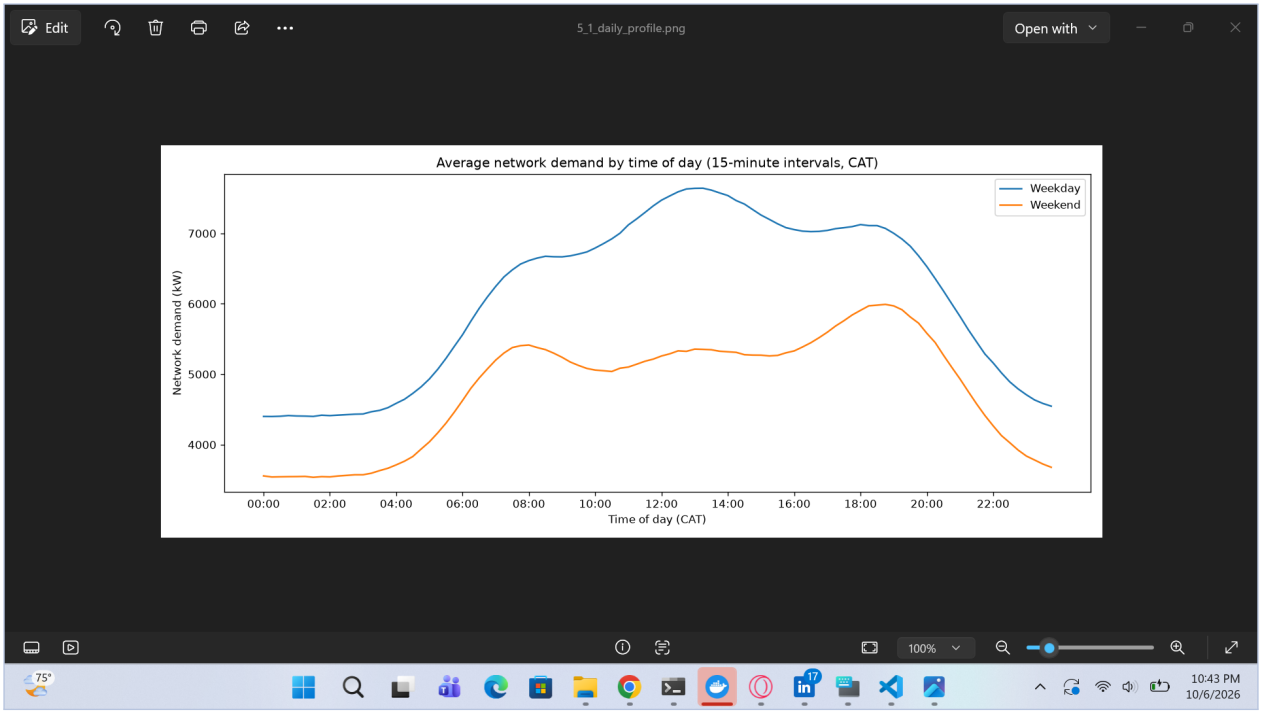


Figure 5.1-4: Average network demand by time of day for weekdays and weekends (15-minute intervals, CAT)

5.2 Regions and customer types

Because every region has the same number of meters (400), total energy by region can be compared directly; the fairer measure is **energy per registered meter**, which `src/analysis_5_2.py` calculated for all 28 days. The check at the end of the script confirmed total energy of 3,853,704.347 kWh and 4,000 meters.

Region	Meters	Total energy (kWh)	kWh per meter	Residential	Commercial	Industrial
Region 1	400	383,539.8	958.8	247	116	37
Region 2	400	378,448.9	946.1	263	98	39
Region 3	400	392,376.6	980.9	260	94	46
Region 4	400	324,404.5	**811.0** (lowest)	277	94	29
Region 5	400	411,040.4	1,027.6	258	94	48
Region 6	400	346,512.8	866.3	263	105	32
Region 7	400	362,288.5	905.7	262	103	35
Region 8	400	503,475.8	**1,258.7** (highest)	234	105	61
Region 9	400	400,467.2	1,001.2	259	96	45
Region 10	400	351,149.8	877.9	261	105	34

Energy per meter ranges from **811.0 kWh (Region 4) to 1,258.7 kWh (Region 8)**, a ratio of 1.552: the average Region 8 meter used 55 % more energy than the average Region 4 meter. The customer types differ far more:

Customer type	Meters	Share of meters	Total energy (kWh)	Share of energy	kWh per meter
Industrial	406	10.2 %	2,220,672.0	57.6 %	**5,469.6**
Commercial	1,010	25.3 %	825,475.2	21.4 %	817.3
Residential	2,584	64.6 %	807,557.2	21.0 %	312.5

Explanation. One industrial meter uses about **17.5 times** the energy of a residential meter (5,469.6 / 312.5) and **6.7 times** that of a commercial meter (5,469.6 / 817.3). Industrial meters are only 10.2 % of meters but use 57.6 % of the energy. Regional differences therefore follow customer mix. Region 8 has the most industrial meters (61) and the fewest residential meters (234); Region 4 has the fewest industrial meters (29) and the most residential meters (277). The ranking of regions by industrial meter count (8, 5, 3, 9, 2, 1, 7, 10, 6, 4) is almost the same as the ranking by energy per meter (8, 5, 9, 3, 1, 2, 7, 10, 6, 4); only the neighbouring pairs 3/9 and 2/1 are swapped, because commercial and residential counts also contribute.

Implication. Regional totals should not be read as efficiency or performance differences; regions should be compared within the same customer type. Network planning, loss monitoring and demand-response programmes should give priority to the industrial-heavy regions (8, 5, 3 and 9), where a small number of large customers determines the load.

Differences that remain after normalisation. Dividing each total by the registered meters of its group removes the effect of group size, yet clear differences remain. Between regions, energy per meter still differs by 55 % (Region 8 against Region 4); between customer types, by a factor of 17.5 (industrial against residential). The regional differences remain because the regions have different customer mixes, and the differences between types remain because the types genuinely use very different amounts of energy. The implication is that planning should be based on customer type and on the number of industrial meters per region, not on regional averages.

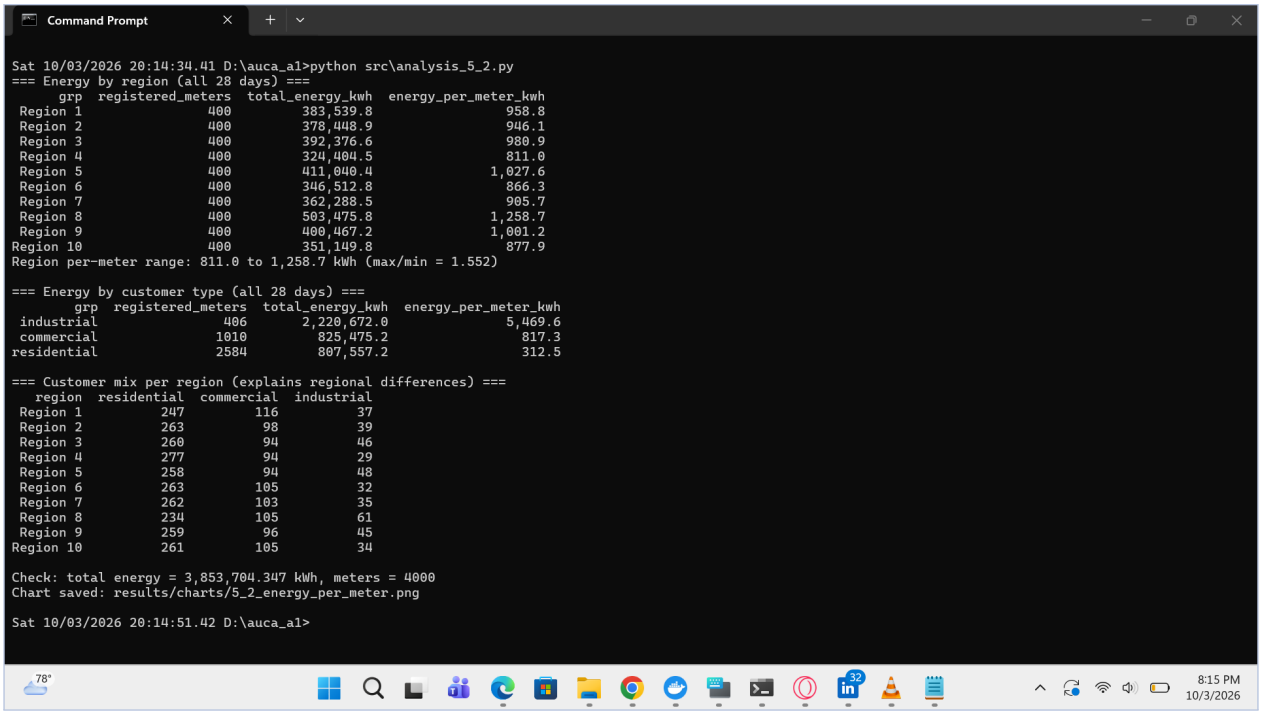


Figure 5.2-1: Output of src/analysis_5_2.py: energy per registered meter by region and customer type, and customer mix per region

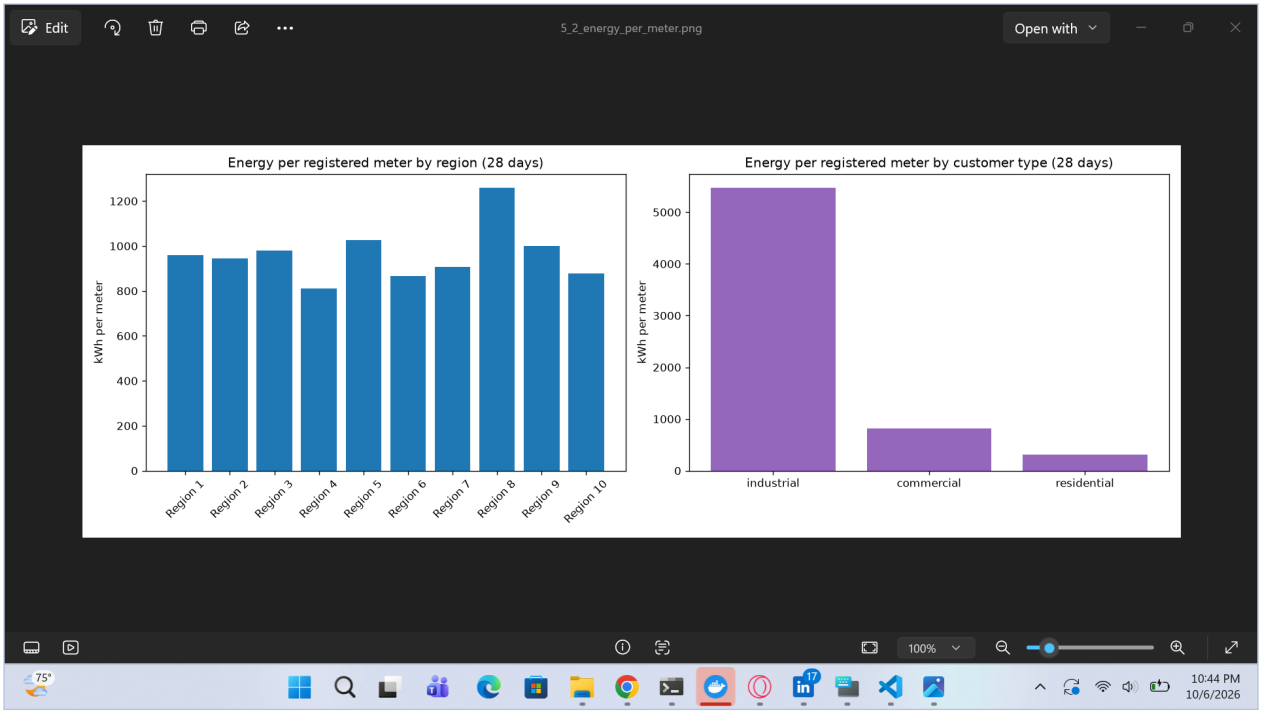


Figure 5.2-2: Energy per registered meter by region (left) and by customer type (right), 28 days

5.3 Reporting quality and late data

Completeness

Completeness = observed unique meter/event_time pairs ÷ planned readings, where planned readings = registered meters x 28 days x 96 intervals per day (400 x 28 x 96 = 1,075,200 per region and 4,000 x 28 x 96 = 10,752,000 for the network). No duplicate meter/event_time pairs remained after the cleaning in Section 2, so each row is one unique observed pair. Missing readings are counted as **missing**, not as zero.

Region	Planned readings	Observed pairs	Completeness
Region 1	1,075,200	1,064,513	99.0060 %
Region 2	1,075,200	1,064,419	98.9973 %
Region 3	1,075,200	1,064,327	98.9887 %
Region 4	1,075,200	1,064,441	98.9993 %
Region 5	1,075,200	1,064,290	98.9853 %
Region 6	1,075,200	1,064,118	**98.9693 %** (lowest)
Region 7	1,075,200	1,064,237	98.9804 %
Region 8	1,075,200	1,064,587	**99.0129 %** (highest)
Region 9	1,075,200	1,064,388	98.9944 %
Region 10	1,075,200	1,064,415	98.9969 %
Network	**10,752,000**	**10,643,735**	**98.9931 %**

About **1.01 %** of planned readings (108,265) are missing, spread almost evenly: the regions differ by only 0.044 percentage points (98.9693 to 99.0129 %), so reporting problems are network-wide and not concentrated in one region.

Effect of late arrivals on the daily total for 21 September

The 21 September total (CAT day) was calculated twice: with only the readings that had **arrived by 00:00 on 22 September**, which is what a report produced at midnight would show, and with **all arrivals**, the final value.

Measure	Arrived by 00:00 on 22 Sep	All arrivals (final)
Readings	379,505	380,036
Energy (kWh)	149,400.545	149,552.723
Planned readings for the day	384,000	384,000
Never sent (missing)	n/a	3,964 (1.03 %)

The midnight total is **152.178 kWh lower, 0.1018 % of the final total**. The 531 late readings (379,505 to 380,036) all belong to the last three hours of the day: 46 readings (11.585 kWh) for 21:00, 186 (66.474 kWh) for 22:00 and 299 (74.119 kWh) for 23:00, so the later the hour, the more readings arrive after midnight. In contrast, the **3,964 readings that never arrived** (1.03 % of 384,000) represent about 1,560 kWh (at the day's average of 0.394 kWh per reading), **about ten times larger** than the effect of late arrival. The completeness of the data matters more for the accuracy of a daily total than its timeliness.

Implication. A same-day total produced at midnight should be labelled provisional, and dashboards should display the completeness percentage next to every total. Missing readings must not be treated as zero consumption.

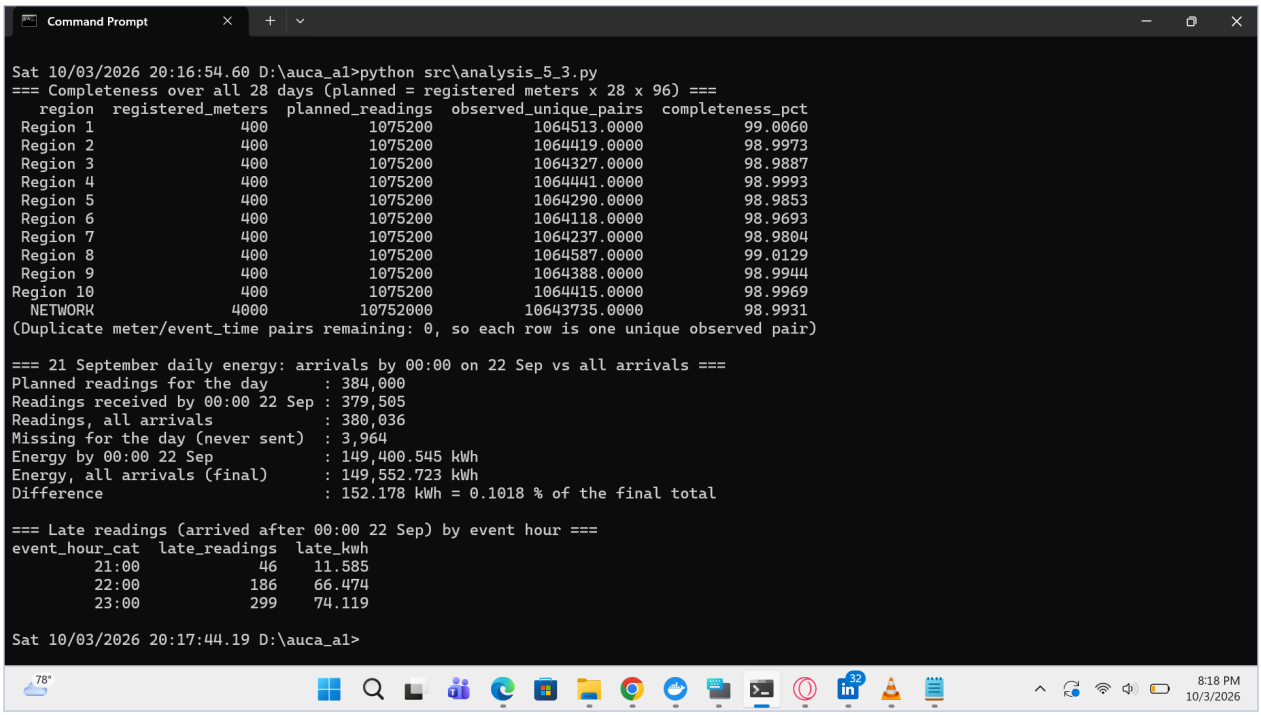


Figure 5.3-1: Output of src/analysis_5_3.py: completeness by region and network, 21 September totals at midnight against all arrivals, and late readings by event hour

5.4 Detecting abnormal consumption

Rule

`src/anomaly_5_4.py` compares each reading with what is normal **for that meter**. A baseline is built as the **median** power of the same meter in the same 15-minute slot (CAT) and the same day type (weekday or

weekend); each meter, slot and day type has 5 to 20 baseline samples. Then:

```
ratio = power_kw / median(power_kw)   (same meter, same 15-minute slot, same day type)
flag HIGH if ratio >= 2.0
flag LOW  if ratio <= 0.4
```

Ratios of normal readings are about 0.8 to 1.2 (random noise of about ±15 % and a trend of about ±4 %), so a ratio of 2.0 or more (at least double the usual value) or 0.4 or less (at most 40 % of the usual value) is far outside normal variation. The median is used instead of the mean because it is not distorted by the abnormal readings themselves, and comparing with the same meter, slot and day type removes the effects of customer size, time of day and weekend patterns.

Result

The rule checked **10,643,735** readings in 224.6 s and raised **474 flags (236 high, 238 low)**. The injected test events (a ground truth used only for evaluation, never in the rule) were 40 events, each lasting 12 consecutive 15-minute readings (3 hours): 20 demand spikes and 20 unusual drops:

Evaluation measure	Result
Injected events detected	**40 of 40**
Injected events missed	0 of 40
False flags (flags outside injected intervals)	**0 = 0.000000 %** of 10,643,261 readings outside the injected intervals
Meters affected by false flags	0 / 4,000 = 0.00 %

Every flag lies inside an injected event: 34 events have all 12 readings flagged and **six events have 11 of 12**, because one reading in each of those intervals never arrived ($34 \times 12 + 6 \times 11 = 474$ flags; the 236 high flags are 20 spikes, four of them one reading short, and the 238 low flags are 20 drops, two of them one reading short). The flag counts therefore equal the readings that exist, and no event was missed because of the missing data.

Missed events and false alarms. No injected event was missed and no reading was falsely flagged, so there is nothing to explain for this dataset. This is plausible because the injected events moved consumption well outside the thresholds (the high flags shown have ratios of 2.40 to 3.15), while normal readings stay at 0.8 to 1.2 times the baseline. The rule was applied to the cleaned readings first and the labels from `anomaly_events()` were used only afterwards for evaluation; each flag stored in `results/anomaly_flags.csv` identifies a meter_id and an event_time. On real data, an event that stayed between 0.4 and 2.0 times the usual level would be missed, and unusual but legitimate behaviour would raise false alarms.

Limitations in real operation

- Holidays, school terms and seasons change normal behaviour and would produce legitimate high or low ratios; the baseline needs day-type calendars.
- Real customers change behaviour permanently (new equipment, a new occupant); a rolling baseline is needed so that the median adapts.
- A flag shows an unusual reading but not its cause: a fault, theft, a faulty meter and a real change look alike and need field or data checks.
- Real anomalies are weaker and noisier than injected ones, so thresholds (2.0 and 0.4) should be tuned on labelled history, and several consecutive flags should be required before action to limit false alarms.

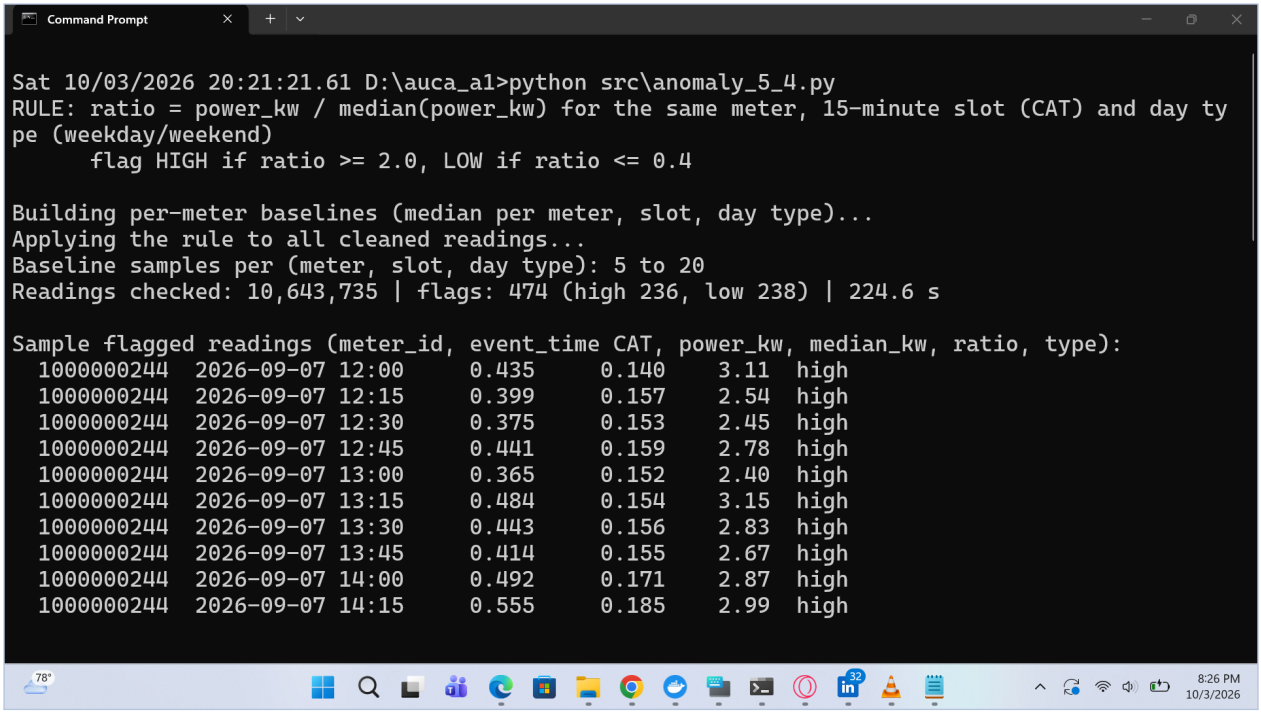


Figure 5.4-1: Output of src/anomaly_5_4.py: rule, baselines, readings checked, flags raised (474: 236 high, 238 low) and sample flagged readings

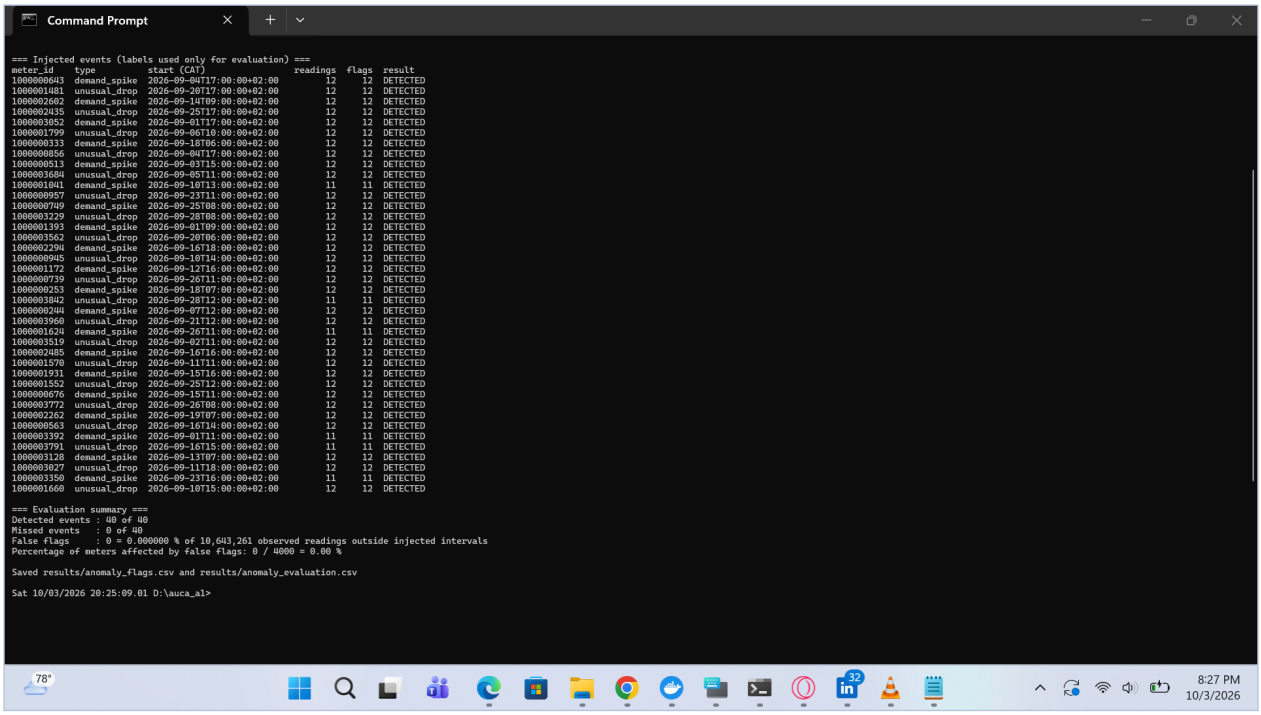


Figure 5.4-2: Evaluation against the 40 injected events: 40 of 40 detected, 0 missed, 0 false flags

5.5 Recommendation to management

To the Management Board. The 28-day analysis of 4,000 smart meters supports four decisions.

Operations. Demand peaks on weekdays around midday: the highest 15-minute interval was 7,993.7 kW on Monday 28 September at 13:00 CAT. Weekend energy is about 21 % lower and weekly energy is growing by about 2.1 %, so capacity planning should be based on the weekday midday peak and its growth. Regional differences (Region 8 at 1,258.7 kWh per meter against Region 4 at 811.0) come from customer mix, not

inefficiency: Region 8 has 61 industrial meters and Region 4 has 29. Industrial meters are 10.2 % of meters but 57.6 % of energy. The abnormal-consumption rule detected all 40 injected events with no false flags; it should run daily, with consecutive flags required before any field visit.

Storage. Keep one-day chunks with time-ordered compression and no segmentation. It reduced the table from 1,343.7 MB to 438.7 MB (ratio 3.06, 67.35 % saved) in 59.2 s, and made Q3 81.7 % and Q5 70.1 % faster. Single-meter lookups become slower (218.4 ms against 5.6 ms) and should be served from the daily per-meter aggregate.

Reporting. Use continuous aggregates for dashboards. The median dashboard load fell from 7,956.9 ms to 1,117.8 ms (7.1 times faster) for 18 MB of extra storage and a one-off 335.7 s refresh. Policies should run every 15 minutes over a three-day window. Dashboards must show completeness (98.99 % of planned readings arrived), label same-day totals as provisional (21 September was 0.10 % low at midnight) and never treat a missing reading as zero.

Forecasting. Adopt the Ridge regression model for next-day hourly regional demand. Its test error was MAE 4.549 kWh and RMSE 5.795 kWh, against 42.708 and 79.403 kWh for the previous-day baseline (89.3 % lower MAE). On Monday peaks it erred by 1.9 to 17.5 kWh where the baseline erred by up to 270 kWh. Run it shortly after midnight, retrain weekly and monitor its error.

Limits. The data are synthetic, cover four weeks and miss about 1 % of readings; holidays and real equipment faults have not been tested.

6 Continuous aggregates and data freshness

6.1 Create the continuous aggregates

`sql/12_caggs.sql` created three continuous aggregates on the cleaned hypertable `energy_readings`. Each stores a time bucket, a grouping key and three measures: `sum_energy_kwh`, `sum_power_kw` and `reading_count`. Sums and counts are stored instead of averages so that results can be combined correctly at any coarser level (an average of averages would be wrong when groups have different numbers of readings).

Aggregate	Bucket	Grouped by	Source
cagg_15min_region	15 minutes	region	energy_readings joined to meters
cagg_hour_region	1 hour (CAT)	region	energy_readings joined to meters
cagg_day_meter	1 day (CAT)	meter_id	energy_readings

All three were created `WITH NO DATA` and with **real-time aggregation switched off** (`materialized_only = true`), so each view returns only what has been materialised. The views were confirmed empty (0 rows each) before the first refresh. The hourly and daily buckets use the time zone `Africa/Kigali`, so a bucket is a CAT hour or CAT day, as in Section 5.

September was then loaded with a **manual refresh of the whole generated period** (1 September 00:00 to 29 September 00:00 CAT), because the refresh policies below only look back a few days from the current time and cannot populate September. The refreshes were timed:

Aggregate	Refresh time	Rows	Data	Index	Total size
cagg_15min_region	102.7 s	26,880	2,152 kB	2,088 kB	4.18 MB
cagg_hour_region	143.7 s	6,720	640 kB	608 kB	1.26 MB
cagg_day_meter	89.3 s	112,000	7,616 kB	5,272 kB	12.59 MB

Aggregate	Refresh time	Rows	Data	Index	Total size
All three	**335.7 s**	145,600	10,408 kB	7,968 kB	**18.03 MB**

The row counts agree with the grouping: 28 days x 96 intervals x 10 regions = 26,880; 28 x 24 x 10 = 6,720; 28 days x 4,000 meters = 112,000. The total size reported by the database is slightly larger than data plus index for the two region views (about 40 kB each) because it also includes small additional relation storage. All three aggregates together occupy **18.03 MB, about 1.3 % of the 1,343.7 MB** uncompressed table, so keeping them is cheap compared with the base data.

Refresh policies. `add_continuous_aggregate_policy` was added to each view with `start_offset` of 3 days, `end_offset` of 1 hour and a schedule interval of 15 minutes (jobs 1002, 1003 and 1004). The first scheduled runs started at 20:39 CAT on 3 October 2026 and all three show `Success` with 1 run and 0 failures. The end offset of one hour keeps the policy away from the newest data that is still being written; the start offset of three days lets late arrivals from the last three days be picked up on the next run.

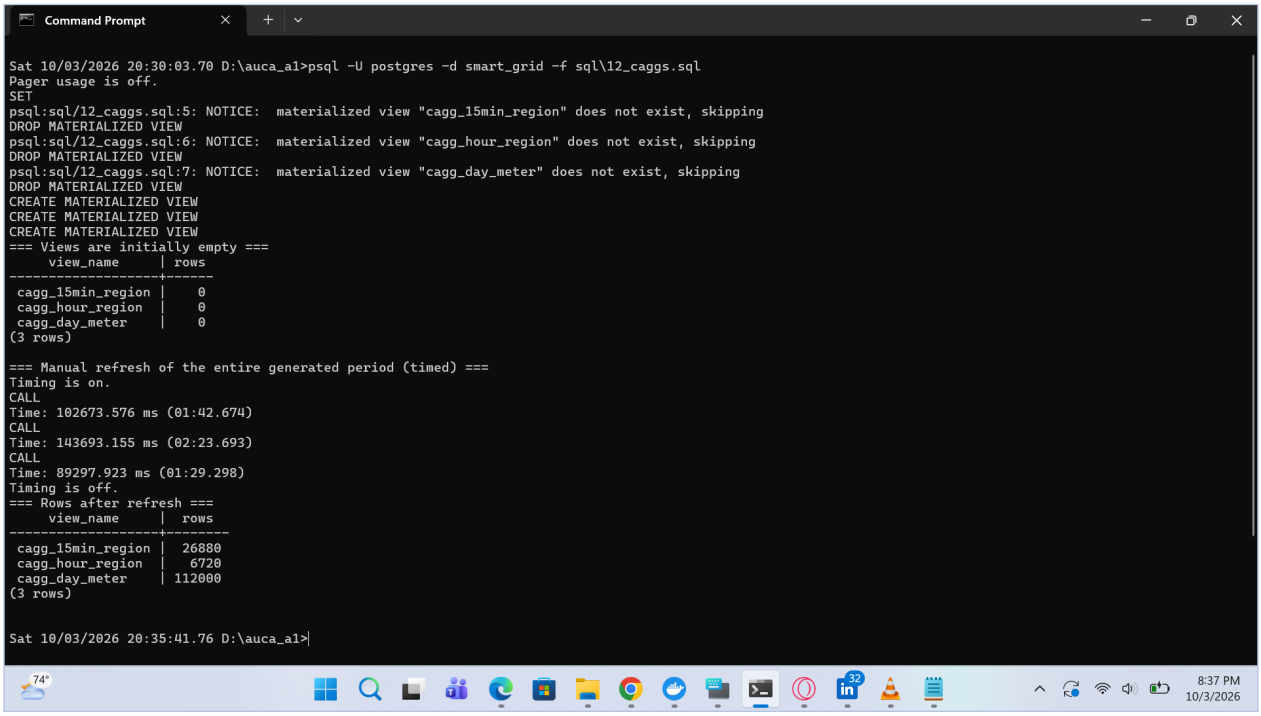


Figure 6.1-1: sql/12_caggs.sql: views created empty, timed manual refresh (102.7 s, 143.7 s, 89.3 s) and row counts after refresh

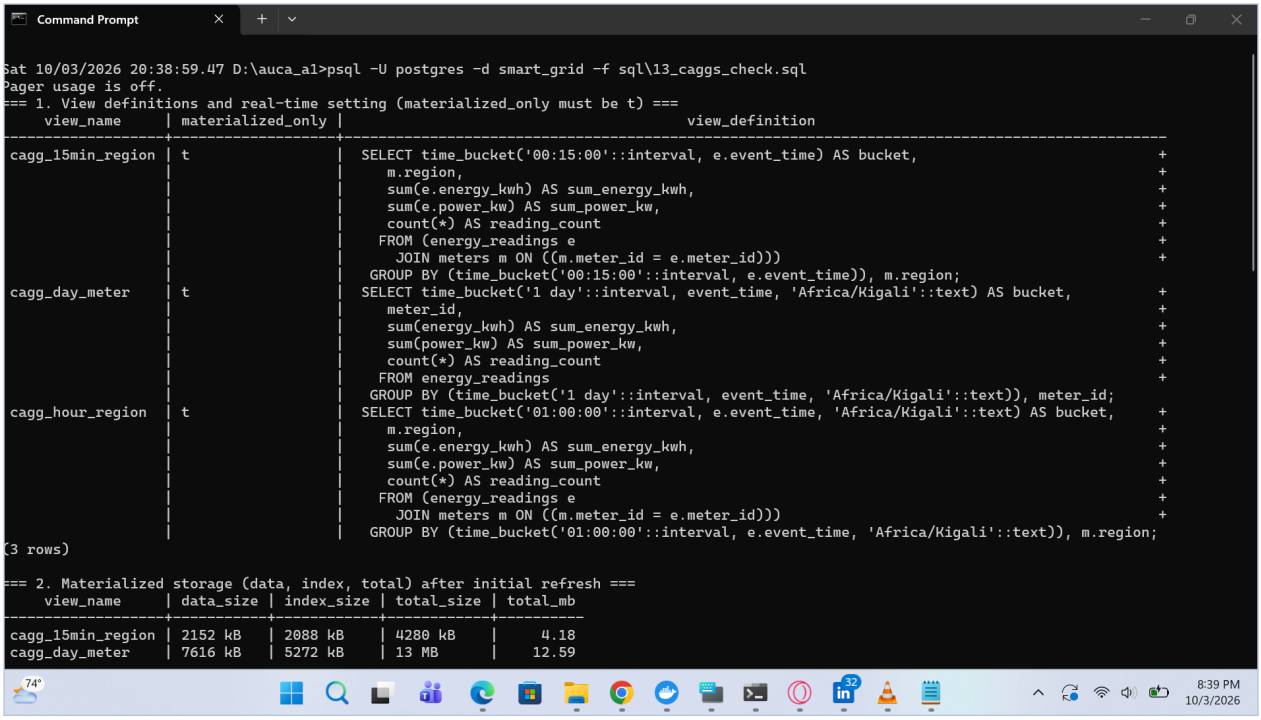


Figure 6.1-2: View definitions and real-time setting (materialized_only = t for all three views)

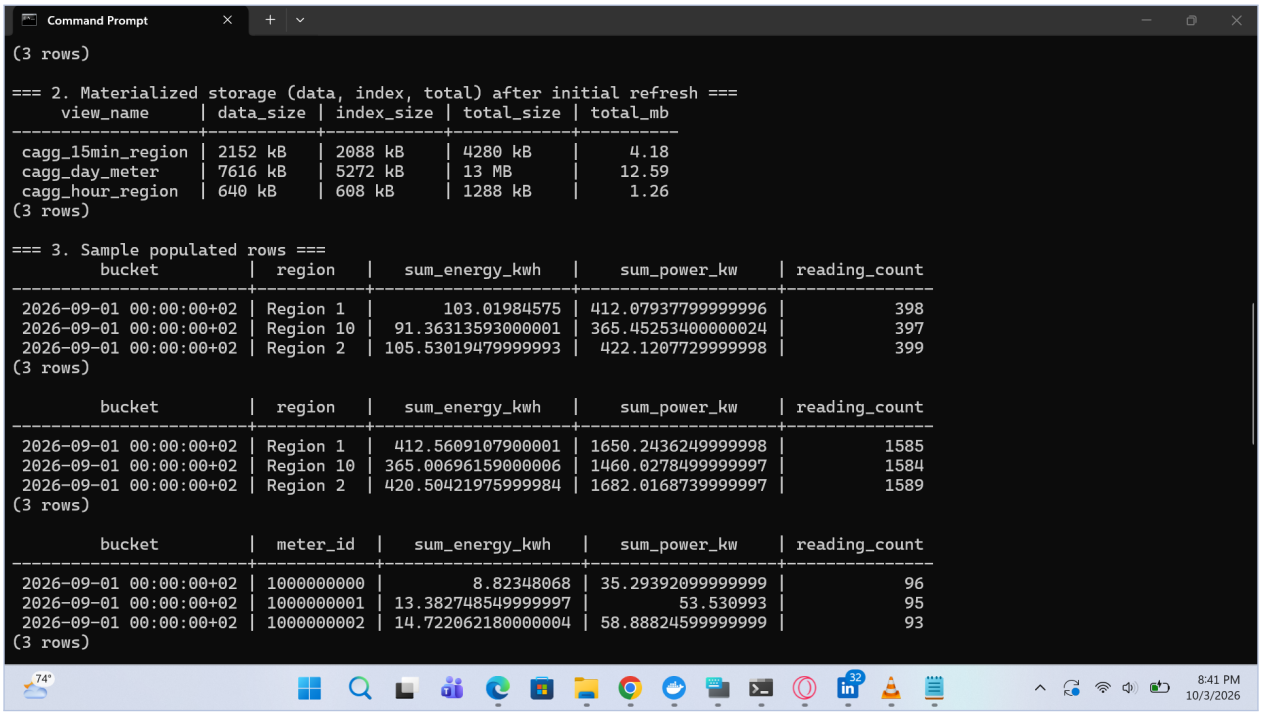


Figure 6.1-3: Materialised storage (data, index, total) after the initial refresh and sample populated rows from each aggregate

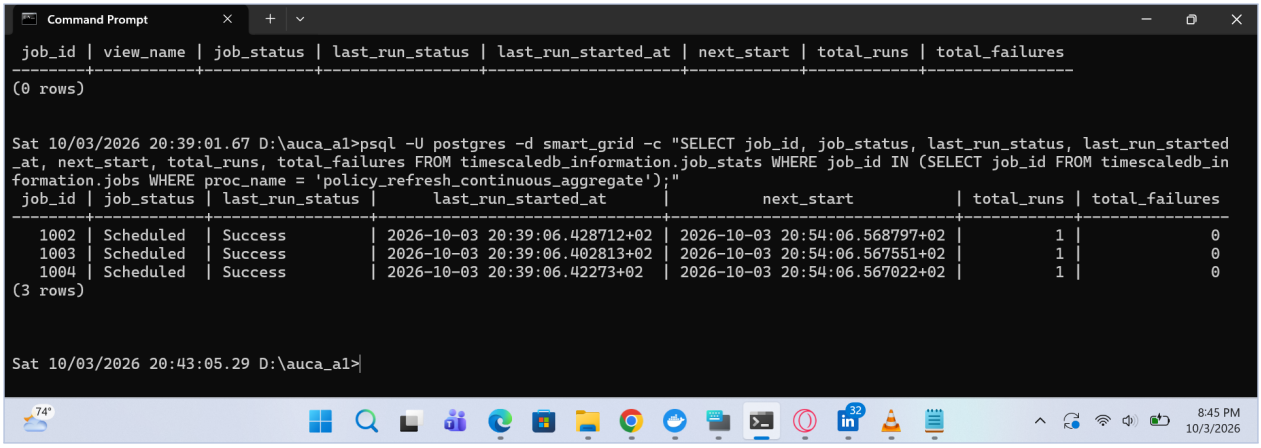


Figure 6.1-4: Refresh policy jobs 1002 to 1004: scheduled, first runs successful, 0 failures

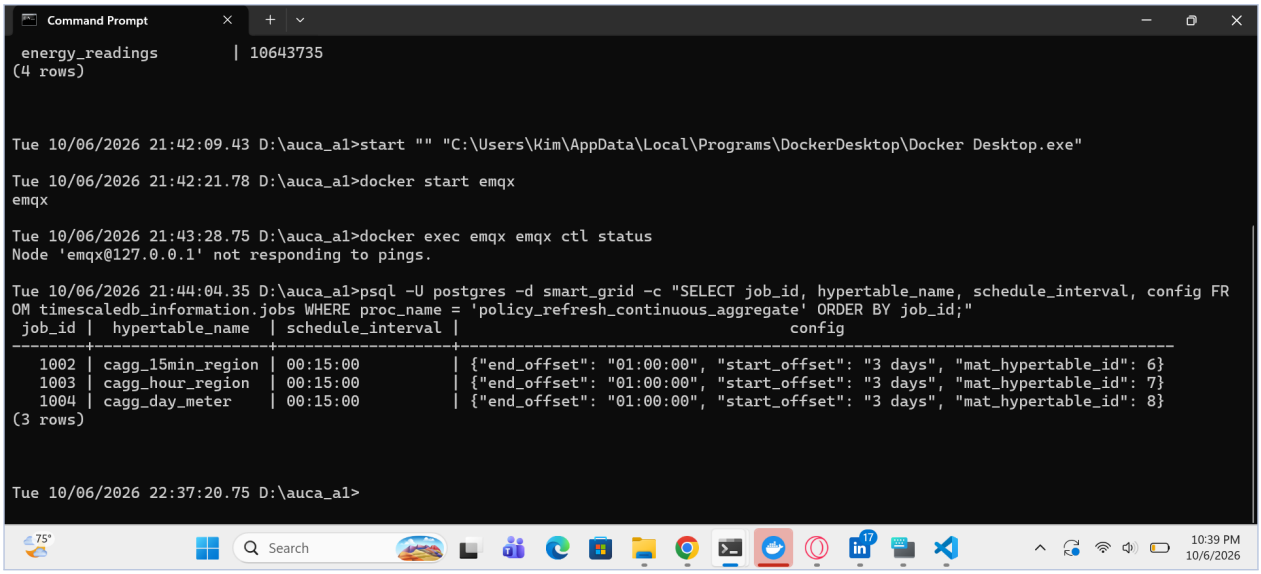


Figure 6.1-5: Policy settings of jobs 1002 to 1004: start_offset 3 days, end_offset 1 hour, schedule interval 15 minutes

6.2 Test data freshness

A continuous aggregate can be behind its source table. To measure this without touching the real data, `src/freshness_6_2.py` created a separate test hypertable `freshness_test` (columns `event_time` and `energy_kwh`, one-day chunks) and a test hourly continuous aggregate `freshness_hourly` over it, with **real-time aggregation off and no refresh policy** (the script printed `materialized_only = True` and no policy job), so every change to the aggregate came from a refresh the script ran itself.

The script used two CAT hours, both measured from the actual test time on 3 October 2026 (about 20:47 CAT): **H** is the most recent completed hour (19:00 to 20:00 on 3 October) and **H − 48 h** is the same hour two days earlier (19:00 to 20:00 on 1 October). Three readings were inserted: **1.0 kWh** at 19:10 on 1 October, **0.5 kWh** at 19:25 on 1 October (a late arrival into an hour that has already been materialised) and **2.0 kWh** at 19:10 on 3 October (inside H).

Stage	Condition	Raw table: H − 48 h / H	Aggregate: H − 48 h / H
1	Older hour refreshed, real-time off, before the late insert	n/a	1.0 kWh, 1 row / no bucket
2	After the inserts, real-time off	1.5 kWh, 2 rows / 2.0 kWh, 1 row	1.0 kWh, 1 row (stale) / no bucket

Stage	Condition	Raw table: H − 48 h / H	Aggregate: H − 48 h / H
3	Real-time aggregation switched on	1.5 kWh, 2 rows / 2.0 kWh, 1 row	1.0 kWh, 1 row (stale) / 2.0 kWh, 1 row (live)
4	Both hours refreshed manually	1.5 kWh, 2 rows / 2.0 kWh, 1 row	1.5 kWh, 2 rows / 2.0 kWh, 1 row

What the stages show

- **Stage 2: a materialised view does not follow its table.** With real-time off, the aggregate still reports 1.0 kWh for the older hour although the raw table now holds 1.5 kWh (the 0.5 kWh late arrival), and it has **no bucket at all** for H, although the raw table holds 2.0 kWh there. The aggregate is stale for an hour that was already materialised and blind to an hour that was never refreshed.
- **Stage 3: real-time aggregation fixes only the newest data.** With real-time on, H now shows the live 2.0 kWh because it lies after the materialised range and is read from the raw table, but the older hour **still shows the stale 1.0 kWh**, because that range is already materialised and the late arrival is not read until a refresh.
- **Stage 4: a refresh closes both gaps.** After refreshing both hours the aggregate matches the raw table: 1.5 kWh (2 rows) and 2.0 kWh (1 row).

Would the automatic policy have refreshed these hours?

The policy refreshes the window from 3 days before to 1 hour before the time it runs. At the test time that window runs from 30 September (about 20:47) to 3 October (about 19:47). The **older hour (1 October, 19:00 to 20:00) lies inside** the window, so the next policy run (every 15 minutes) would correct it to 1.5 kWh. **H (19:00 to 20:00 on 3 October) lies outside**, because the window ends before H ends at 20:00; it becomes refreshable only after 21:00, when the window end passes 20:00. Hence, with real-time off, an aggregate maintained only by the policy lags the newest one to two hours, while late arrivals into older hours are corrected at the next policy run, within 15 minutes.

Implication for reporting. A missing bucket does not mean zero consumption; it means the aggregate has not been refreshed for that hour. Dashboards that read the aggregate must either enable real-time aggregation for the newest hours or show the time of the last refresh next to the data.

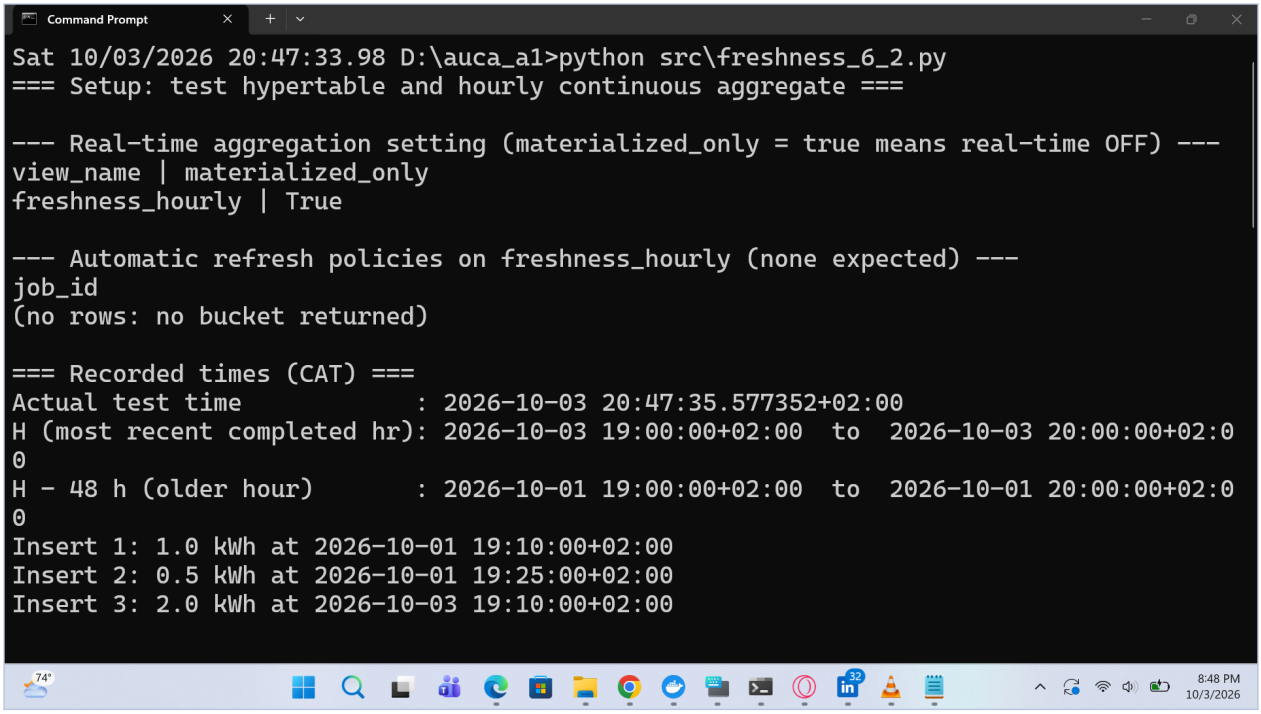


Figure 6.2-1: Freshness test setup: real-time off and no refresh policy, recorded test time, H and H − 48 h, and the three inserts

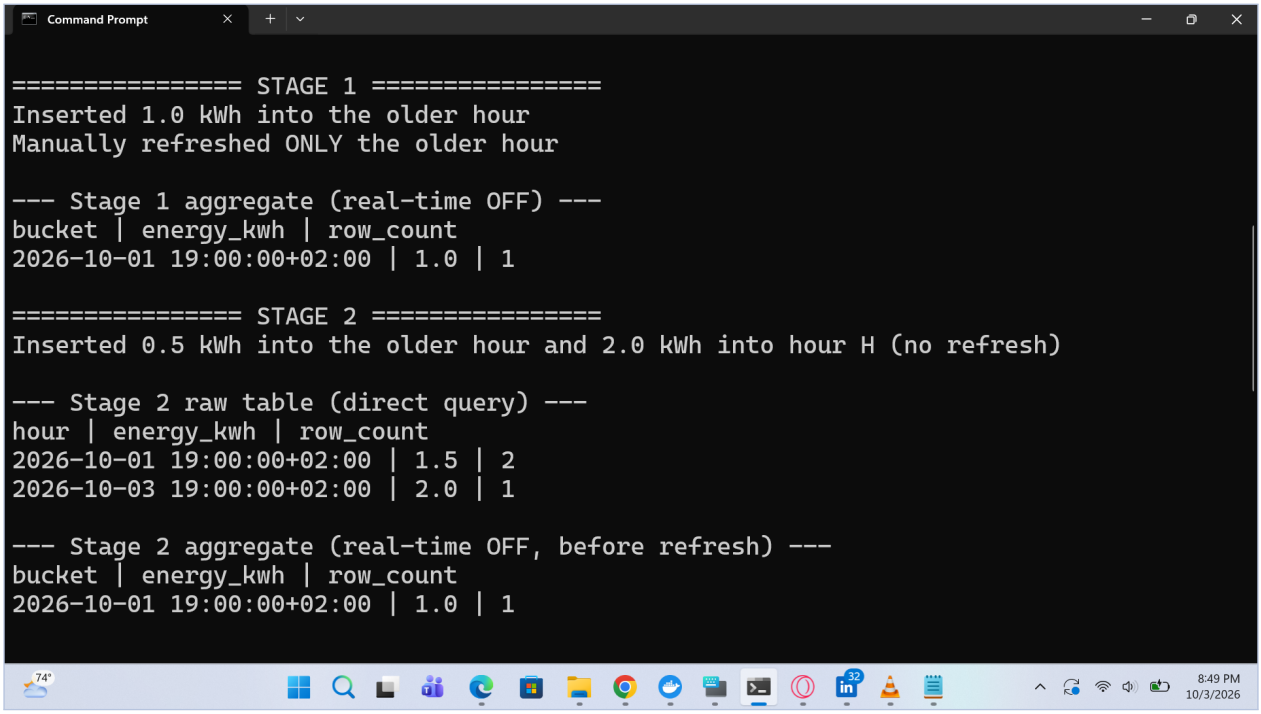


Figure 6.2-2: Stages 1 and 2: raw table against aggregate before and after the inserts (real-time off)

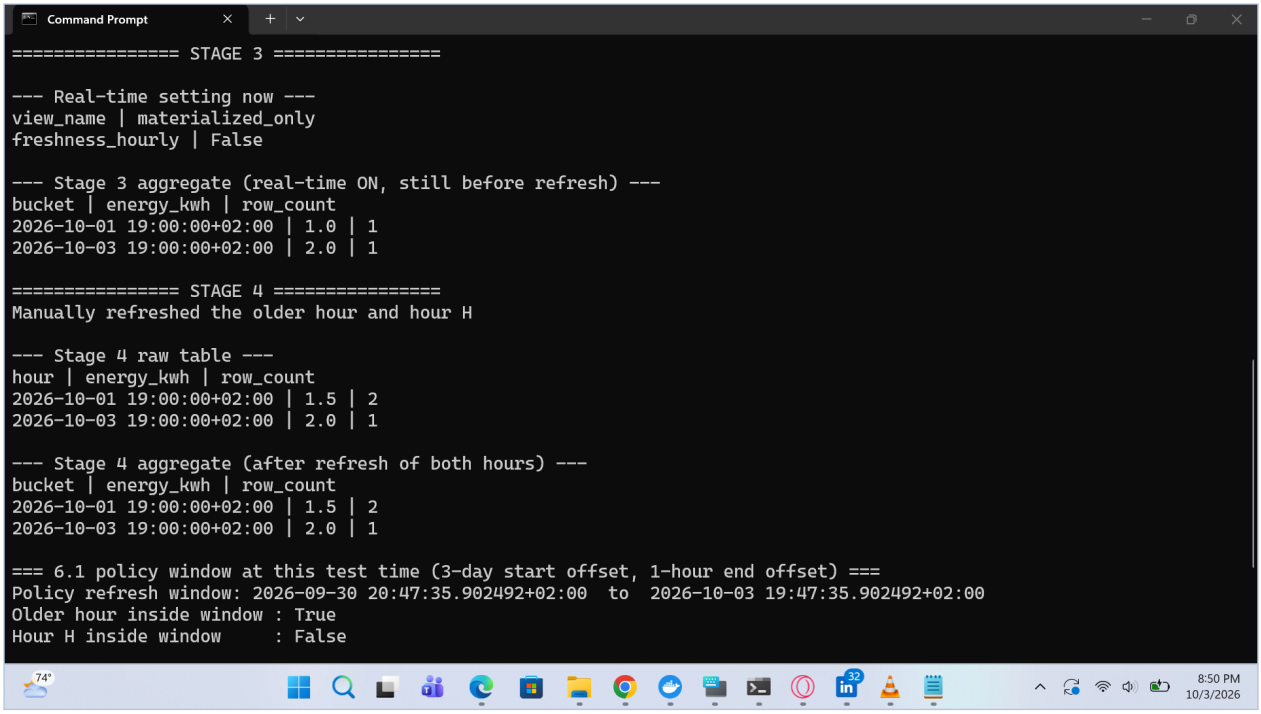


Figure 6.2-3: Stages 3 and 4: real-time on, then both hours refreshed; policy window check at the test time (older hour inside, H outside)

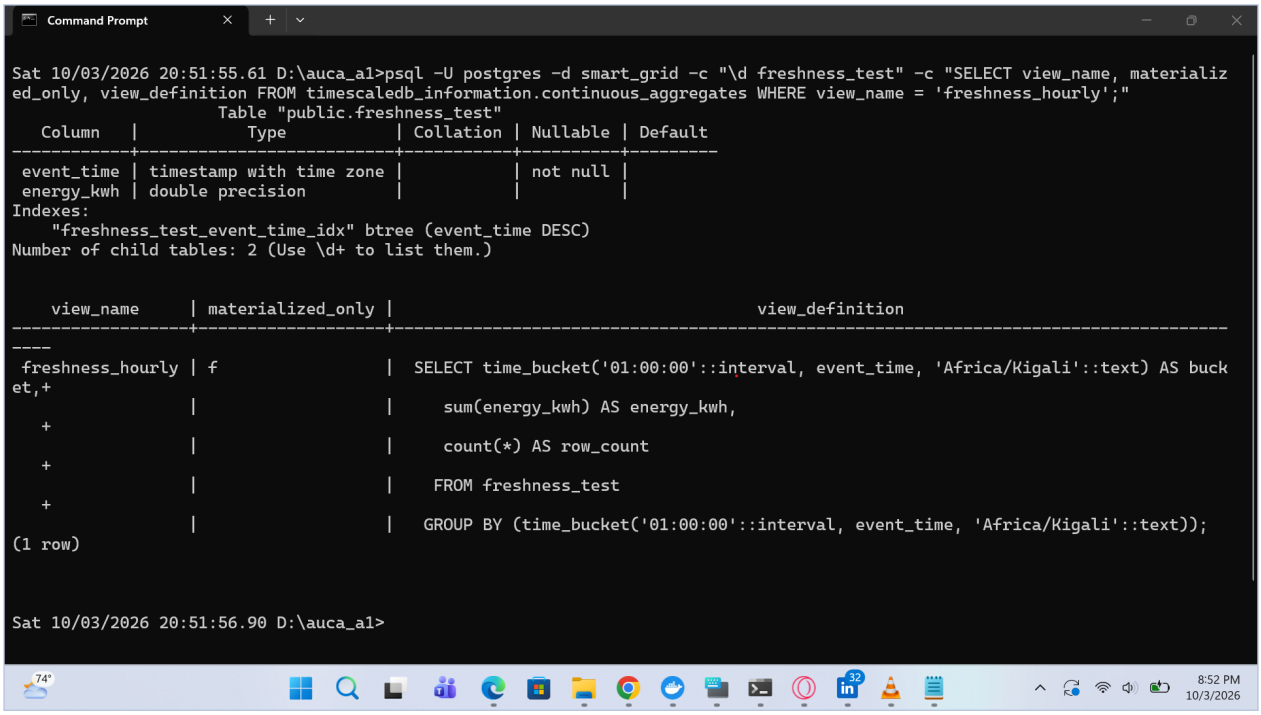


Figure 6.2-4: Test table and freshness_hourly definition, taken after the test (materialized_only = f because real-time was switched on in Stage 3)

7 Dashboards

7.1 Raw dashboard

The dashboards were built with **Dash 4.4.1** (Python). `src/dashboard_app.py` defines the application and its two pages, `src/dashboard_queries.py` holds the SQL for every panel, and `src/assets/style.css` pro-

vides the "GridPulse" theme. The **Raw** dashboard (`http://127.0.0.1:8050/raw`) reads only the base tables: `energy_readings` joined to `meters` where regions or registered meters are needed.

Filters. The page has a CAT date range (inclusive days) and a meter selector, plus a **Refresh** button and a **Run benchmark (1 + 10)** button used in 7.4. The screenshots use 22 to 28 September 2026, all regions and meter 1000000000; the SQL receives the range as a half-open interval (22 September 00:00 to 29 September 00:00 CAT, end exclusive).

Panels. Five panels answer the questions of Section 5: hourly regional energy (kWh), daily network energy (kWh), network energy per seven-day period (kWh), daily energy of the selected meter (kWh) and completeness by region (%). Eight KPI cards (total energy, readings received, average daily energy, peak day, peak network hour, selected-meter energy, network completeness and lowest regional completeness) and the regional completeness table are **calculated from the data already returned by the panels**, so they cause no additional queries. No query results are cached, so every refresh runs all five panel queries again.

For the selected week the dashboard shows total energy **992,560 kWh** from **2,660,983 readings** (the same values as the 22 to 28 September period in Section 5.1), an average of 141,794 kWh per day, a peak day of Monday 28 September (152,464 kWh), a selected-meter energy of 64.68 kWh and a network completeness of 98.9949 % (2,660,983 of 2,688,000 planned readings), with Region 6 the lowest at 98.9598 %.

Raw panel queries

Panel	Raw query (summary)
Hourly regional energy	`time_bucket('1 hour', event_time, 'Africa/Kigali')` and region from `energy_readings` joined to `meters`, `sum(energy_kwh)`, `count(*)`
Daily network energy	`time_bucket('1 day', event_time, 'Africa/Kigali')` over `energy_readings`
Seven-day periods	`time_bucket('7 days', event_time, origin => 2026-09-01 00:00+02)` over `energy_readings`
Selected meter, daily	daily `time_bucket` over `energy_readings` filtered on `meter_id`
Completeness by region	registered meters per region (from `meters`) and observed readings (join with `energy_readings`); planned = registered meters x intervals

Every query filters on `event_time >= start AND event_time < end`, so chunk exclusion limits the scan to the days selected, but each panel still reads and aggregates every raw reading in that range (about 2.66 million rows for one week).

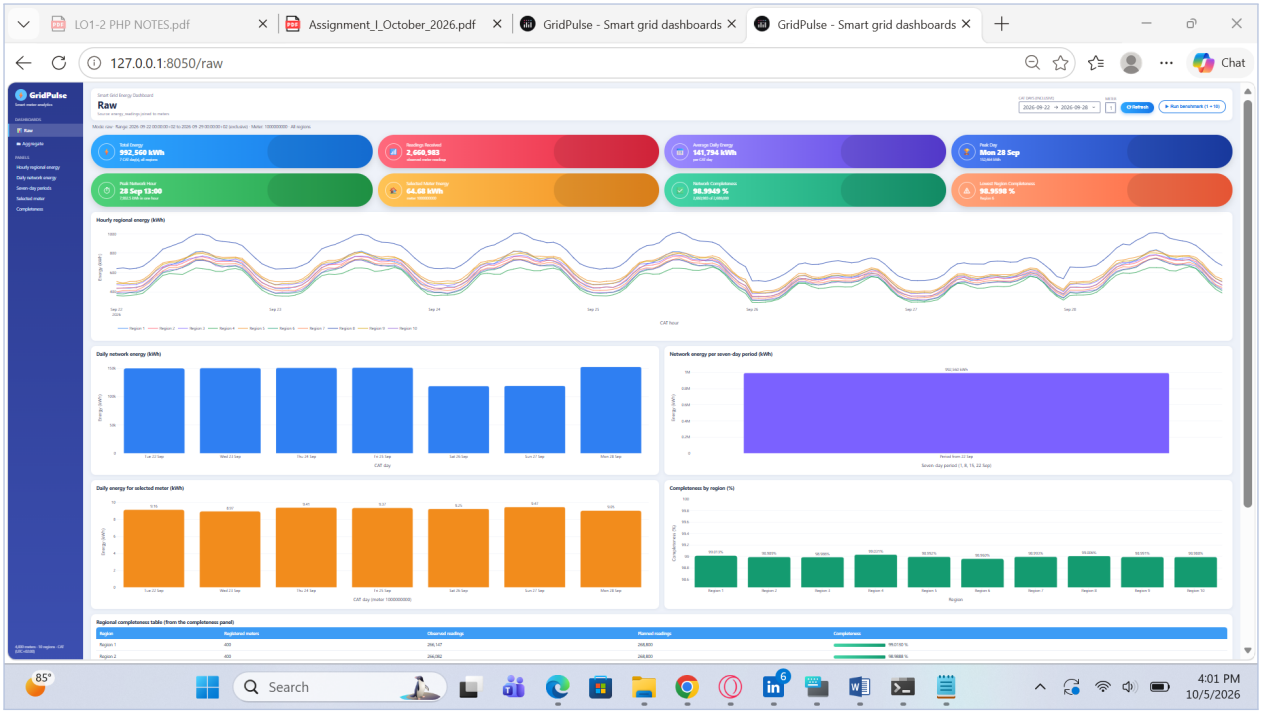


Figure 7.1-1: Raw dashboard (/raw) for 22 to 28 September 2026: KPI cards and the five panels

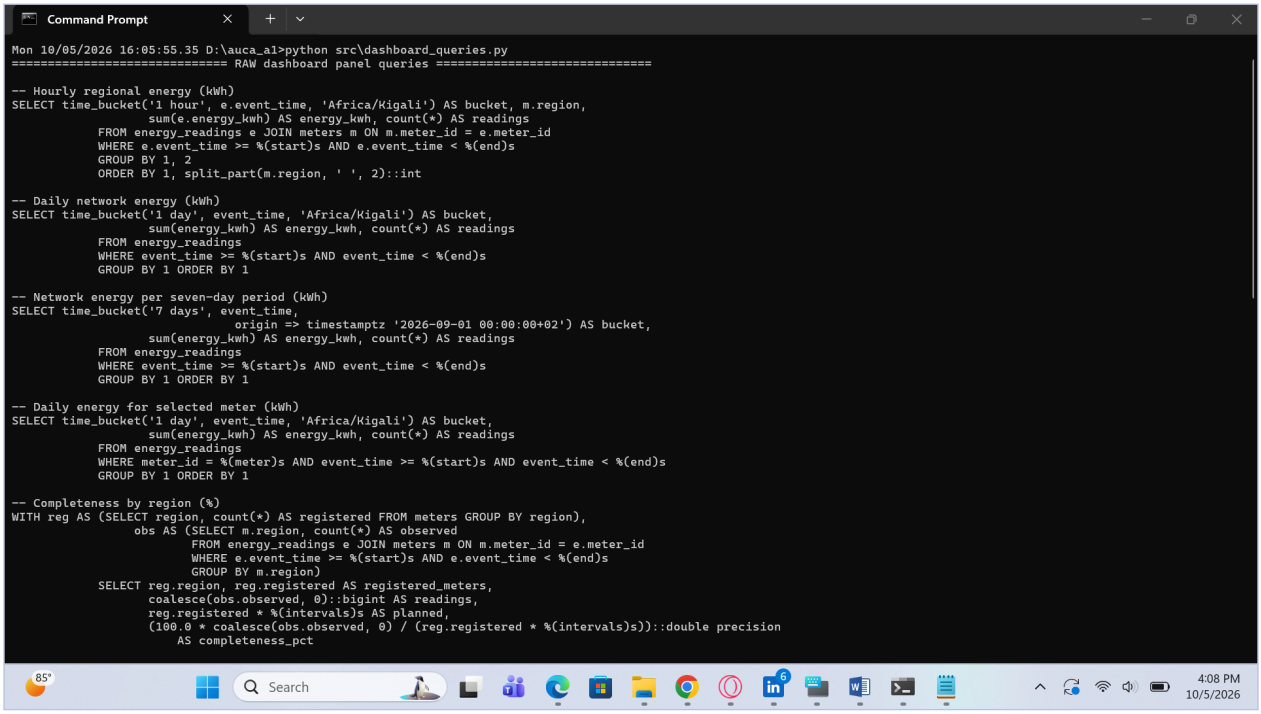


Figure 7.1-2: Raw dashboard panel queries (src/dashboard_queries.py)

7.2 Aggregate dashboard

The **Aggregate** dashboard (`http://127.0.0.1:8050/aggregate`) has the **identical layout, filters, panels and KPI calculations**; only the data source changes. Its panels read the continuous aggregates of Section 6 (with `meters` used only for labels and registered-meter counts), as stated in the page subtitle:

Panel	Aggregate source and query
Hourly regional energy	`cagg_hour_region` (bucket, region, `sum_energy_kwh`, `reading_count`) filtered by bucket
Daily network energy	`cagg_15min_region`, re-bucketed to CAT days with `time_bucket('1 day', bucket, 'Africa/Kigali')` and summed
Seven-day periods	`cagg_15min_region`, re-bucketed with `time_bucket('7 days', bucket, origin => 2026-09-01 00:00+02)`
Selected meter, daily	`cagg_day_meter` filtered on `meter_id`
Completeness by region	`sum(reading_count)` from `cagg_15min_region` grouped by region, against registered meters x intervals

This works because the aggregates store **sums and counts**: daily and weekly totals are sums of 15-minute sums, and completeness uses the stored reading counts. The aggregate panels scan at most 7 x 96 x 10 = 6,720 rows (15-minute view), 168 x 10 = 1,680 rows (hourly view) or 7 rows (daily meter view) for the selected week instead of 2.66 million raw readings.

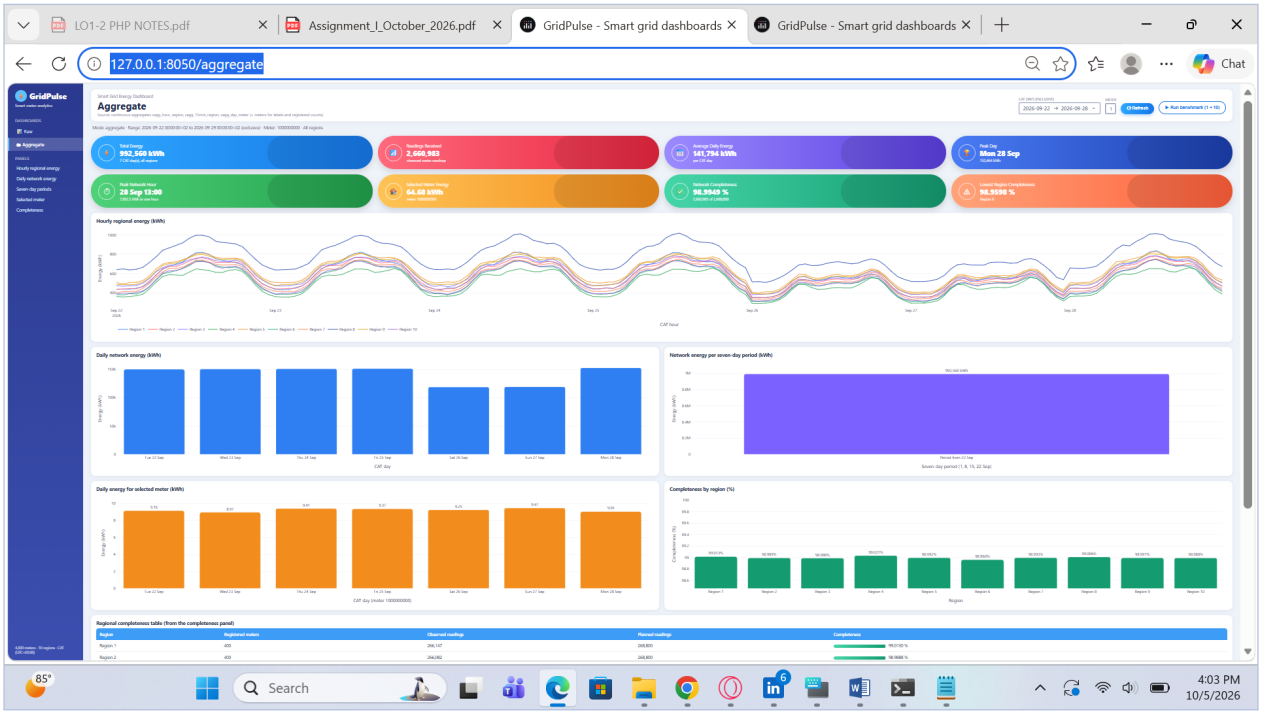


Figure 7.2-1: Aggregate dashboard (/aggregate) for the same filters: identical layout and values

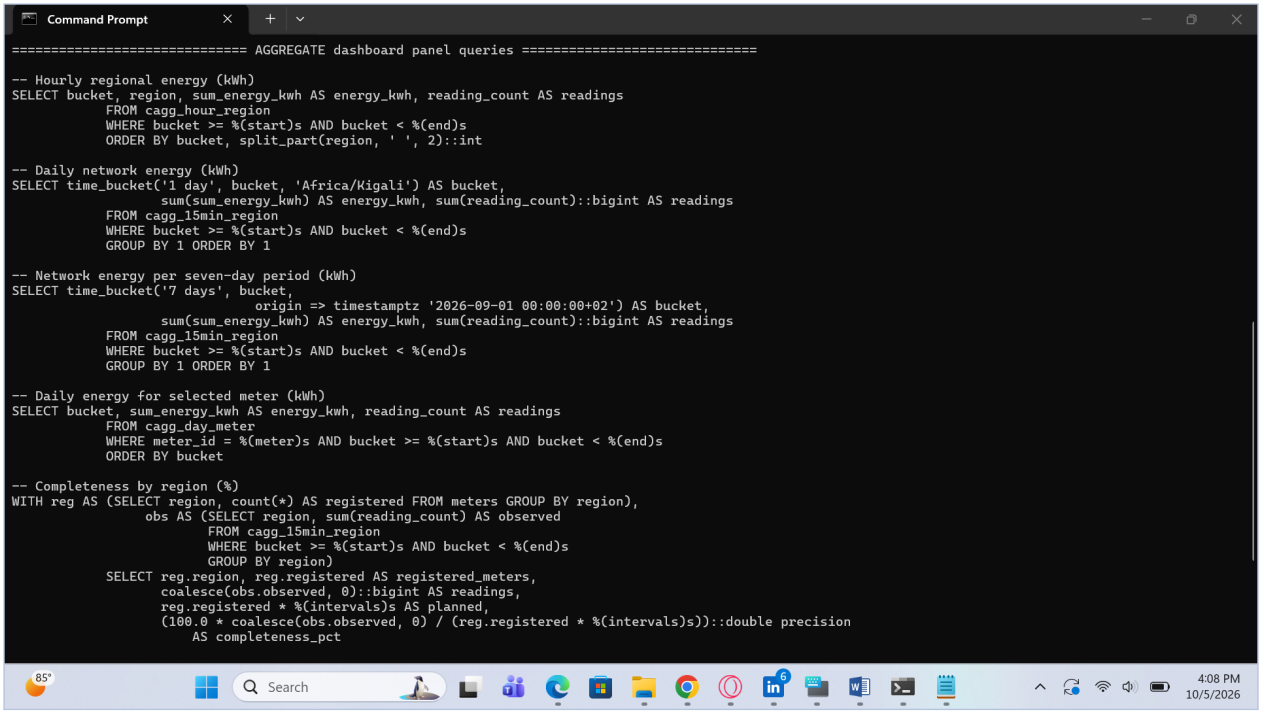


Figure 7.2-2: Aggregate dashboard panel queries on the continuous aggregates

7.3 Check that the two dashboards agree

A faster dashboard is only useful if it shows the same numbers. `src/dashboard_verify.py` ran both query sets with the same filters (22 September 00:00 to 29 September 00:00 CAT, all regions, meter 1000000000) and compared them panel by panel: number of rows, the keys (buckets, regions), the reading counts and the **maximum absolute difference** in the values, against limits of 0.001 kWh for energy and 1e-6 percentage points for completeness.

Panel	Raw rows	Aggregate rows	Keys same	Counts same	Max abs diff	Limit	Result
hourly_region	1,680	1,680	True	True	3.524e-12	0.001 kWh	**PASS**
daily_network	7	7	True	True	4.948e-09	0.001 kWh	**PASS**
weekly_network	1	1	True	True	1.048e-08	0.001 kWh	**PASS**
meter_daily	7	7	True	True	0.000e+00	0.001 kWh	**PASS**
completeness	10	10	True	True	0.000e+00	1e-6 pct points	**PASS**

All five panels match (`ALL FIVE PANELS MATCH: True`). The largest difference, 1.048e-8 kWh, is far below the limit and is only the floating-point rounding that results from summing the same numbers in a different order (the aggregate adds up partial sums). Row counts, keys and reading counts are identical, so the aggregates lose no information for these panels. The panel data of both dashboards was exported to `results/dashboard_export/` as `<panel>_raw.csv` and `<panel>_aggregate.csv`.

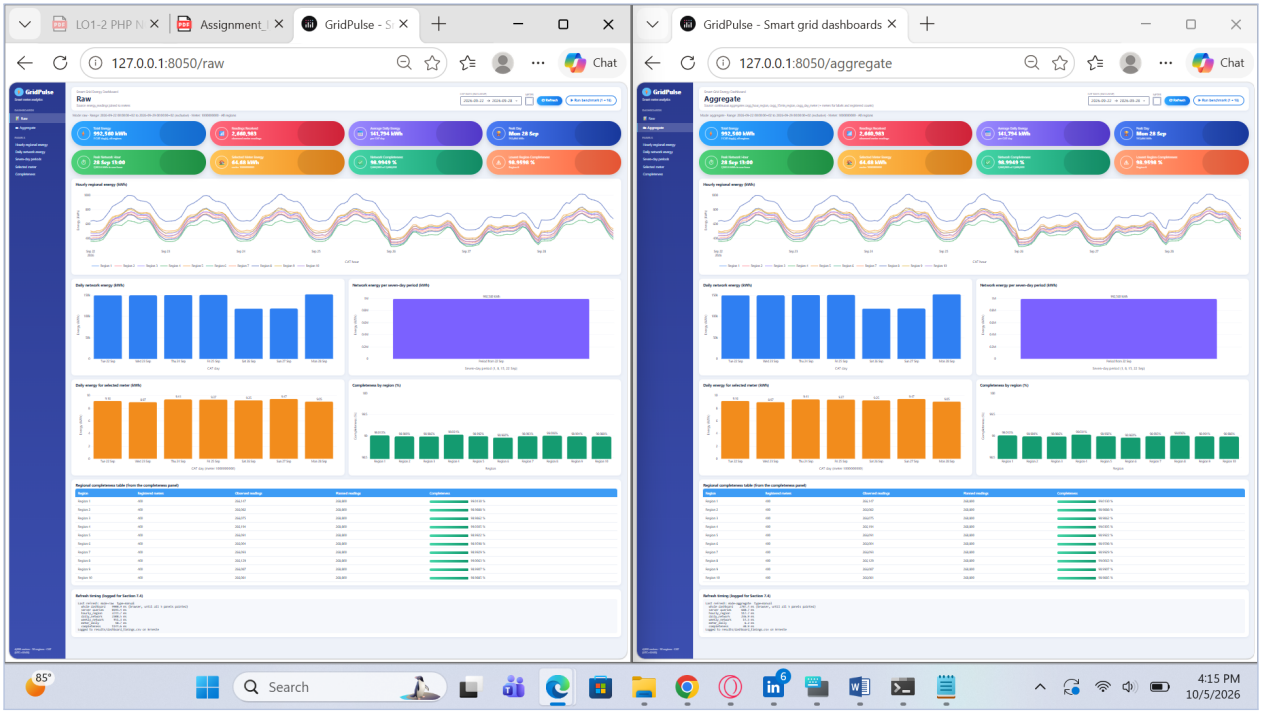


Figure 7.3-1: Raw (left) and Aggregate (right) dashboards side by side with the same filters: same KPI values and charts

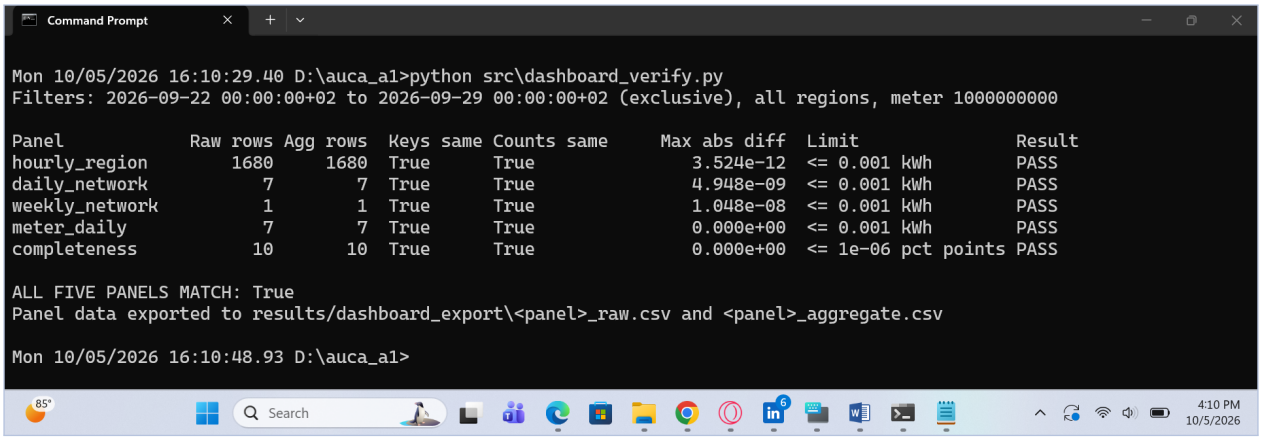


Figure 7.3-2: src/dashboard_verify.py: row counts, keys, counts and maximum absolute differences; all five panels PASS

7.4 Compare refresh performance

Method. Before the benchmark, the five background jobs (compression policies 1000 and 1001 and refresh policies 1002 to 1004) were **paused** with `alter_job(job_id, scheduled => false)` so that no compression or refresh could disturb the timings; the `scheduled` column shows `f` for all five jobs. Each dashboard was then benchmarked with its **Run benchmark (1 + 10)** button in the Edge browser on machine Erneste: one warm-up refresh, discarded, followed by **10 measured refreshes** with the same filters as in 7.3. Dashboard query-result caching is disabled (every refresh runs all five panel queries again) and every refresh is logged to `results/dashboard_timings.csv`. The query time of each panel was measured on the Dash server with `time.perf_counter()`; the time of the **whole dashboard** was measured in the browser with `performance.now()` from the refresh request until all five panels were painted. Raw benchmark: 16:20:40 to 16:22:01 CAT on 5 October 2026; aggregate benchmark: 16:25:16 to 16:25:28 CAT.

Median (minimum to maximum) in milliseconds over the 10 refreshes, and the speed-up = raw median ÷ aggregate median:

Measure	Raw median (min–max)	Aggregate median (min–max)	Speed-up
Hourly regional energy	2,955.7 (2,833.4–3,093.7)	16.4 (10.7–20.6)	x180.5
Daily network energy	2,168.2 (2,105.0–2,539.7)	24.6 (14.5–35.2)	x88.1
Seven-day periods	820.1 (795.2–904.4)	11.1 (6.0–15.7)	x74.2
Selected meter, daily	9.1 (6.0–12.6)	1.8 (1.5–2.4)	x5.0
Completeness by region	979.1 (899.6–1,014.3)	20.3 (8.0–22.2)	x48.1
Whole dashboard	**7,956.9** (7,846.6–8,735.4)	**1,117.8** (882.5–2,224.9)	**x7.1**

Cost of the aggregates, from Section 6.1, for comparison with these timings: **18.03 MB** of storage and an initial refresh of **335.7 s**.

Observations. Each panel query is 5 to 180 times faster on the aggregates; the largest gains are on the panels that scan the most raw rows (hourly regional and daily network, which aggregate about 2.66 million readings), and the smallest gain is on the single-meter panel, which is already cheap on raw data because it touches the readings of only one meter. The **whole dashboard is only 7.1 times faster** because it includes work that the aggregates cannot speed up: the five panel medians add up to about 6.9 s (raw) and 0.07 s (aggregate), leaving about 1.0 s in both cases for transfer and drawing the charts in the browser (`run_panels` runs the five queries one after another; medians of separate measures do not add exactly). The dashboard time on the aggregates is therefore dominated by rendering, not by the database.

After the measurements, all five jobs were **resumed** with `alter_job(job_id, scheduled => true)` at 16:28 CAT and show `scheduled = t`.

Is the aggregate worth its cost?

Judgement: yes, justified. The cost is **18.03 MB** of storage (about 1.3 % of the base table) and a one-off initial refresh of **335.7 s** (Section 6.1), plus the small cost of the 15-minute refresh policies. The gain is 6.8 s per dashboard refresh (7,956.9 ms − 1,117.8 ms), so the one-off refresh is repaid after about **50 dashboard refreshes** (335.7 s ÷ 6.84 s = 49.1), which a regularly used dashboard reaches quickly. The trade-off is **freshness** (Section 6.2): with real-time aggregation off and a policy that stops one hour before now, the aggregate dashboard lags the raw dashboard by one to two hours and late arrivals are corrected only at the next policy run. The aggregate dashboard is therefore recommended for regional, daily and weekly reporting, and the raw dashboard (or real-time aggregation) for the most recent hours.

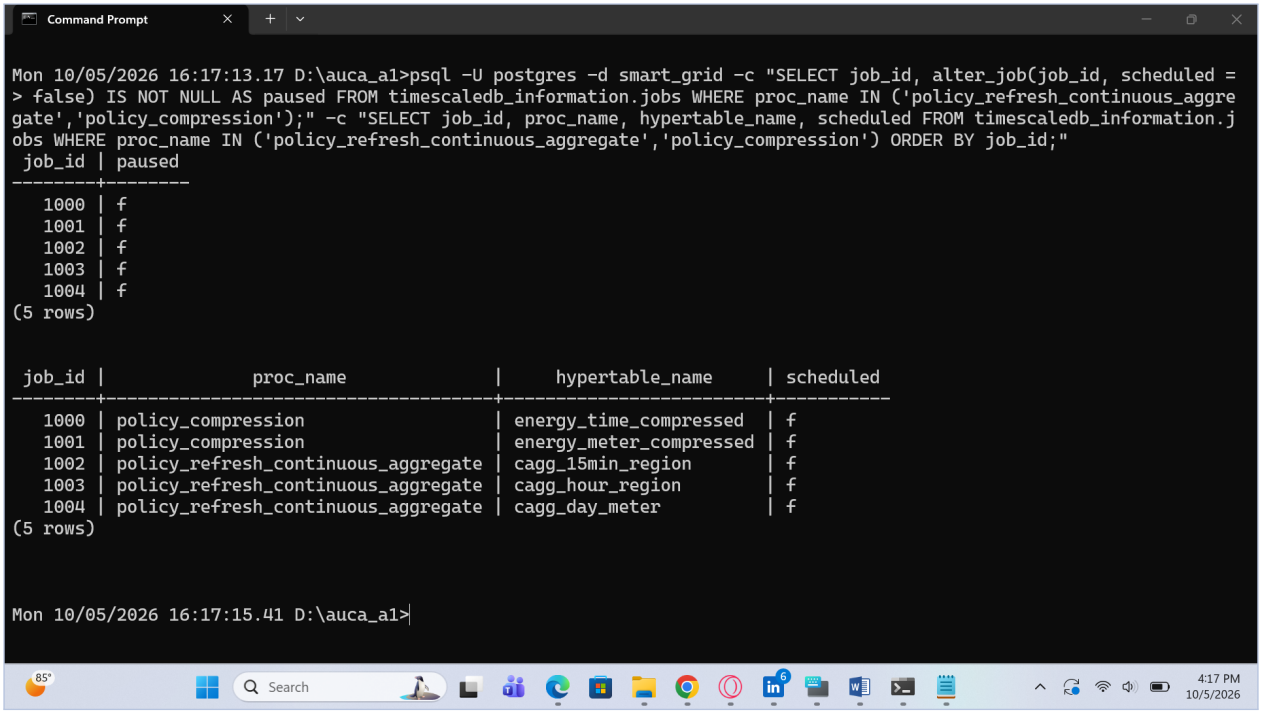


Figure 7.4-1: Compression and refresh jobs 1000 to 1004 paused before the benchmark (scheduled = f)

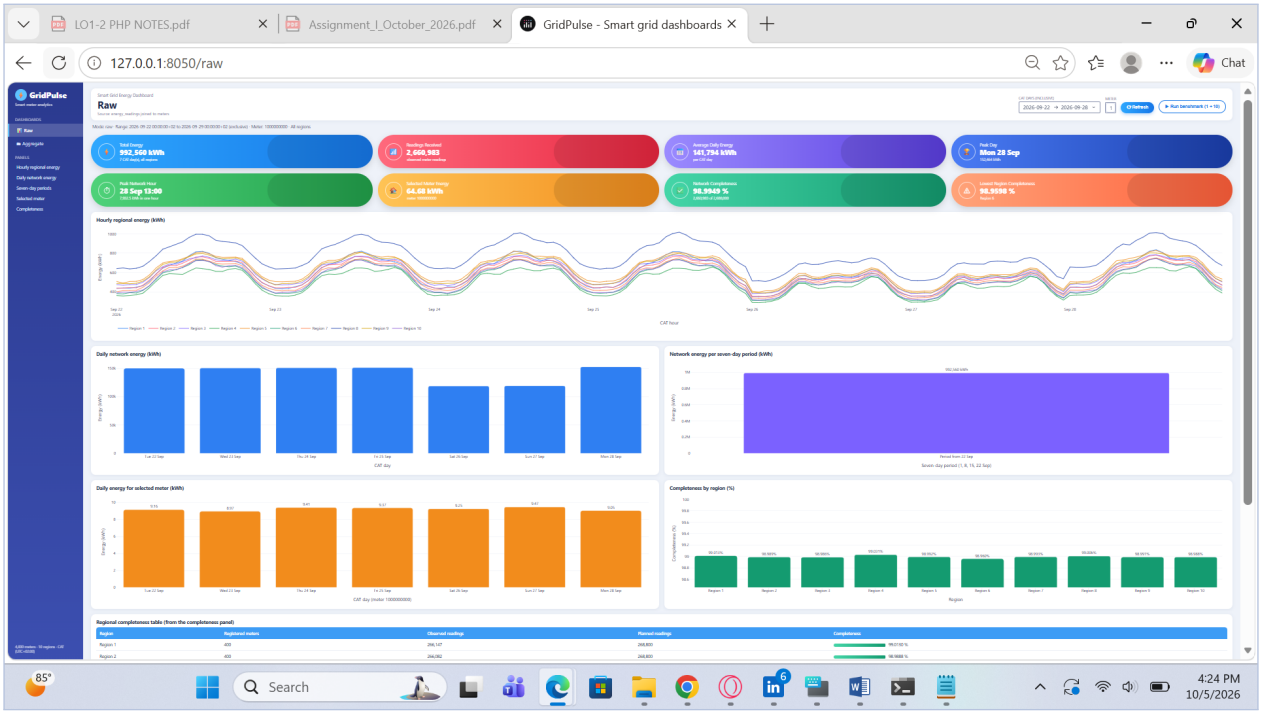


Figure 7.4-2: Raw dashboard after its benchmark (1 warm-up + 10 measured refreshes)

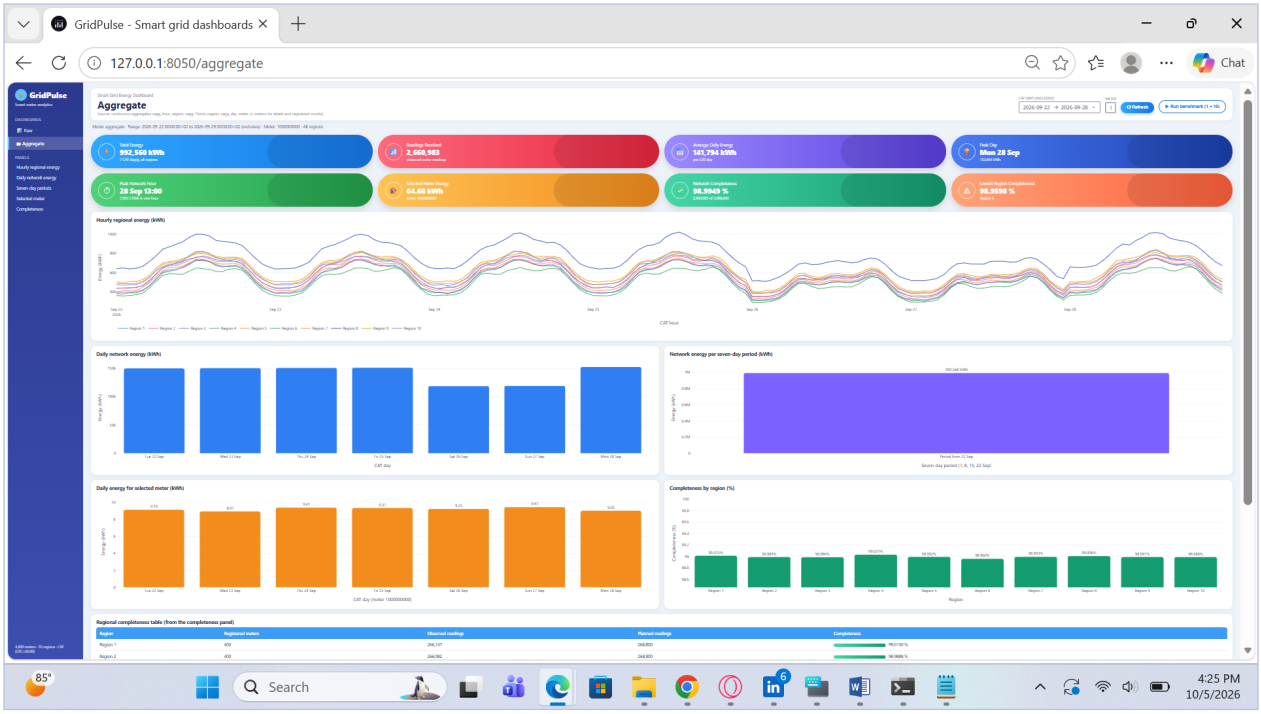


Figure 7.4-3: Aggregate dashboard after its benchmark (1 warm-up + 10 measured refreshes)

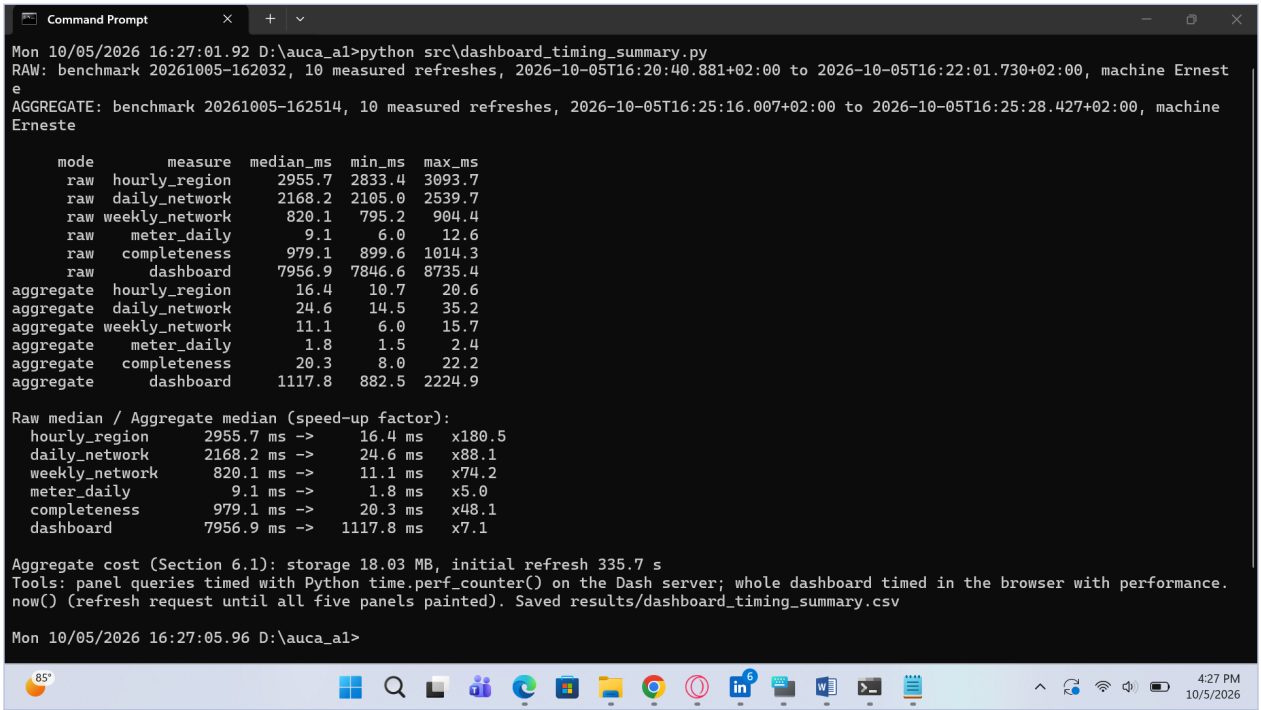


Figure 7.4-4: src/dashboard_timing_summary.py: medians, minimums, maximums and speed-up factors for raw and aggregate

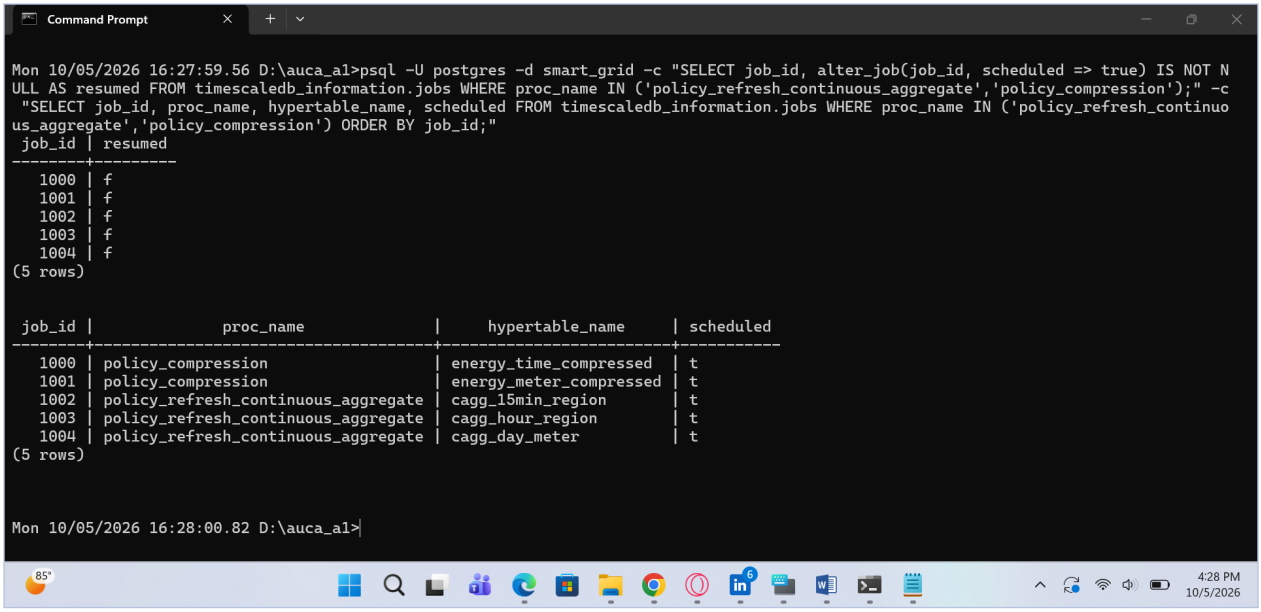


Figure 7.4-5: All five jobs resumed after the benchmark (scheduled = t)

Repeat of the raw benchmark on 8 October 2026. As additional evidence, the raw dashboard benchmark was repeated on 8 October 2026 with the background jobs paused (benchmark 20261008-111156: 1 warm-up and 10 measured refreshes, completed). The medians reported above are those of the 5 October benchmark; the figures below show the repeat run, the state of the jobs during that session and their resumption afterwards.

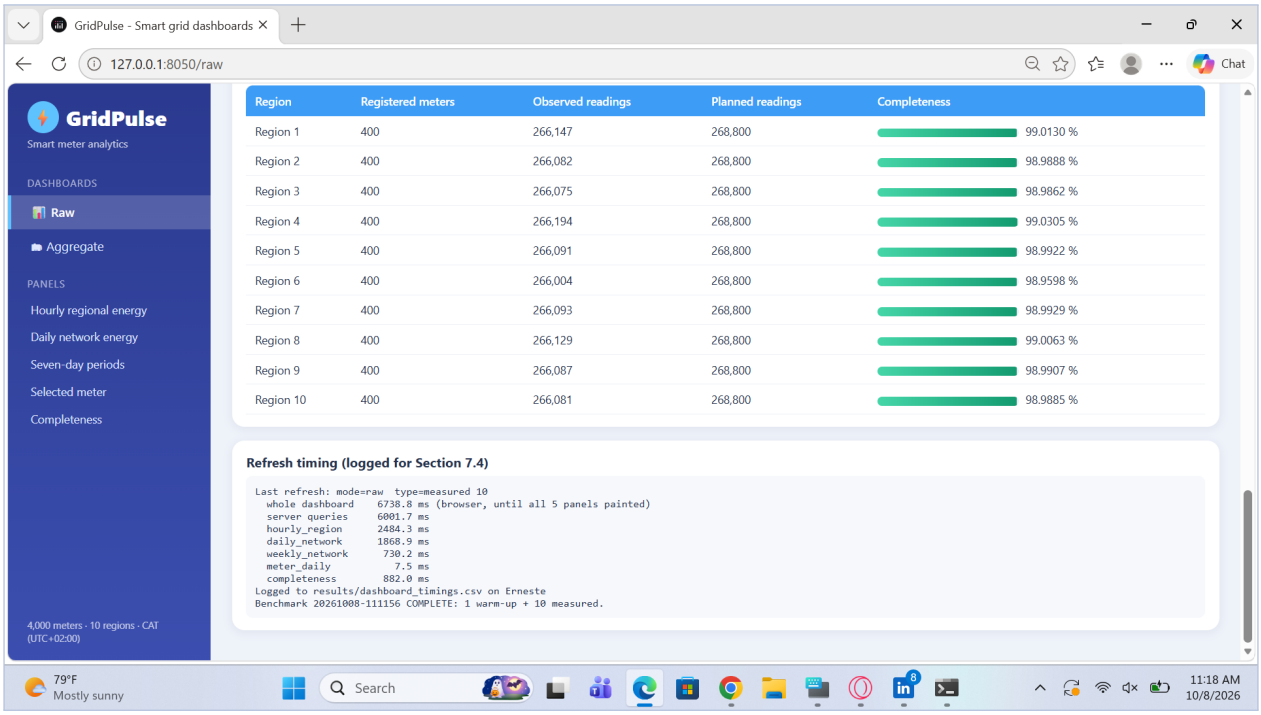


Figure 7.4-6: Repeat on 8 October 2026: Raw dashboard timing panel after the benchmark (Benchmark 20261008-111156 COMPLETE: 1 warm-up + 10 measured)

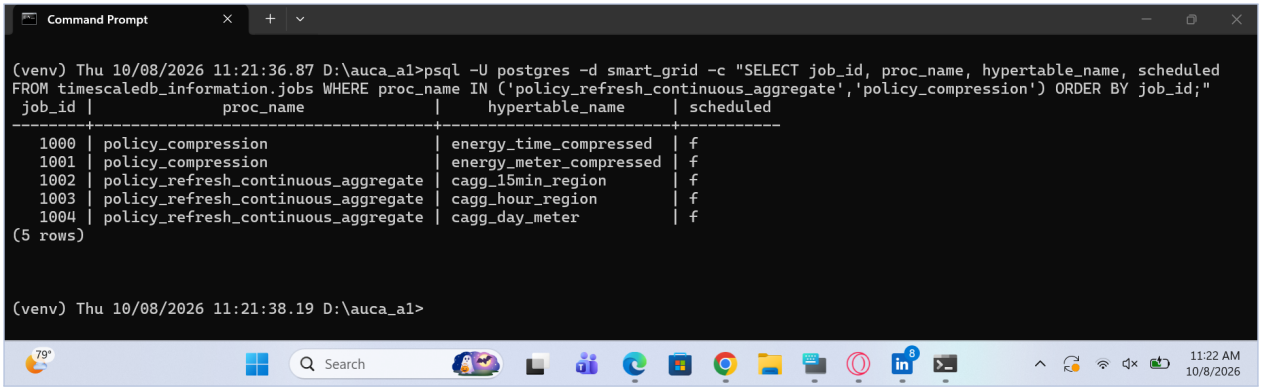


Figure 7.4-7: Repeat on 8 October 2026: jobs 1000 to 1004 show scheduled = f (paused) during the benchmark session (status check at 11:21 CAT; the jobs were paused at 11:08 CAT)

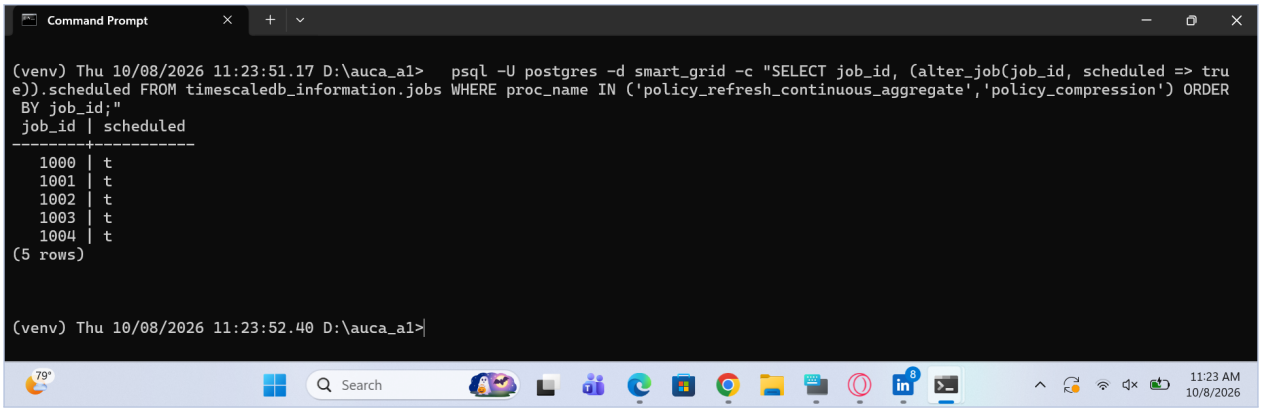


Figure 7.4-8: Repeat on 8 October 2026: all five jobs resumed after the benchmark (scheduled = t, 11:23 CAT)

8 Forecasting hourly demand

The forecasting task is to predict, at **00:00 CAT of a target day**, the energy of every region for each hour of that day. `src/forecast_8.py` builds the dataset, a simple baseline and a Ridge regression model, and evaluates them on a week that was never used for training or model selection.

8.1 Dataset, features and split

The dataset has **one row per region and CAT hour**: the sum of `energy_kwh` of the cleaned readings, 10 regions x 672 hours = **6,720 rows** (total energy 3,853,704.347 kWh). Each region-hour is built from between 1,562 and 1,597 readings (mean 1,583.9) of the planned 1,600 (400 meters x 4 intervals). No region-hour is absent; an absent hour would have been kept as missing and never filled with zero. There are no missing predictor values: the only rows without a complete set of lags are the first days of the period, which are excluded (see below) rather than imputed.

Features

All features are known at the forecast origin (00:00 CAT of the target day). The target is the region-hour energy.

Feature	Description	Lag / window	Earliest usable target
hour	calendar: hour of day (CAT)	0 h (target time)	1 Sep 00:00
dow	calendar: day of week	0 h (target time)	1 Sep 00:00
region	categorical: region (one-hot)	n/a	1 Sep 00:00
lag_24	same hour, previous day	24 h	2 Sep 00:00
lag_168	same hour, one week earlier	168 h	8 Sep 00:00
prev_day_total	region total over the 24 h before the origin	24 h window	2 Sep 00:00
same_hour_mean_7d	mean of the same hour over the previous 7 days	24–168 h (7 values)	8 Sep 00:00

No leakage. For every row the script checked that each lag source timestamp precedes its forecast origin (`src_lag_24`, `src_lag_168`, `src_prev_day_total_last`, `src_same_hour_mean_7d_last`: all True, `ALL LAG SOURCES BEFORE ORIGIN: True`). In the example rows (Region 1, 22 September 12:00), the target 813.996 kWh is predicted from `lag_24` = 800.710 kWh (21 September 12:00) and `lag_168` = 793.305 kWh (15 September 12:00), both before 00:00 on 22 September.

Chronological split

Set	Rows	Period (CAT)	Purpose
Train	3,360	1 Sep 00:00 to 14 Sep 23:00	fit the model
Validation	1,680	15 Sep 00:00 to 21 Sep 23:00	choose the Ridge setting
Test	1,680	22 Sep 00:00 to 28 Sep 23:00	final evaluation, used once

The split is by **target time**, never random, so the model never sees the future. The training rows from 1 to 7 September lack the 168-hour lookback, so **1,680 of the 3,360 training rows are excluded** and **1,680 are retained** (8 to 14 September). The same retained rows are used for every configuration.

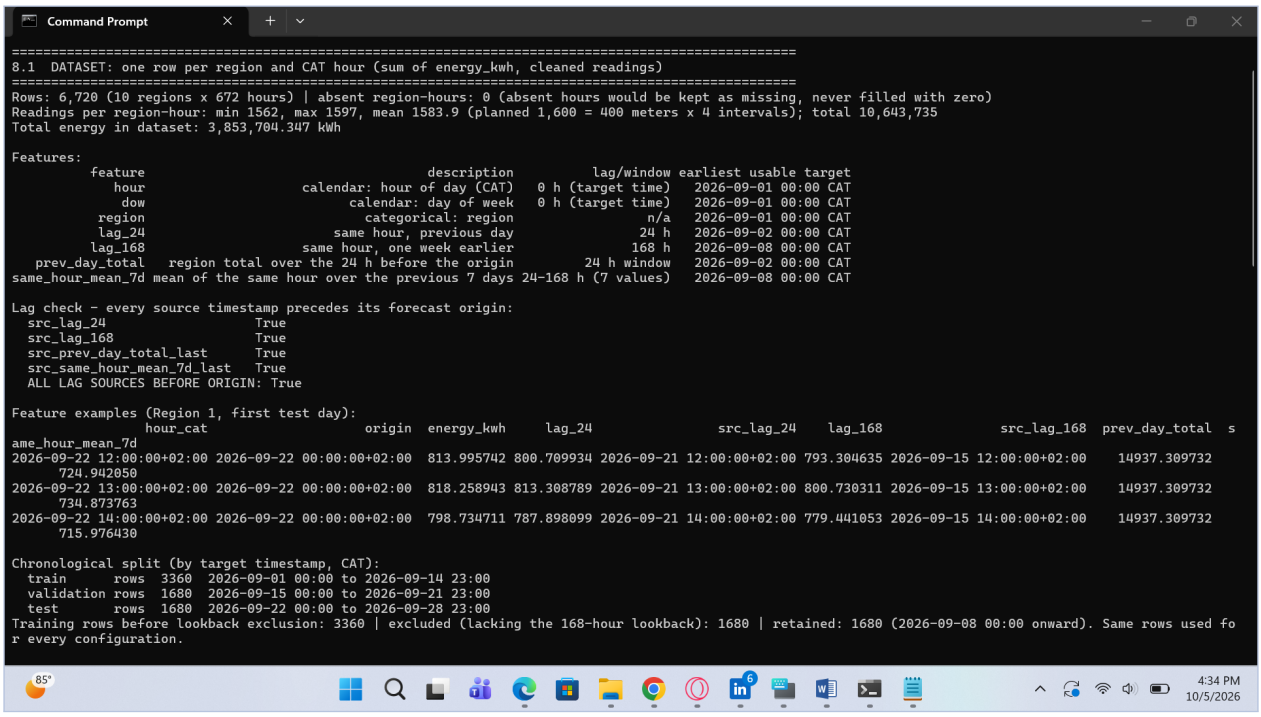


Figure 8.1-1: Dataset size and completeness, features with lags and earliest usable targets, lag check, feature examples and chronological split

8.2 Baseline: previous day, same hour

The baseline predicts each hour with the **same hour of the previous day**. On the 1,680 test observations (7 days x 24 hours x 10 regions) it gives a **mean absolute error (MAE) of 42.708 kWh** and a **root mean squared error (RMSE) of 79.403 kWh**. The first rows illustrate it: for Region 1 at 22 September 00:00 the actual energy is 435.590 kWh and the baseline 441.869 kWh. The baseline works when two consecutive days have the same pattern and fails at a change of day type, which is analysed in 8.4.

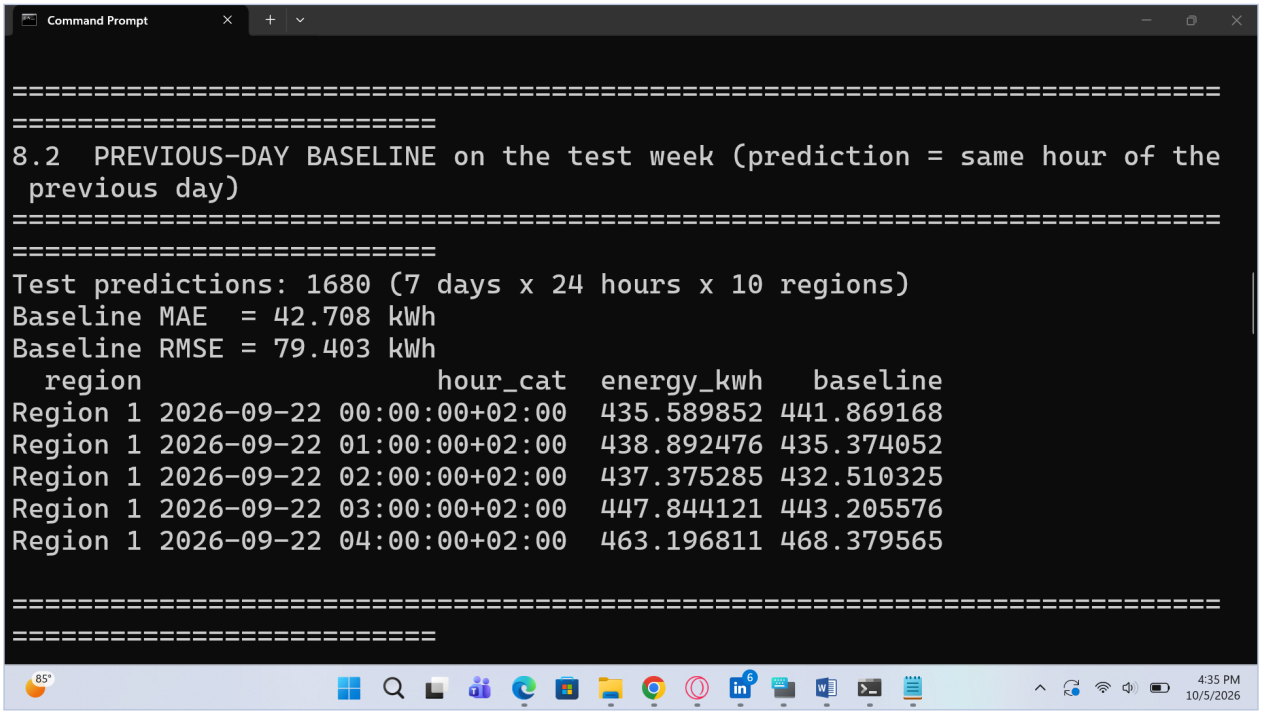


Figure 8.2-1: Previous-day baseline on the test week: 1,680 predictions, MAE 42.708 kWh, RMSE 79.403 kWh, first rows

8.3 Ridge regression and model selection

The model is **Ridge regression** (linear regression with an L2 penalty `alpha`) on the shared predictors: one-hot encoded `region`, `hour` and `dow`, plus the standardised numeric features `lag_24`, `lag_168`, `prev_day_total` and `same_hour_mean_7d`. The preprocessing was fitted on the training data only. Four penalties were trained on the 1,680 retained training rows and compared on the 1,680 validation rows:

Config	Setting	Train rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
C1	**Ridge(alpha = 0.1)**	1,680	1,680	**4.458**	**5.657**
C2	Ridge(alpha = 1.0)	1,680	1,680	4.469	5.672
C3	Ridge(alpha = 10.0)	1,680	1,680	5.030	6.424
C4	Ridge(alpha = 100.0)	1,680	1,680	8.492	12.161

C1 (alpha = 0.1) has the lowest validation MAE and was selected. The error grows as the penalty increases because stronger shrinkage pulls the coefficients of the informative lag features away from their best values. The selected model and its preprocessing were then **frozen**: no refitting on validation or test data. Test predictions were made once (1,680 rows); for Region 1 at 22 September 00:00 the model predicts 439.849 kWh against an actual 435.590 kWh.

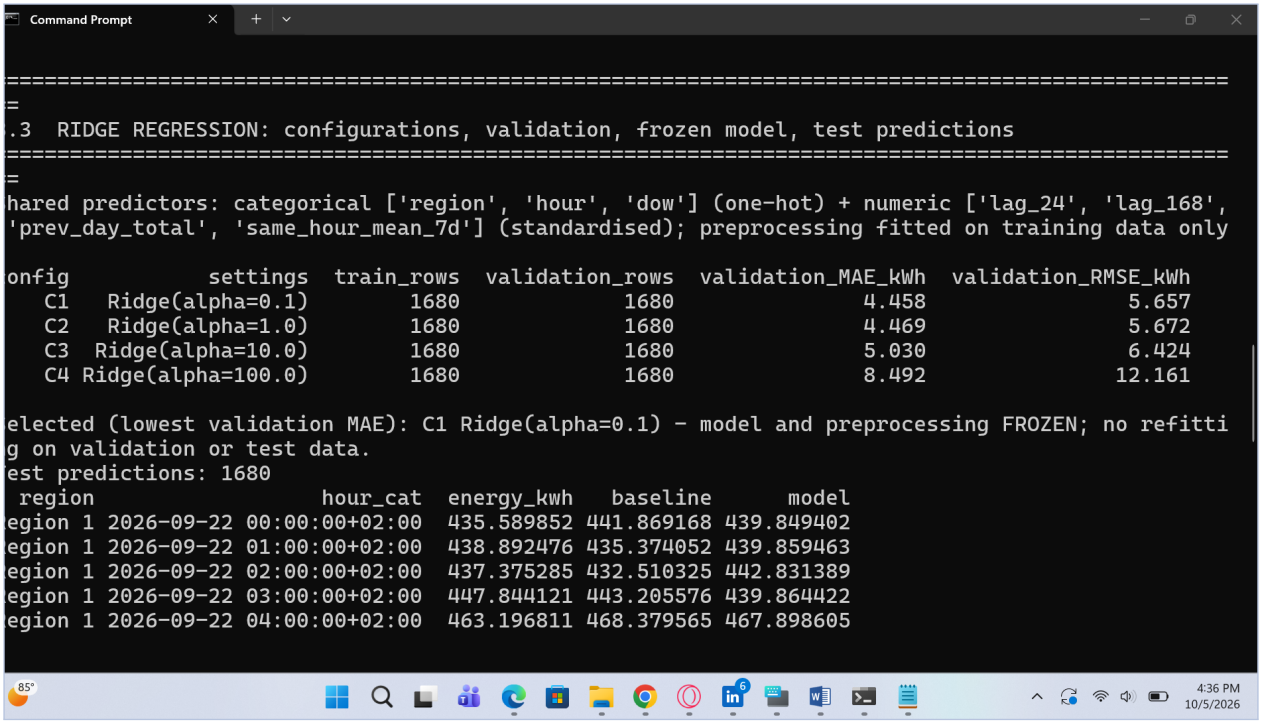


Figure 8.3-1: Ridge configurations C1 to C4 on validation, selection of C1 (frozen model) and the first test predictions

8.4 Is the forecast useful?

On all 1,680 test observations the model has **MAE 4.549 kWh and RMSE 5.795 kWh**, against **42.708 and 79.403 kWh** for the baseline: the MAE is **89.3 % lower**. The test error is close to the validation error (4.458 kWh), so the model generalises to the unseen week. It is better than the baseline in **all 10 regions**:

Region	Baseline MAE	Baseline RMSE	Model MAE	Model RMSE	Model better (MAE)
Region 1	44.873	83.696	4.645	5.808	True
Region 2	41.629	76.811	4.486	5.598	True
Region 3	42.754	77.607	4.279	5.607	True
Region 4	34.856	64.233	3.967	5.072	True
Region 5	44.400	81.084	5.221	6.655	True
Region 6	39.426	74.766	3.729	4.666	True
Region 7	39.688	73.945	4.464	5.795	True
Region 8	55.967	102.057	5.838	7.231	True
Region 9	43.772	80.178	4.698	5.814	True
Region 10	39.717	74.282	4.162	5.279	True

The model MAE is between **3.729 kWh (Region 6) and 5.838 kWh (Region 8)**; the baseline MAE between 34.856 kWh (Region 4) and 55.967 kWh (Region 8). Region 8 has the largest energy (Section 5.2), so it has the largest absolute errors in both methods.

The peak hour of each region

A forecast is most valuable at the highest demand, because that is what determines capacity. The table compares the forecasts for the **highest actual test hour of each region** (ties broken by the earliest time):

Region	Peak hour (CAT)	Actual (kWh)	Baseline (kWh)	Baseline abs. error	Model (kWh)	Model abs. error
Region 1	Mon 28 Sep 13:00	835.609	565.270	270.339	827.472	8.137
Region 2	Mon 28 Sep 13:00	780.126	546.913	233.213	774.220	5.905
Region 3	Mon 28 Sep 13:00	783.001	554.685	228.316	790.542	7.541
Region 4	Mon 28 Sep 18:00	659.903	561.406	98.497	657.030	2.873
Region 5	Mon 28 Sep 13:00	828.232	577.357	250.875	826.321	1.911
Region 6	Mon 28 Sep 13:00	749.112	504.408	244.704	731.613	17.499
Region 7	Fri 25 Sep 13:00	751.049	736.639	14.411	744.917	6.133
Region 8	Fri 25 Sep 13:00	1,016.555	1,010.534	6.021	1,014.815	1.740
Region 9	Wed 23 Sep 13:00	808.960	796.886	12.073	805.341	3.618
Region 10	Mon 28 Sep 13:00	743.035	502.327	240.707	735.070	7.965

For the seven regions whose peak fell on **Monday 28 September**, the baseline missed the peak by **98 to 270 kWh** (it copied the lower Sunday level), while the model erred by only **1.9 to 17.5 kWh**. For the three regions with a weekday peak (Regions 7, 8 and 9), the previous day was also a normal weekday, so the baseline was close (6 to 14 kWh) and the model still better (1.7 to 6.1 kWh).

The plot for Region 8 shows the mechanism. The baseline reproduces the previous day, so it **over-predicts Saturday 26 September** (it repeats the high Friday pattern, about 1,000 kWh at the peak, when the actual peak is about 720 kWh) and **under-predicts Monday 28 September** (it repeats the low Sunday level, about 750 kWh, when the actual peak is about 1,010 kWh). The model follows the actual curve on every day, because it knows the day of week (`dow`) and the same hour of the previous week (`lag_168`, `same_hour_mean_7d`).

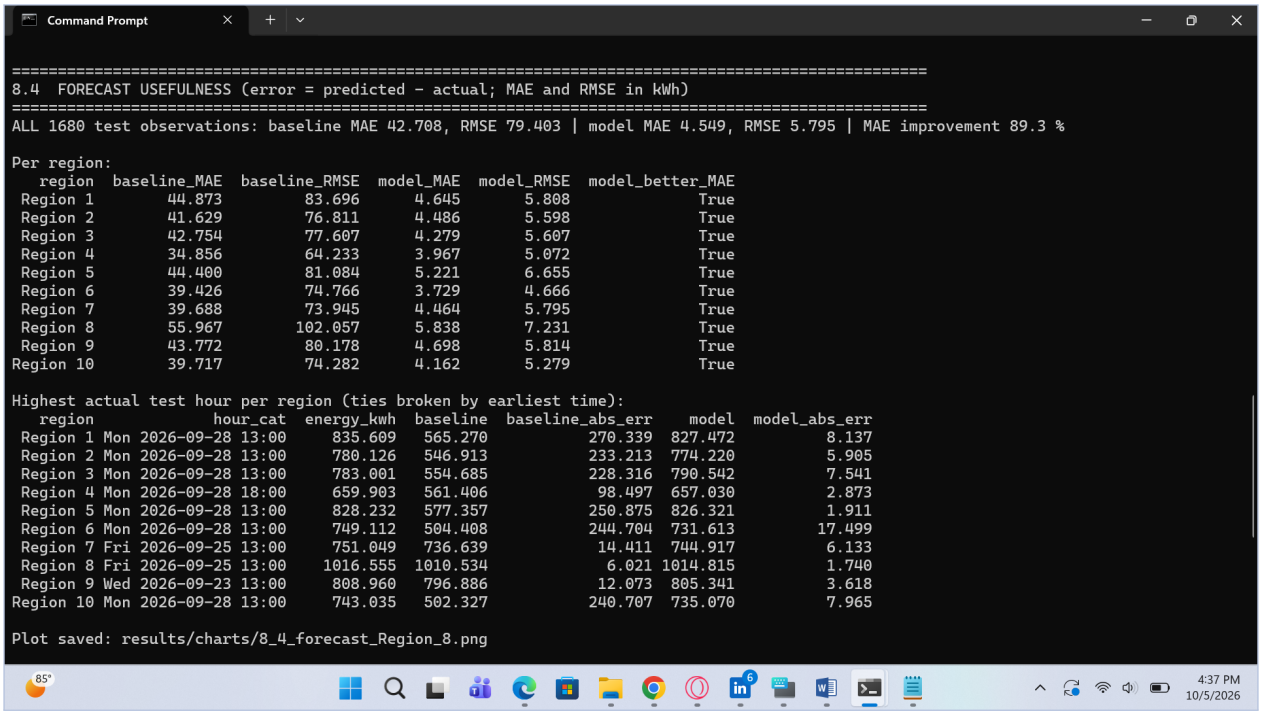


Figure 8.4-1: Test metrics for all observations and per region, and the highest actual test hour of each region with baseline and model errors

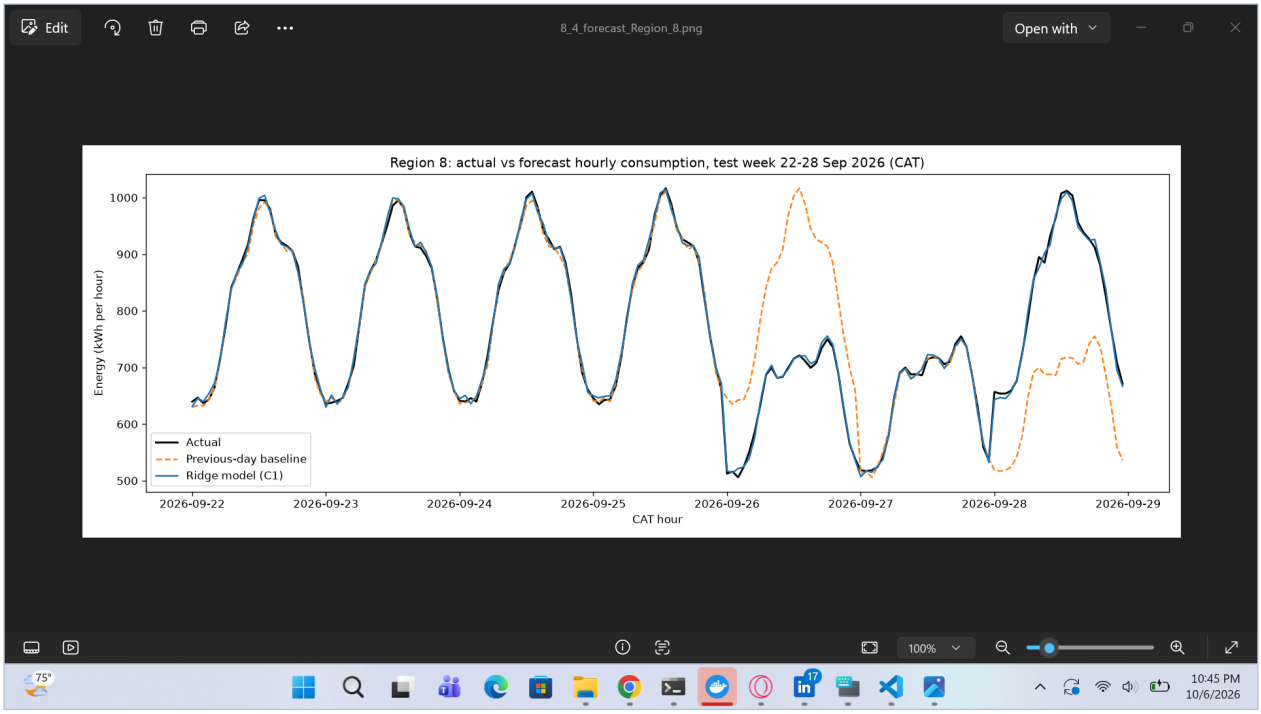


Figure 8.4-2: Region 8: actual, previous-day baseline and Ridge model (C1), test week 22 to 28 September 2026

Conclusion and limitations

Conclusion. The Ridge model is accurate enough for next-day operational planning: its error (about 4.5 kWh per region-hour, roughly 0.6 % of a typical 700 to 1,000 kWh peak hour) is small, it is better than the baseline in every region, and it is especially reliable where the baseline fails, at the change from weekend to weekday. It can be run shortly after midnight using data up to the end of the previous day, retrained weekly, and its error monitored against actuals.

Limitations.

- About **1 % of readings are missing** in each region-hour (1,583.9 of 1,600 on average), so the hourly sums are slightly low and the missing share varies from hour to hour.
- **Late arrivals:** at 00:00 about 0.1 % of the previous day's energy, mostly from 21:00 to 23:00, has not yet arrived (Section 5.3), so lags built at the forecast origin are slightly understated; the model was trained on final values.
- Only **one week of training data** was available (8 to 14 September after the lookback exclusion), and the validation week was used only to choose alpha; seasonality, holidays and weather could not be learned.
- The data are **synthetic**, with regular daily and weekly patterns, so real data would give larger errors and would need weather and calendar features.

Recommendation

Recommendation. Adopt the Ridge regression model for next-day hourly planning by region, keep the previous-day, same-hour baseline as a fallback, and put the following controls around it so that its accuracy stays visible and its limits are respected.

- **Schedule.** Run the forecast shortly after midnight, using data up to the end of the previous day, and give the operations team one forecast for each region and hour.
- **Retrain weekly** on the most recent complete weeks, so that the model follows the weekly growth of about 2.1 % (Section 5.1), and choose alpha again on the latest validation week.

- **Monitor the error daily.** When a day is complete, compare the forecast with the actual energy and report MAE by region. Review the model when the error is clearly above the test value of 4.549 kWh (for example, twice as large), or when it is no longer better than the baseline in a region.
- **Treat missing and late readings openly.** Keep missing readings as gaps, show the completeness next to each forecast, and train on the values that were available at the forecast time, so that training matches the way the model is used at 00:00 (Section 5.3).
- **Use it for ordinary days until more data exist.** Add weather and calendar features (holidays and special events) and several months of real readings before relying on the forecast for exceptional days; until then, operators should apply their own judgement on those days.

9 Submit the report and code

9.1 Organise the report

This report is submitted as **one PDF per group**. Its sections follow the subsections of the assignment in order (1.1 to 9.3). Each required screenshot is embedded beside the subsection that asks for it and shows the executed command or action, its result, the taskbar and the clock. Where a later subsection asks only for interpretation (for example 3.3, 4.4 and 5.5), it refers to the evidence of the earlier measurement subsections instead of repeating it. The report contains the measurements, charts, observations, judgements and the management recommendation (5.5).

9.2 Make the work reproducible

Generator unchanged

The supplied `smart_meter_simulator.py` is used unchanged. Its SHA-256 hash was computed for the copy in the project (`src\smart_meter_simulator.py`) and for the issued copy in the Downloads folder with `certutil -hashfile ... SHA256`. Both hashes are identical, **6b16d6bb3191e528844d0ec75656096d8bfb2e92c99f83e2b75d586bf21cfff8**, so the project copy matches the issued file byte for byte.

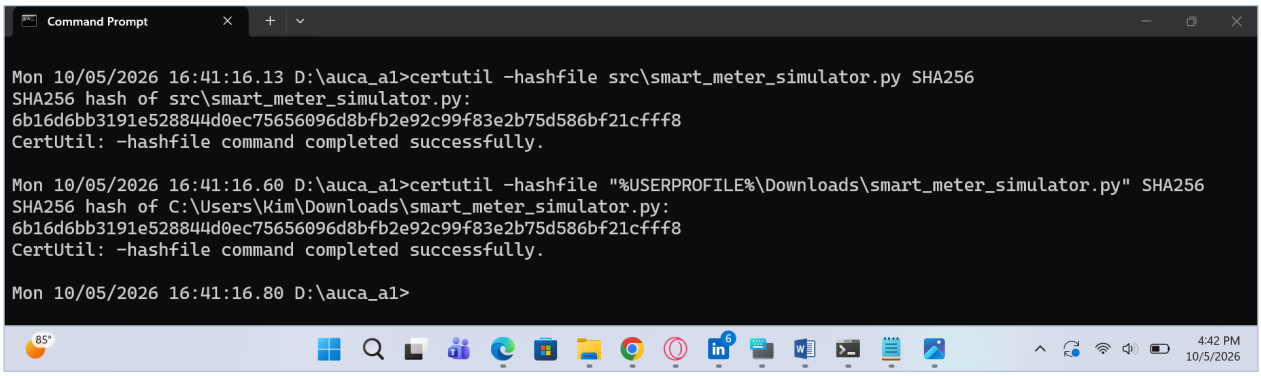


Figure 9.2-1: SHA-256 of the project copy and of the issued copy of smart_meter_simulator.py: identical

Repository and ZIP

GitHub repository: `https://github.com/erneste-uwimpuhwe/Advanced-Big-Data-Analytics-Assignment-I-Smart-Energy-Grid-Analytics`

The submission ZIP, `auca_assignment1_submission.zip`, is created with `git archive -o auca_assignment1_submission.zip HEAD` and contains exactly the files of the repository.

The code was pushed to the `main` branch in commit `68c7bd7`; this report (`101414_101420_101366_Assignment_I_Report.pdf`, in the root of the repository) and all screenshots used in it (the folder `screenshots/`) were added in later commits. The generated dataset (`data/`), the virtual environment (`venv/`) and the installer files are excluded by `.gitignore`.

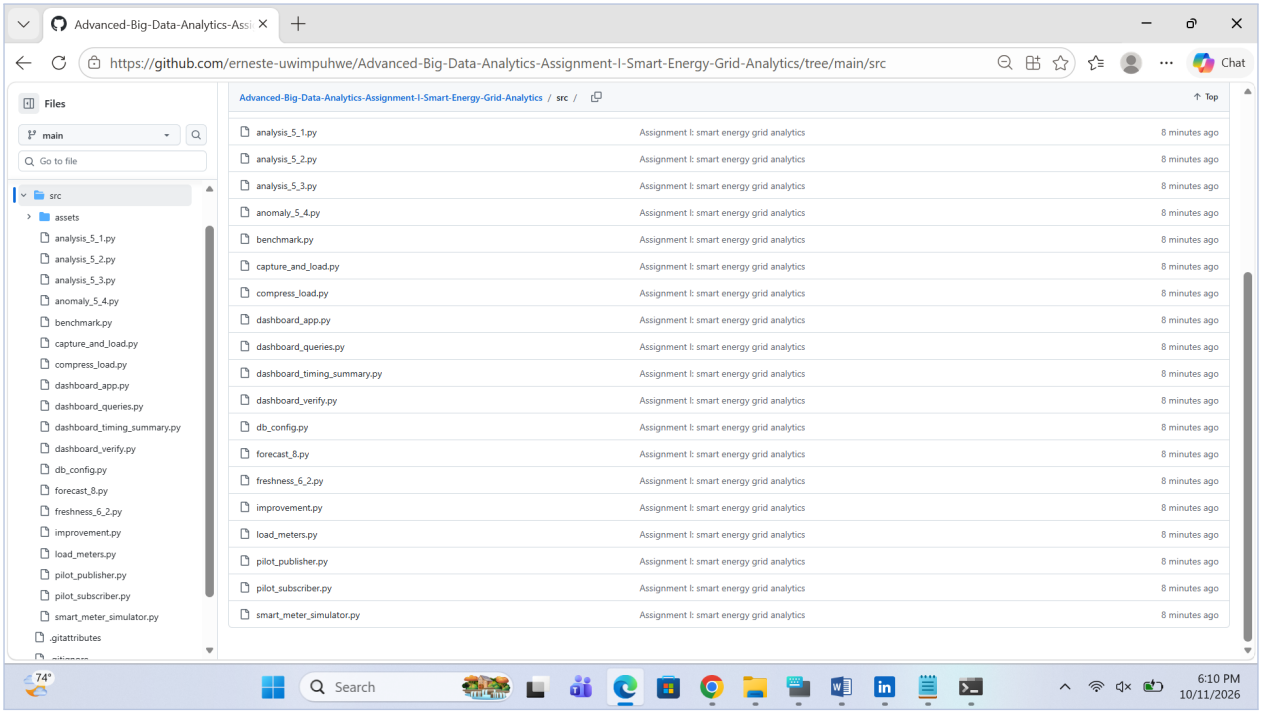


Figure 9.2-2: GitHub repository, folder src: the Python programs, including the unchanged generator smart_meter_simulator.py (commit 68c7bd7, branch main)

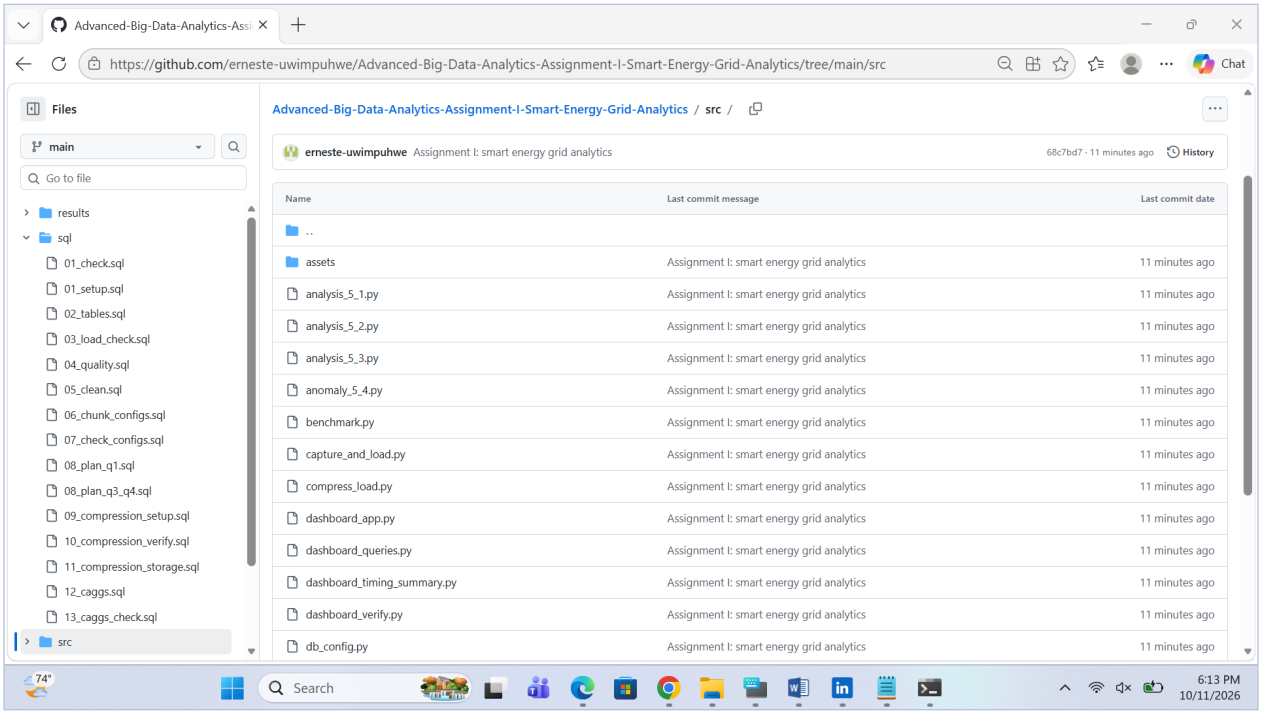


Figure 9.2-3: GitHub repository file tree: the 15 SQL scripts in the sql folder, next to the results folder and the src folder

The repository and the ZIP contain the unchanged generator, the capture and ingestion programs, the SQL files, the analysis and model code, the recorded benchmark timings and results, the dashboard queries and configuration, and the recorded environment details. The files referred to in this report are:

Section	Files
All sections (shared)	`src/db_config.py` (connection settings; the password is read from the `PGPASSWORD` environment variable, with the placeholder `your_password` as default)

Section	Files
1 Ingestion pipeline	sql/01_setup.sql, sql/01_check.sql, sql/02_tables.sql, src/load_meters.py, src/pilot_publisher.py, src/pilot_subscriber.py
2 Capture, quality, cleaning	src/capture_and_load.py, sql/03_load_check.sql, sql/04_quality.sql, sql/05_clean.sql
3 Chunk intervals	sql/06_chunk_configs.sql, sql/07_check_configs.sql, src/benchmark.py, sql/08_plan_q1.sql, sql/08_plan_q3_q4.sql
4 Compression	sql/09_compression_setup.sql, src/compress_load.py, sql/10_compression_verify.sql, sql/11_compression_storage.sql, src/benchmark.py, src/improvement.py
5 Analysis	src/analysis_5_1.py, src/analysis_5_2.py, src/analysis_5_3.py, src/anomaly_5_4.py
6 Continuous aggregates	sql/12_caggs.sql, sql/13_caggs_check.sql, src/freshness_6_2.py
7 Dashboards	src/dashboard_app.py, src/dashboard_queries.py, src/dashboard_verify.py, src/dashboard_timing_summary.py, src/assets/style.css
8 Forecast	src/forecast_8.py
Results	results/ (benchmark runs and summaries, load and compression timings, pilot logs, anomaly evaluation, dashboard timings and exports, forecast outputs) and results/charts/

The full generated dataset (10,696,848 rows) is **not stored** in the repository. src/capture_and_load.py regenerates it from the unchanged generator into data\energy_readings_full.csv and loads it into an empty energy_readings; the cleaning and later steps are then run in the order given in the README. Passwords and connection secrets appear only as placeholders (your_password).

README: execution order

The README.md in the repository gives the setup, the pilot check and the execution order below. Every command is run from the project folder in a Command Prompt after activating the virtual environment and setting the placeholders PATH and PGPASSWORD (set PGPASSWORD=your_password).

Step	Command (summary)	Section
Setup	python -m venv venv, pip install -r requirements.txt; psql -U postgres -f sql\01_setup.sql (tablespace d_space, database smart_grid, time zone Africa/Kigali, TimescaleDB extension) and sql\01_check.sql; docker start emqx	1.1
1	psql ... -f sql\02_tables.sql	1.2
2	python src\load_meters.py	1.2
3	Pilot: python src\pilot_subscriber.py (wait for READY), then python src\pilot_publisher.py	1.3
4	python src\capture_and_load.py (regenerates the CSV and loads it), then sql\03_load_check.sql	2.1
5 to 6	sql\04_quality.sql, then sql\05_clean.sql	2.2, 2.3
7 to 9	sql\06_chunk_configs.sql, sql\07_check_configs.sql; python src\benchmark.py --label section3 --tables energy_readings energy_readings_3h energy_readings_week; sql\08_plan_q1.sql, sql\08_plan_q3_q4.sql	3.1 to 3.3
10 to 13	sql\09_compression_setup.sql; python src\compress_load.py; sql\10_compression_verify.sql, sql\11_compression_storage.sql; python src\benchmark.py --label section4 --tables energy_readings energy_time_compressed energy_meter_compressed; python src\improvement.py	4.1 to 4.3
14 to 15	python src\analysis_5_1.py, analysis_5_2.py, analysis_5_3.py; python src\anomaly_5_4.py	5.1 to 5.4

Step	Command (summary)	Section
16 to 17	`sql\12_caggs.sql`, `sql\13_caggs_check.sql`; `python src\freshness_6_2.py`	6.1, 6.2
18 to 20	`python src\dashboard_app.py` (pages `/raw` and `/aggregate`); `python src\dashboard_verify.py`; run the benchmark button, then `python src\dashboard_timing_summary.py`	7.1 to 7.4
21	`python src\forecast_8.py`	8.1 to 8.4

Pilot re-run from the README instructions

The pilot was run again by following the README instructions on **6 October 2026 between 23:04 and 23:06 CAT**: EMQX was started, `energy_live` was truncated and shown to contain 0 rows, the subscriber was started and reached READY, and then the publisher was run. All four counts from Section 1.3 equal the target:

Stage	Count
Dictionaries taken from the generator	**2,000** (1,994 unique meter and event_time pairs plus 6 deliberate duplicates)
Messages acknowledged by the broker (QoS 1)	**2,000** (2,000 PUBACK callbacks, 0 publish errors)
Messages received by the subscriber	**2,000** (0 redelivered, 0 topic mismatches, 0 parse errors, 0 insert errors)
Rows in energy_live after all inserts were committed	**2,000** (1,994 unique pairs)

Earlier attempt on the same evening. A first rerun at 22:53 CAT stored only **728** of the 2,000 messages. The broker log shows why: at the same moment EMQX disconnected the client `auca-pilot-subscriber` with `reason => mailbox_overflow` (1,064 queued messages against a limit of 1,000), because the publisher sent all 2,000 messages in about one second while the machine was busy with other programs, including the dashboard server. The subscriber reconnected two seconds later as a new session (`session.resumed: 0`), so the messages queued for the old session were discarded. The broker metrics agree: 2,000 messages published, 728 delivered, 76 dropped because no subscriber was connected, and no `queue_full` drops. The subscriber log showed no parse errors, insert errors or redelivery, and the database held exactly the 728 delivered messages, so nothing was lost inside the subscriber or the database. After the other programs were closed and EMQX was restarted, the pilot was repeated with an empty `energy_live` and all four counts reached 2,000, with no change to the programs. The fault depends on load; as hardening, a persistent MQTT session and a publisher that waits for acknowledgements in small batches would make the pilot robust on a busy machine.

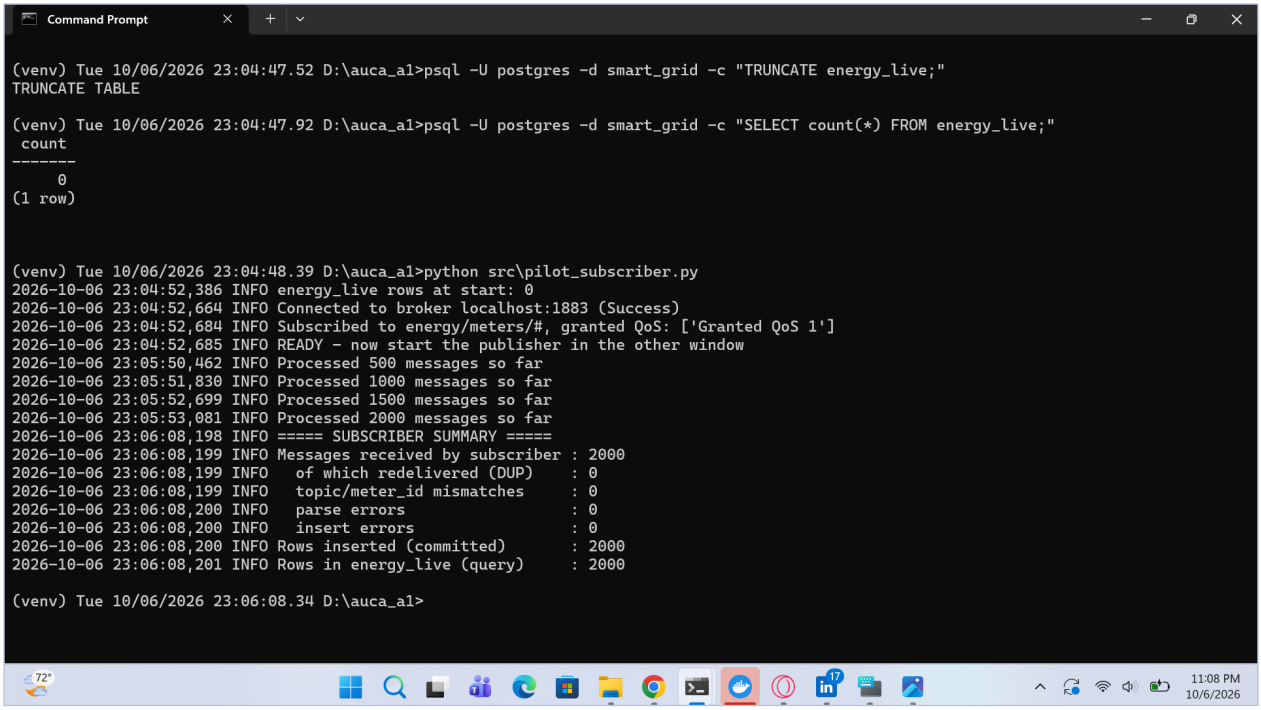


Figure 9.2-4: README pilot re-run, subscriber: energy_live truncated and empty, subscriber READY, summary with 2,000 received and 2,000 rows

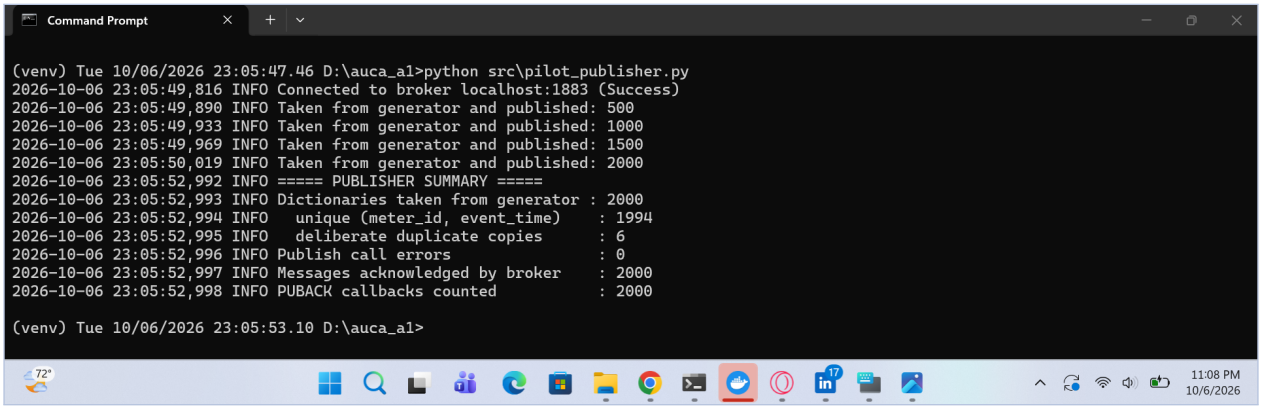


Figure 9.2-5: README pilot re-run, publisher: 2,000 dictionaries taken, 2,000 acknowledged by the broker

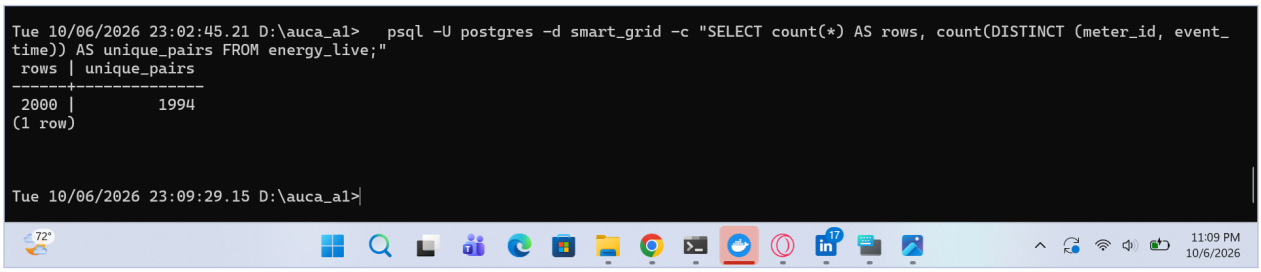


Figure 9.2-6: Database check after the re-run: 2,000 rows and 1,994 unique meter and event_time pairs in energy_live

9.3 Document contributions

The work was divided by section as listed below. The management recommendation (5.5), the interpretation of results and the final review of the report were done together, and every member can explain and present any part of the work if called upon.

No.	Name	Student ID	Individual contribution
1	Patrick Niyonshuti	101420	**Sections 1, 2 and 7.** Environment record and database setup; table and hypertable creation and the meters loader (`load_meters.py`); the MQTT pilot, its fault investigation and fix (`pilot_publisher.py`, `pilot_subscriber.py`); full capture and bulk load (`capture_and_load.py`); data-quality diagnostics and cleaning (`sql/04_quality.sql`, `sql/05_clean.sql`); the Raw and Aggregate dashboards, their verification and the refresh benchmark (`dashboard_app.py`, `dashboard_queries.py`, `dashboard_verify.py`, `dashboard_timing_summary.py`).
2	Erneste Uwimpuhwe	101414	**Sections 3, 4 and 6.** The three chunk-interval hypertables and the five-workload benchmark (`sql/06_chunk_configs.sql`, `benchmark.py`, `sql/08_plan_q1.sql`); the two compressed configurations, storage and compression measurements and query comparison (`sql/09_compression_setup.sql`, `compress_load.py`, `improvement.py`); the three continuous aggregates, refresh policies and the freshness test (`sql/12_caggs.sql`, `sql/13_caggs_check.sql`, `freshness_6_2.py`).
3	Gisele Uwibereyeho	101366	**Sections 5, 8 and 9.** Demand, regional, reporting-quality and anomaly analysis (`analysis_5_1.py`, `analysis_5_2.py`, `analysis_5_3.py`, `anomaly_5_4.py`); the next-day forecast with baseline, Ridge model and evaluation (`forecast_8.py`); the README, GitHub repository, ZIP and final assembly and checking of the report.